

CS3520
Computer Organisation and Architecture I
Section 5 – Computer Memory System

Mr K.O, Setetemela
BJO 308
Department of Mathematics & Computer Science
National University of Lesotho
w: 59105441 | e: khobatha.setetemela@gmail.com

Principle of Locality

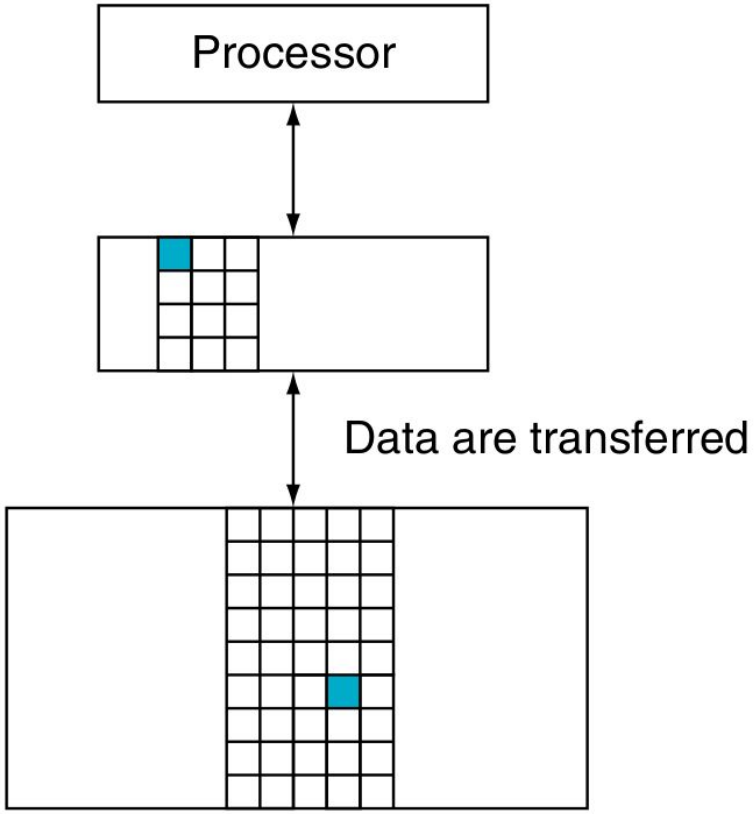
- Programs access a small proportion of their address space at any time
- Temporal locality
  - Items accessed recently are likely to be accessed again soon
  - e.g., instructions in a loop, induction variables
- Spatial locality
  - Items near those accessed recently are likely to be accessed soon
  - E.g., sequential instruction access, array data

Taking Advantage of Locality

- Memory hierarchy
- Store everything on disk
- Copy recently accessed (and nearby) items from disk to smaller DRAM memory
  - Main memory
- Copy more recently accessed (and nearby) items from DRAM to smaller SRAM memory
  - Cache memory attached to CPU

Taking Advantage of Locality

- Block (aka line): unit of copying
  - May be multiple words
- If accessed data is present in upper level
  - Hit: access satisfied by upper level
    - Hit ratio: hits/accesses
- If accessed data is absent
  - Miss: block copied from lower level
    - Time taken: miss penalty
    - Miss ratio: misses/accesses = 1 – hit ratio
  - Then accessed data supplied from upper level

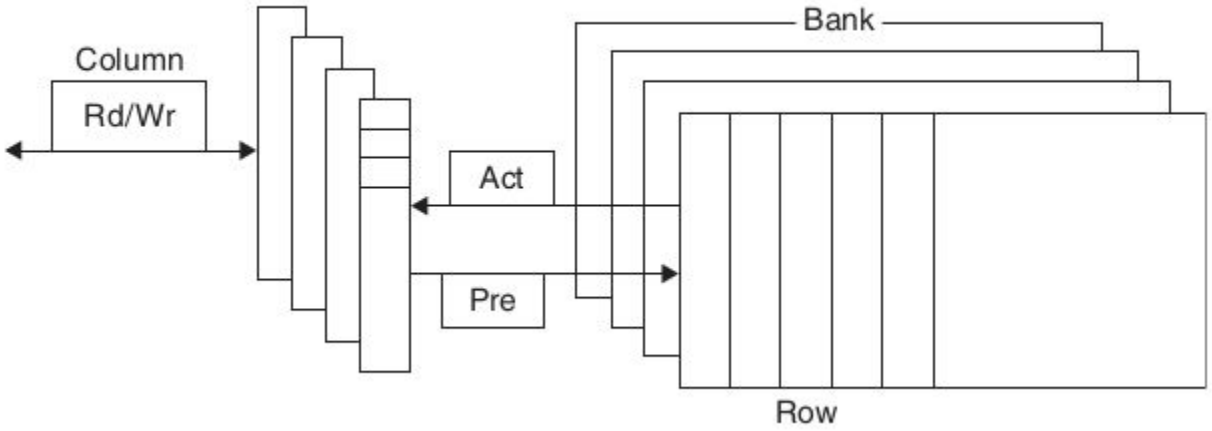


Memory Technology

- Static RAM (SRAM)
  - 0.5ns – 2.5ns, $2000 – $5000 per GB
- Dynamic RAM (DRAM)
  - 50ns – 70ns, $20 – $75 per GB
- Magnetic disk
  - 5ms – 20ms, $0.20 – $2 per GB
- Ideal memory
  - Access time of SRAM
  - Capacity and cost/GB of disk

DRAM Technology

- Data stored as a charge in a capacitor
  - Single transistor used to access the charge
  - Must periodically be refreshed
    - Read contents and write back
    - Performed on a DRAM “row”



Advanced DRAM Organization

- Bits in a DRAM are organized as a rectangular array
  - DRAM accesses an entire row
  - Burst mode: supply successive words from a row with reduced latency
- Double data rate (DDR) DRAM
  - Transfer on rising and falling clock edges
- Quad data rate (QDR) DRAM
  - Separate DDR inputs and outputs

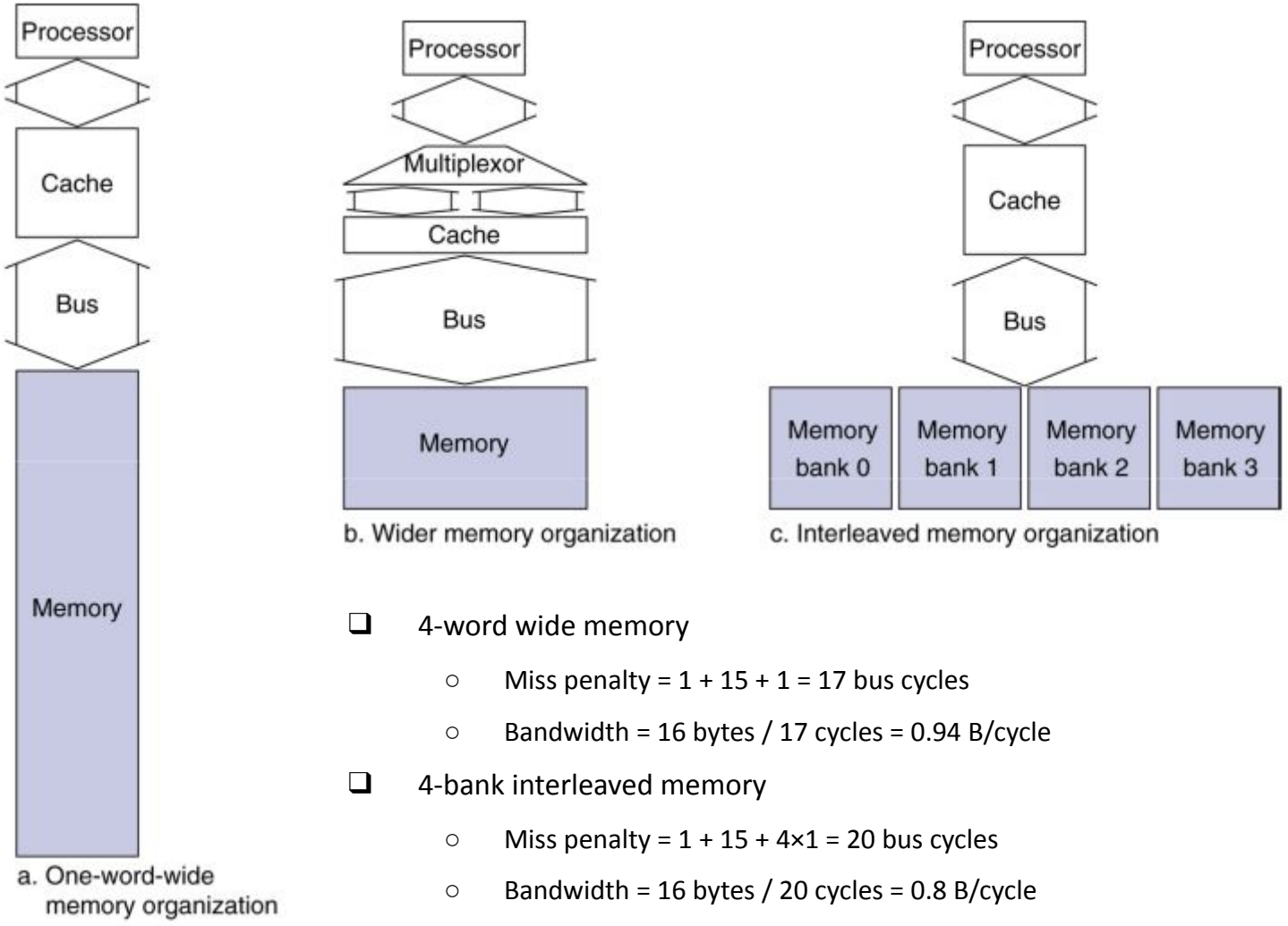
DRAM Generations

Year introduced	Chip size	$ per GiB	Total access time to a new row/column	Average column access time to existing row
1980	64 Kibibit	$6,480,000	250 ns	150 ns
1983	256 Kibibit	$1,980,000	185 ns	100 ns
1985	1 Mebibit	$720,000	135 ns	40 ns
1989	4 Mebibit	$128,000	110 ns	40 ns
1992	16 Mebibit	$30,000	90 ns	30 ns
1996	64 Mebibit	$9,000	60 ns	12 ns
1998	128 Mebibit	$900	60 ns	10 ns
2000	256 Mebibit	$840	55 ns	7 ns
2004	512 Mebibit	$150	50 ns	5 ns
2007	1 Gibibit	$40	45 ns	1.25 ns
2010	2 Gibibit	$13	40 ns	1 ns
2012	4 Gibibit	$5	35 ns	0.8 ns
2015	8 Gibibit	$7	30 ns	0.6 ns
2018	16 Gibibit	$6	25 ns	0.4 ns

DRAM Performance Factors

- Row buffer
  - Allows several words to be read and refreshed in parallel
- Synchronous DRAM
  - Allows for consecutive accesses in bursts without needing to send each address
  - Improves bandwidth
- DRAM banking
  - Allows simultaneous access to multiple DRAMs
  - Improves bandwidth

Increasing Memory Bandwidth



- 4-word wide memory
  - Miss penalty = 1 + 15 + 1 = 17 bus cycles
  - Bandwidth = 16 bytes / 17 cycles = 0.94 B/cycle
- 4-bank interleaved memory
  - Miss penalty = 1 + 15 + 4×1 = 20 bus cycles
  - Bandwidth = 16 bytes / 20 cycles = 0.8 B/cycle

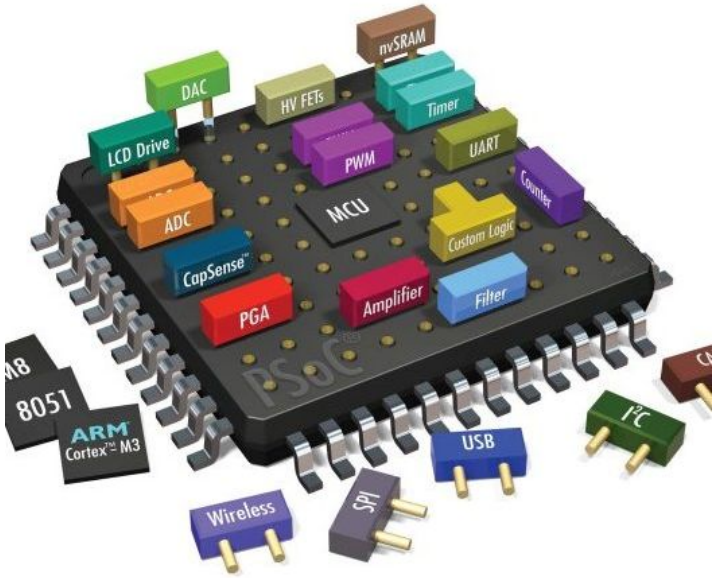
Flash Storage

- Nonvolatile semiconductor storage
  - 100× – 1000× faster than disk
  - Smaller, lower power, more robust
  - But more $/GB (between disk and DRAM)



Flash Types

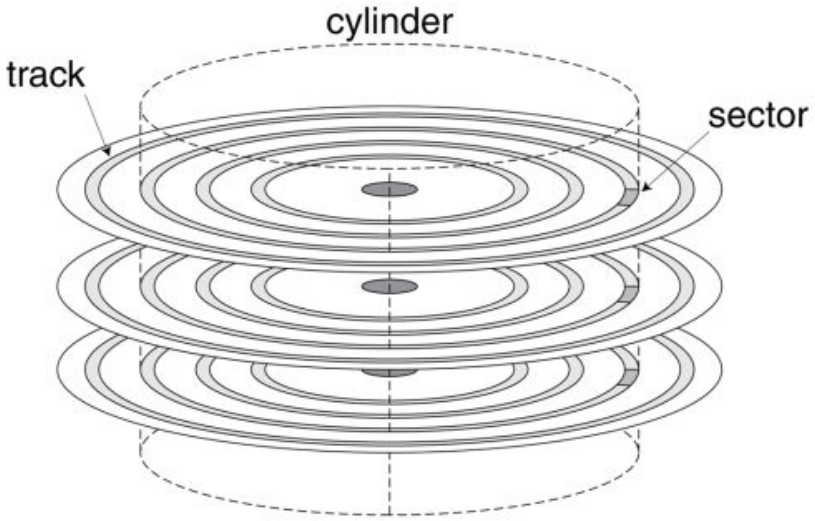
- **NOR flash: bit cell like a NOR gate**
  - Random read/write access
  - Used for instruction memory in embedded systems
- **NAND flash: bit cell like a NAND gate**
  - Denser (bits/area), but block-at-a-time access
  - Cheaper per GB
  - Used for USB keys, media storage, …
- **Flash bits wears out after 1000's of accesses**
  - Not suitable for direct RAM or disk replacement
  - **Wear leveling:** remap data to less used blocks



Disk Storage

- Nonvolatile, rotating magnetic storage





Flash Types

- **Each sector records**
  - Sector ID
  - Data (512 bytes, 4096 bytes proposed)
  - Error correcting code (ECC)
    - Used to hide defects and recording errors
  - Synchronization fields and gaps
- **Access to a sector involves**
  - Queuing delay if other accesses are pending
  - Seek: move the heads
  - Rotational latency
  - Data transfer
  - Controller overhead

Disk Access Example

- **Given**
  - 512B sector, 15,000rpm, 4ms average seek time, 100MB/s transfer rate, 0.2ms controller overhead, idle disk
- **Average read time**
  - 4ms seek time

    + 0.5 rotation / (15,000rpm/60sec/min) = 2ms rotational latency

    + 512B / 100MB/s = 0.005ms transfer time

    + 0.2ms controller delay

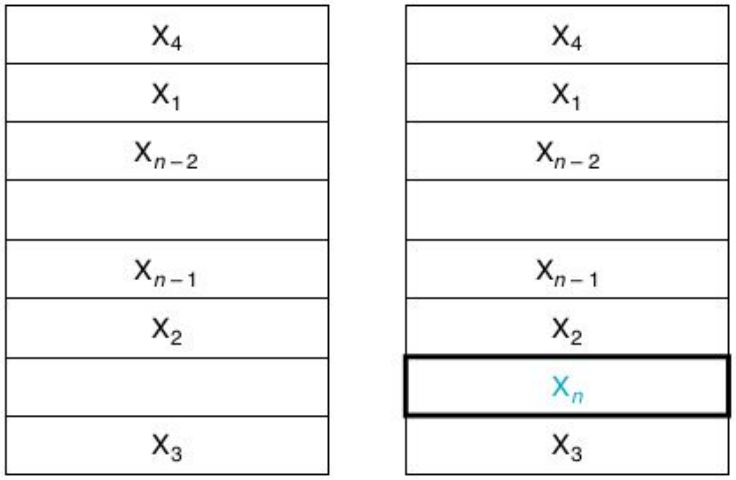
    = 6.2ms
- **If actual average seek time is 1ms**
  - Average read time = 3.2ms

Disk Performance Issues

- **Manufacturers quote average seek time**
  - Based on all possible seeks
  - Locality and OS scheduling lead to smaller actual average seek times
- **Smart disk controller allocate physical sectors on disk**
  - Present logical sector interface to host
  - SCSI, ATA, SATA
- **Disk drives include caches**
  - Prefetch sectors in anticipation of access
  - Avoid seek and rotational delay

Cache Memory

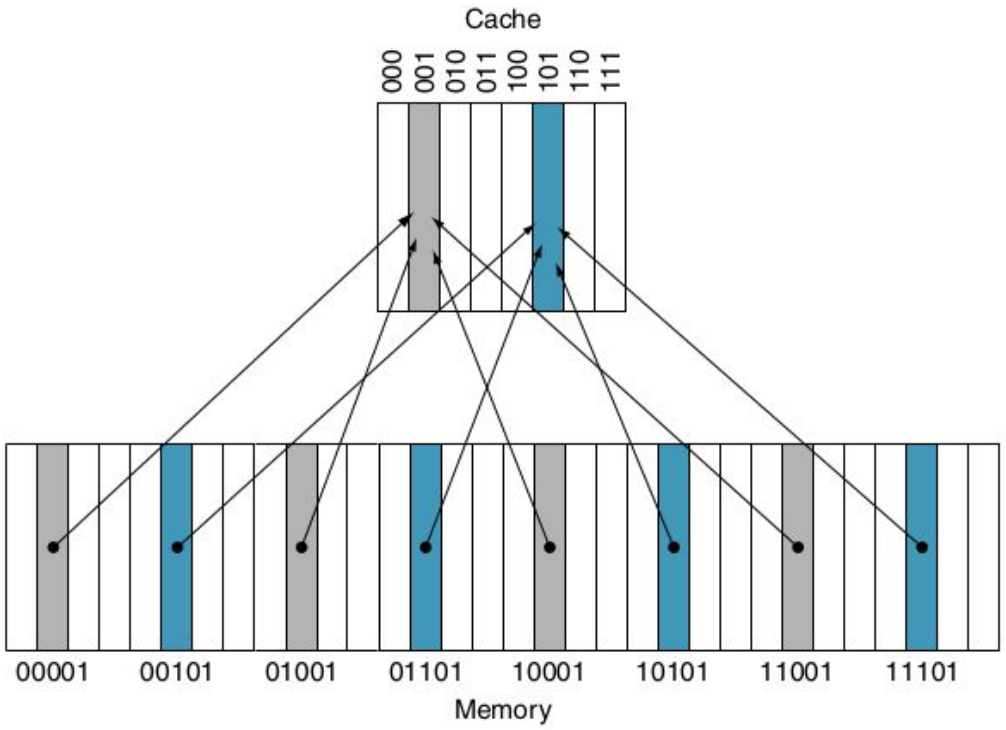
- Cache memory
  - The level of the memory hierarchy closest to the CPU
- Given accesses X1, …, Xn–1, Xn
  - How do we know if the data is present?
  - Where do we look?



a. Before the reference to X_n b. After the reference to X_n

Direct Mapped Cache

- Location determined by address
- **Direct mapped:** only one choice
  - (Block address) modulo (#Blocks in cache)
  - #Blocks is a power of 2
  - Use low-order address bits



Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
  - Store block address as well as the data
  - Actually, only need the high-order bits
  - Called the tag
- What if there is no data in a location?
  - Valid bit: 1 = present, 0 = not present
  - Initially 0

Cache Example

- 8-blocks, 1 word/block, direct mapped
- Initial state

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Miss	110

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
26	11 010	Miss	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Hit	110
26	11 010	Hit	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

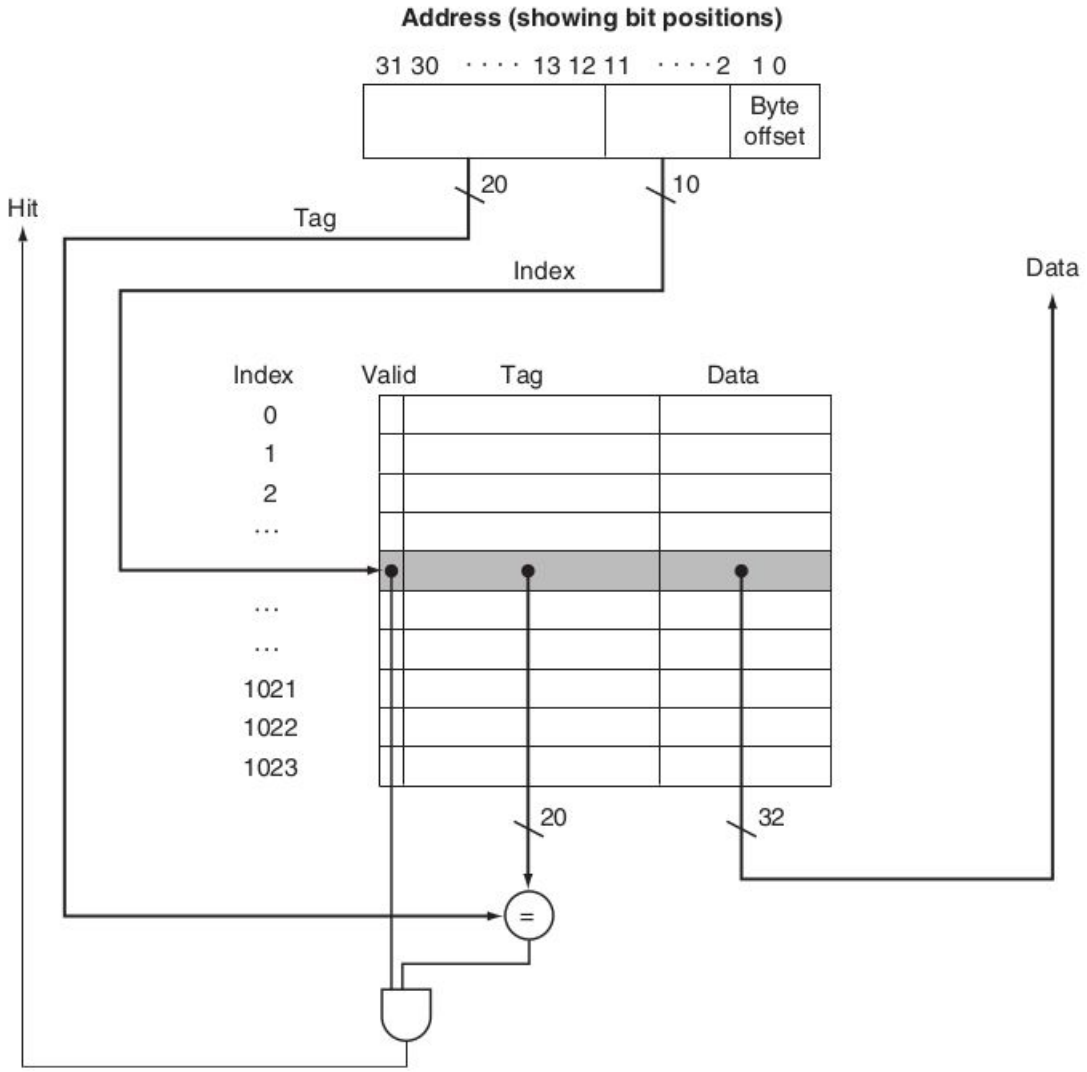
Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	11	Mem[11010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
18	10 010	Miss	010

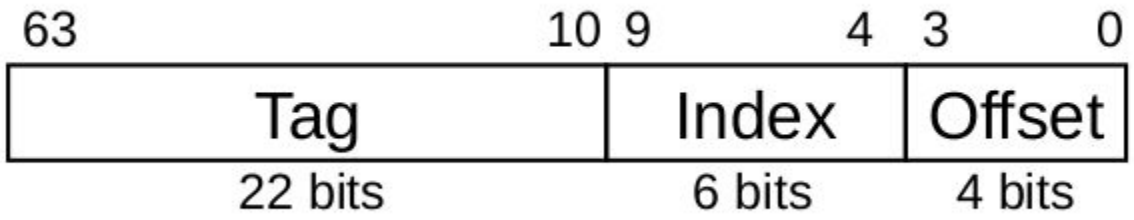
Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	**Y**	**10**	**Mem[10010]**
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Address Subdivision



Example: Larger Block Size

- 64 blocks, 16 bytes/block
  - To what block number does address 1200 map?
- Block address = 1200/16 = 75
- Block number = 75 modulo 64 = 11



Block Size Considerations

- **Larger blocks should reduce miss rate**
  - Due to spatial locality
- **But in a fixed-sized cache**
  - Larger blocks ⇒ fewer of them
    - More competition ⇒ increased miss rate
  - Larger blocks ⇒ pollution
- **Larger miss penalty**
  - Can override benefit of reduced miss rate
  - Early restart and critical-word-first can help

Cache Misses

- On cache hit, CPU proceeds normally
- On cache miss
  - Stall the CPU pipeline
  - Fetch block from next level of hierarchy
  - Instruction cache miss
    - Restart instruction fetch
  - Data cache miss
    - Complete data access

Write-Through

- On data-write hit, could just update the block in cache
  - But then cache and memory would be inconsistent
- Write through: also update memory
- But makes writes take longer
  - e.g., if base CPI = 1, 10% of instructions are stores, write to memory takes 100 cycles
    - Effective CPI = 1 + 0.1×100 = 11
- Solution: write buffer
  - Holds data waiting to be written to memory
  - CPU continues immediately
    - Only stalls on write if write buffer is already full

Write-Back

- Alternative: On data-write hit, just update the block in cache
  - Keep track of whether each block is dirty
- When a dirty block is replaced
  - Write it back to memory
  - Can use a write buffer to allow replacing block to be read first

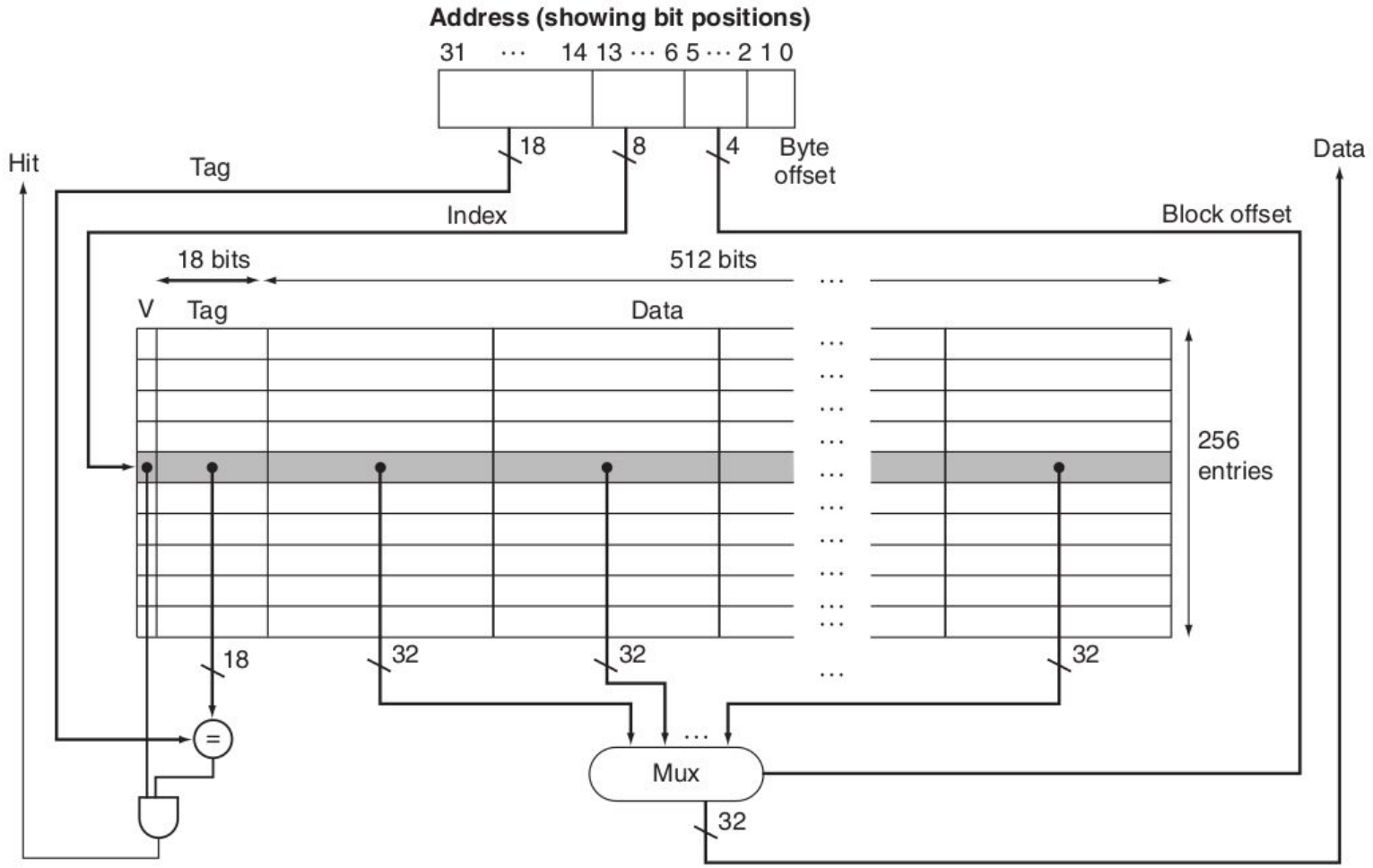
Write Allocation

- What should happen on a write miss?
- Alternatives for write-through
  - Allocate on miss: fetch the block
  - Write around: don't fetch the block
    - Since programs often write a whole block before reading it (e.g., initialization)
- For write-back
  - Usually fetch the block

Example: Intrinsity FastMATH

- Embedded MIPS processor
  - 12-stage pipeline
  - Instruction and data access on each cycle
- Split cache: separate I-cache and D-cache
  - Each 16KB: 256 blocks × 16 words/block
  - D-cache: write-through or write-back
- SPEC2000 miss rates
  - I-cache: 0.4%
  - D-cache: 11.4%
  - Weighted average: 3.2%

Example: Intrinsity FastMATH



Main Memory Supporting Caches

- Use DRAMs for main memory
  - Fixed width (e.g., 1 word)
  - Connected by fixed-width clocked bus
    - Bus clock is typically slower than CPU clock
- Example cache block read
  - 1 bus cycle for address transfer
  - 15 bus cycles per DRAM access
  - 1 bus cycle per data transfer
- For 4-word block, 1-word-wide DRAM
  - Miss penalty = 1 + 4×15 + 4×1 = 65 bus cycles
  - Bandwidth = 16 bytes / 65 cycles = 0.25 B/cycle

Main Memory Supporting Caches

- Components of CPU time
  - Program execution cycles
    - Memory stall cycles
  - Includes cache hit time
    - Mainly from cache misses
- With simplifying assumptions:

Cache Performance Example

- Given
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty = 100 cycles
  - Base CPI (ideal cache) = 2
  - Load & stores are 36% of instructions
- Miss cycles per instruction
  - I-cache: 0.02 × 100 = 2
  - D-cache: 0.36 × 0.04 × 100 = 1.44
- Actual CPI = 2 + 2 + 1.44 = 5.44
  - Ideal CPU is 5.44/2 =2.72 times faster

Average Access Time

- Hit time is also important for performance
- Average memory access time (AMAT)
  - AMAT = Hit time + Miss rate × Miss penalty
- Example
  - CPU with 1ns clock, hit time = 1 cycle, miss penalty = 20 cycles, I-cache miss rate = 5%
  - AMAT = 1 + 0.05 × 20 = 2ns
    - 2 cycles per instruction

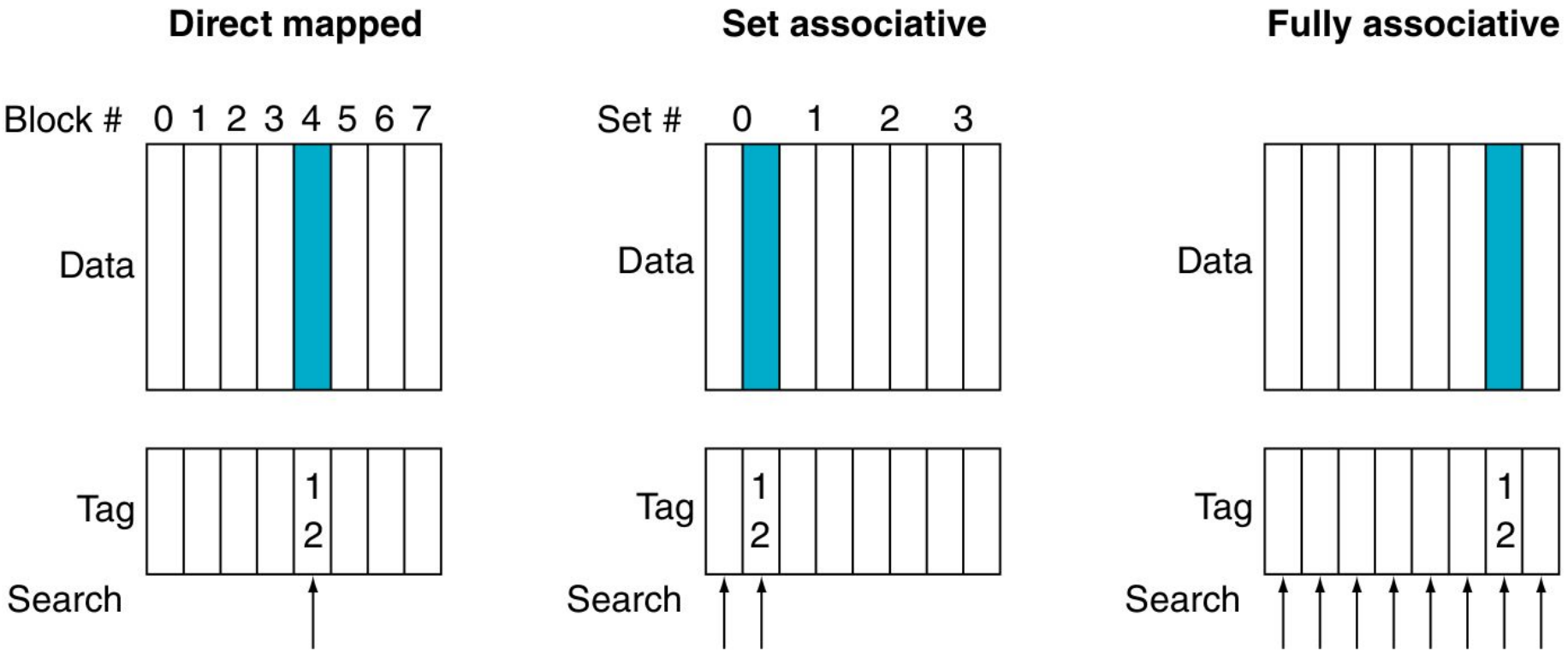
Performance Summary

- When CPU performance increased
  - Miss penalty becomes more significant
- Decreasing base CPI
  - Greater proportion of time spent on memory stalls
- Increasing clock rate
  - Memory stalls account for more CPU cycles
- Can’t neglect cache behavior when evaluating system performance

Associative Caches

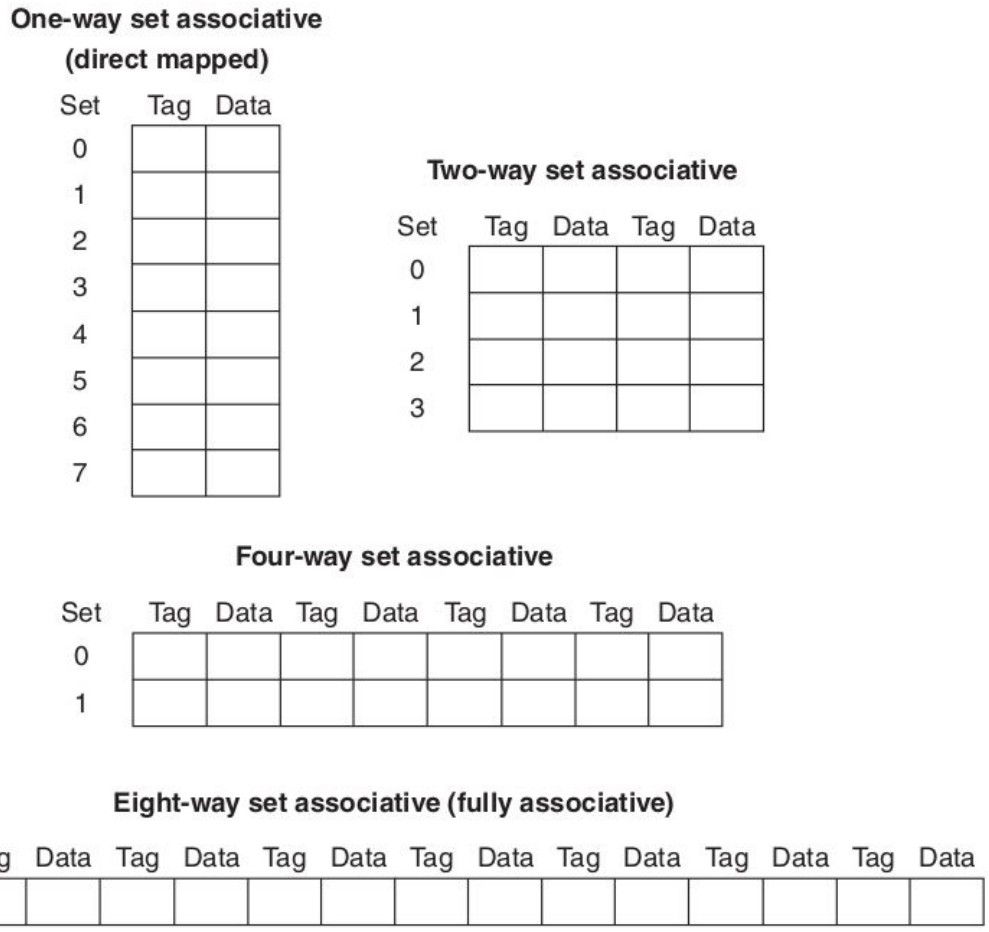
- Fully associative
  - Allow a given block to go in any cache entry
  - Requires all entries to be searched at once
  - Comparator per entry (expensive)
- n-way set associative
  - Each set contains n entries
  - Block number determines which set
    - (Block number) modulo (#Sets in cache)
  - Search all entries in a given set at once
  - n comparators (less expensive)

Associative Cache Example



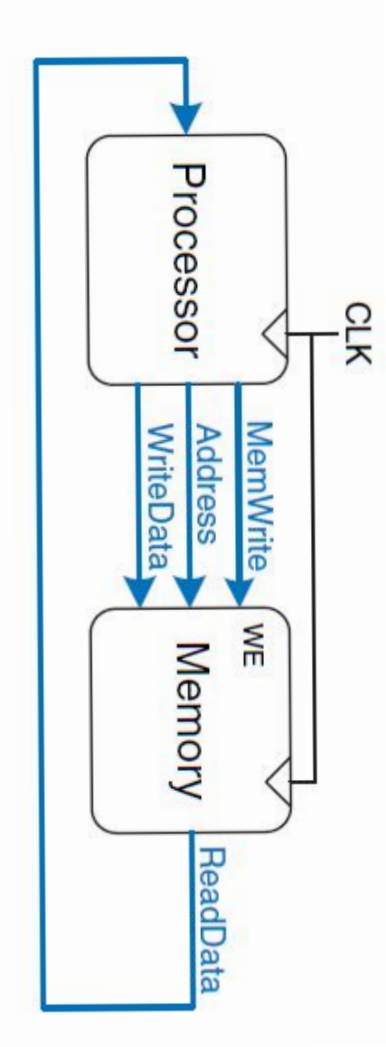
Spectrum of Associativity

- For a cache with 8 entries



Computer System Overview: Recap

- Recall that the overall operation of a computer system is defined by the constant interaction between the **processor** and **memory**
- Processor performs **computations** and memory provides **storage**
- Processor communicates with the memory system over a **memory interface**
  - Processors sends an **address** over the address bus to the memory system
  - For a read, *MemWrite* is de-asserted and the memory returns the data on the *ReadData* bus
  - For a write, *MemWrite* is asserted and the processor sends data to memory on the *WriteData* bus
- The performance of a computing systems is affected by the memory system
- Therefore good design principles should be applied to achieve a memory system with good performance



Characteristics of Ideal Memory System

- Low cost
- High speed
- High density
- Non-volatile
- Read/Write capable
- Low power
- Durable
- Removable

The Memory System Problem Statement

(1) Optimizing the performance of memory systems has always been a big problem due to technological and cost limitations.

(2) For the vast majority of tasks, **computer systems tend to require much more code and data storage than computational hardware**; thus, it is generally not cost-effective to build the memory system using the same technology as the processor.

(3) Over the history of electronic computers, **CPUs have increased in speed (or decreased their clock cycle times) more rapidly than memory devices.** There has always been a performance gap between the CPU and main memory (and a much bigger gap between the CPU and secondary memory), and these gaps have only increased with time.

(4) Thus, **design techniques that effectively close the gap between CPU and memory system performance are more important now than ever.**

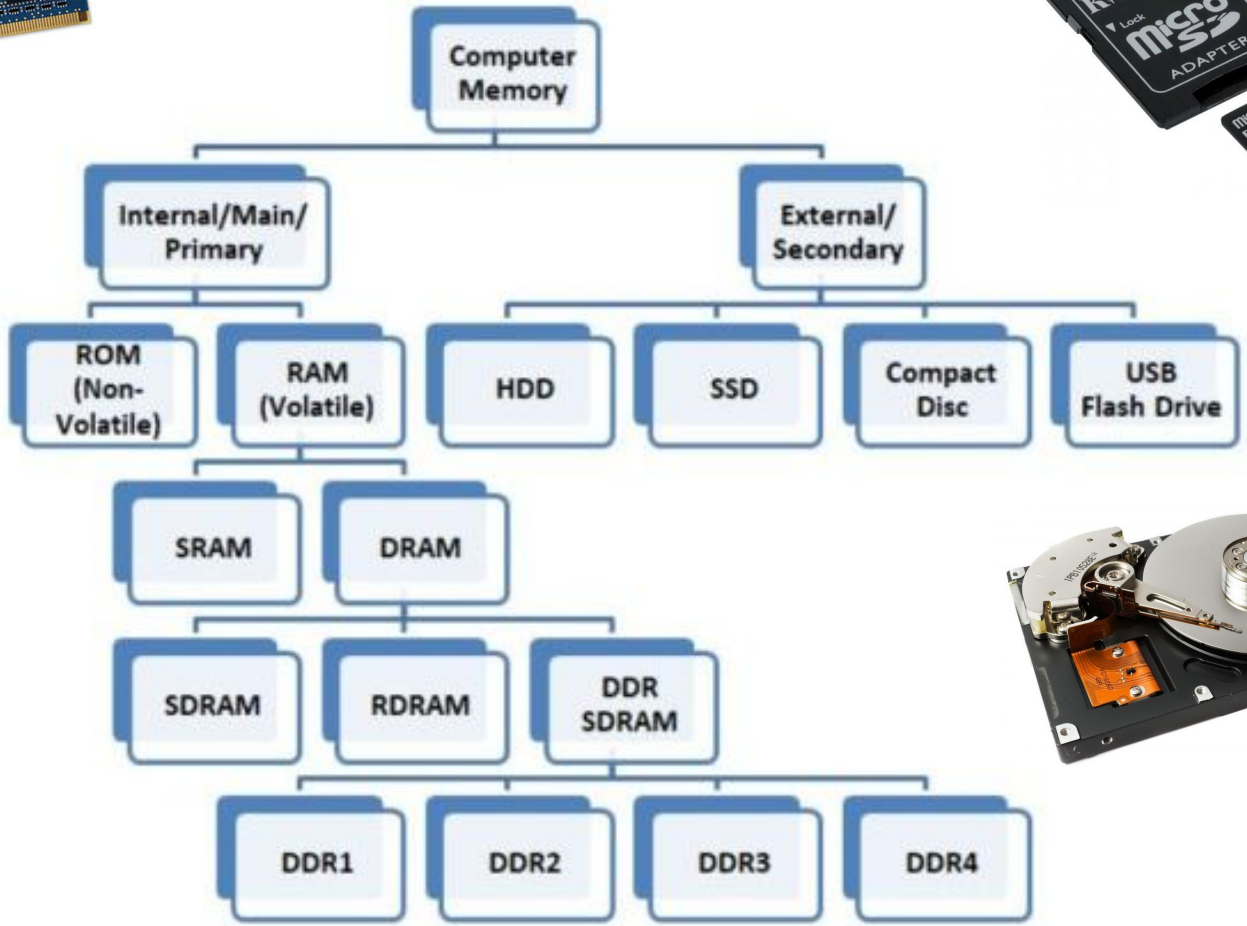
(5) The question has been, still is, and is likely to remain, **"How do we fix things so that the memory system can keep up with the processor's demand for instructions and data?"**

PRACTICAL MEMORY TECHNOLOGIES:

Overview


OWC
OWC Serial Number: 7898001
Warranty Void If Label Removed


Kingston
Lock
micro SD
ADAPTER

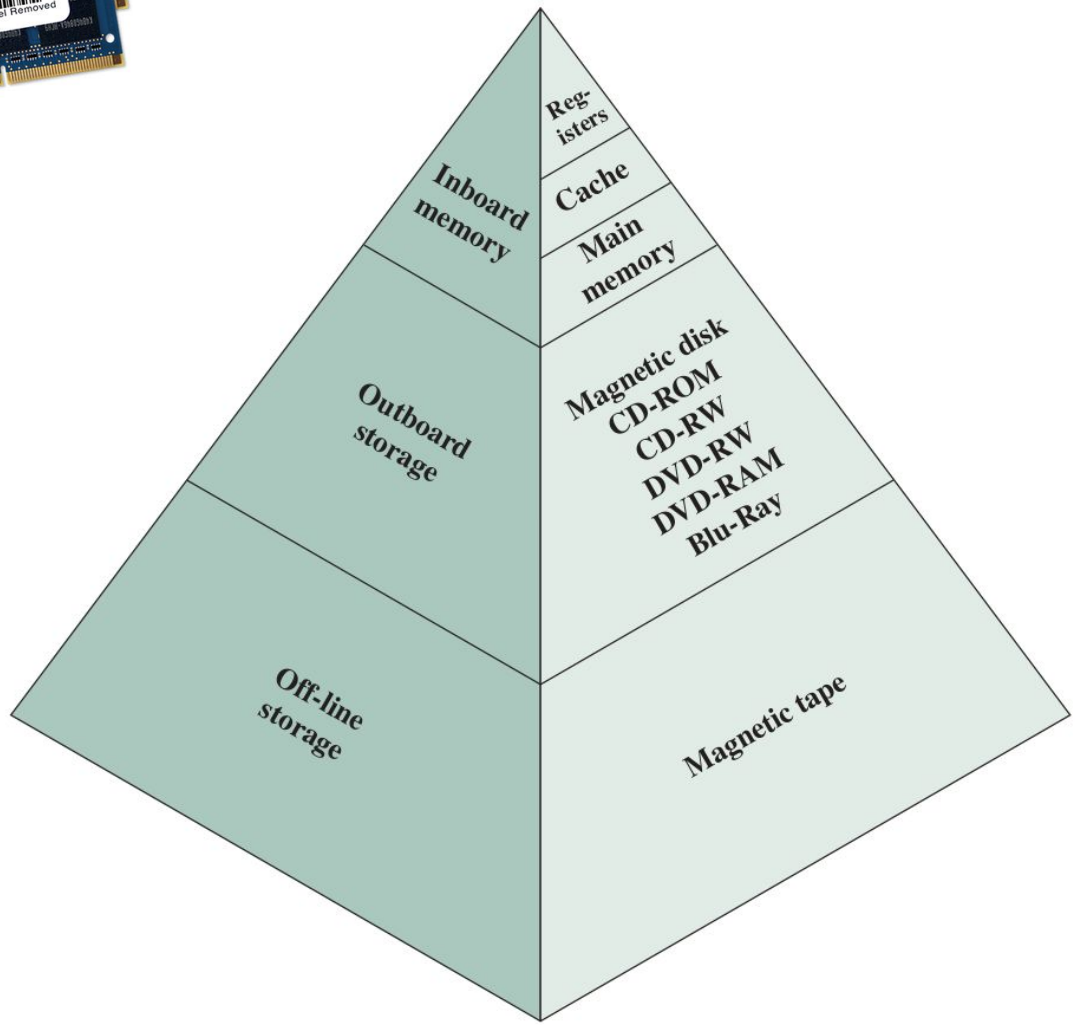

Computer Memory
Internal/Main/ Primary
External/ Secondary
ROM (Non-Volatile)
RAM (Volatile)
HDD
SSD
Compact Disc
USB Flash Drive
SRAM
DRAM
SDRAM
RDRAM
DDR SDRAM
DDR1
DDR2
DDR3
DDR4




Blu-ray Disc

Overview


OWC
OWC Serial Number: 7898001
Warranty Void If Label Removed


Reg-isters
Cache
Main memory
Inboard memory
Magnetic disk
CD-ROM
CD-RW
DVD-RW
DVD-RAM
Blu-Ray
Outboard storage
Magnetic tape
Off-line storage


Kingston
Micro SD
ADAPTER





Overview

- Three main primary technologies used to construct computer memories
  - Semiconductor
  - Magnetic
  - Optical
- Logical organisation
  - Random access memory (RAM)
  - Sequential access memory (SAM)
  - Associative memories
- Semiconductor memory
  - Dynamic random access memory (RAM)
  - Static random access memory (SRAM)
  - Flash
- Magnetic memory
  - Magnetic RAM
- Optical memory
  - CD/DVD
  - Blu-ray

Memory technology	Typical access time	$ per GiB in 2012
SRAM semiconductor memory	0.5–2.5 ns	$500–$1000
DRAM semiconductor memory	50–70 ns	$10–$20
Flash semiconductor memory	5,000–50,000 ns	$0.75–$1.00
Magnetic disk	5,000,000–20,000,000 ns	$0.05–$0.10

Overview

Location
- Internal (e.g., processor registers, cache, main memory)
- External (e.g., optical disks, magnetic disks, tapes)

Capacity
- Number of words
- Number of bytes

Unit of Transfer
- Word
- Block

Access Method
- Sequential
- Direct
- Random
- Associative

Performance
- Access time
- Cycle time
- Transfer rate

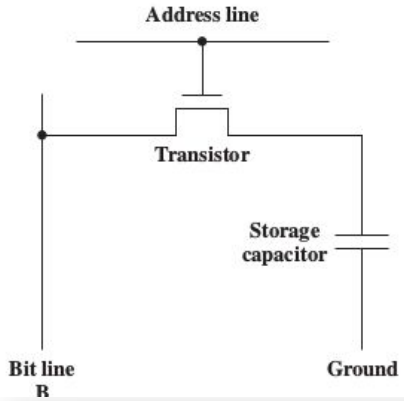
Physical Type
- Semiconductor
- Magnetic
- Optical
- Magneto-optical

Physical Characteristics
- Volatile/nonvolatile
- Erasable/nonerasable

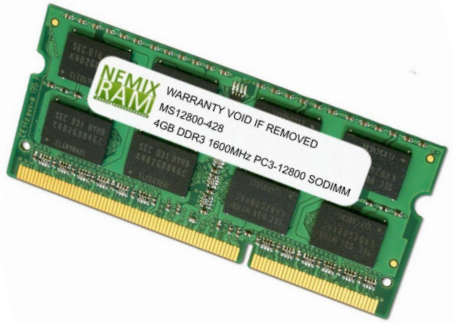
Organization
- Memory modules

Semiconductor Memories: DRAM

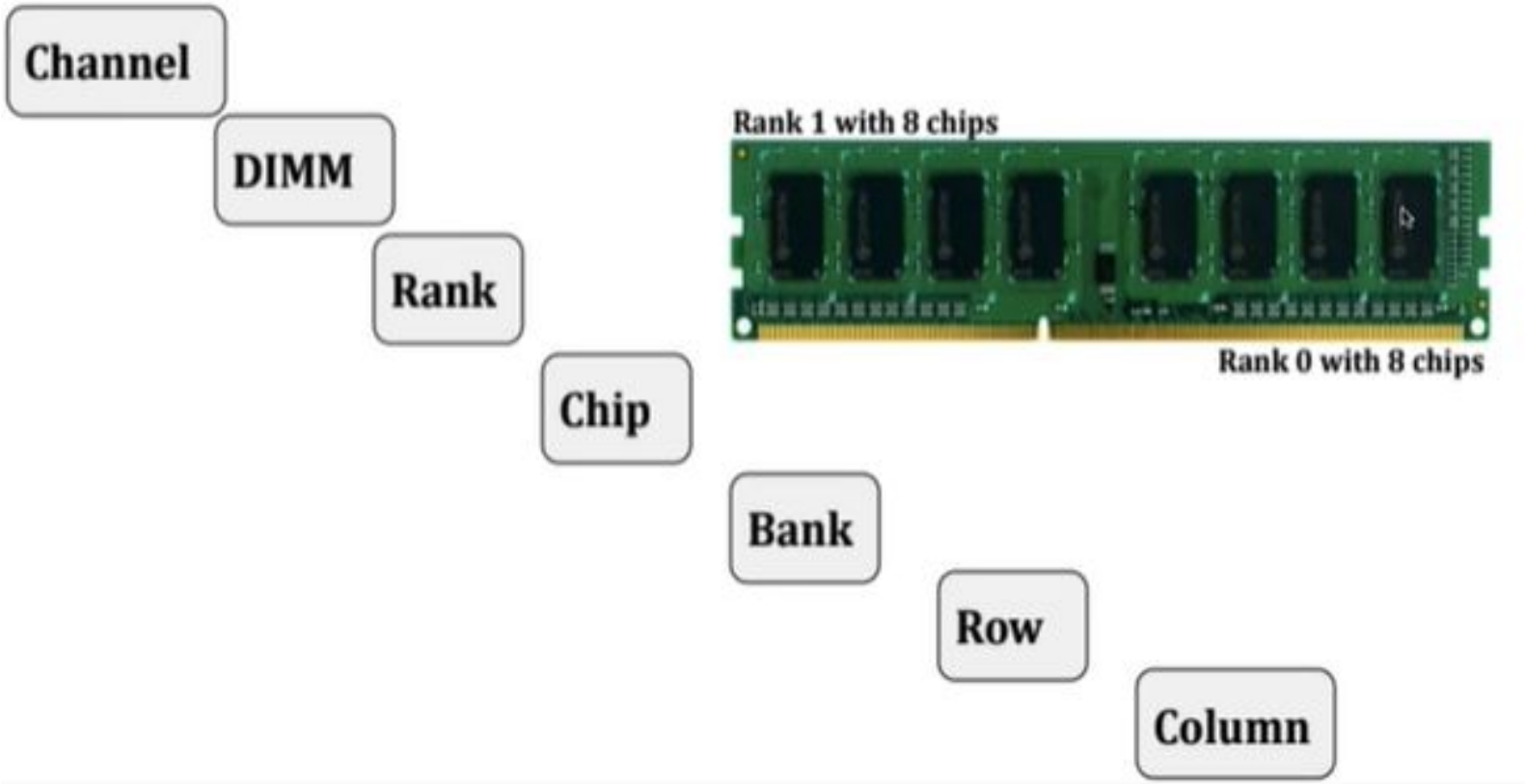
- Memory cell structure
  - Capacitor-based storage
- Advantages
  - Density
  - Cost
  - Speed
- Disadvantages
  - Refreshing
  - Volatile
  - Speed
- Organisation and speeds
  - Dual in-line memory module (DIMM) Small outline DIMM (SODIMM)
  - Double data rate (DDR) – DDR2, DDR3, DDR4, DDR5
- Applications
  - Main memory in modern computers and graphics cards
  - Mobile devices and game consoles



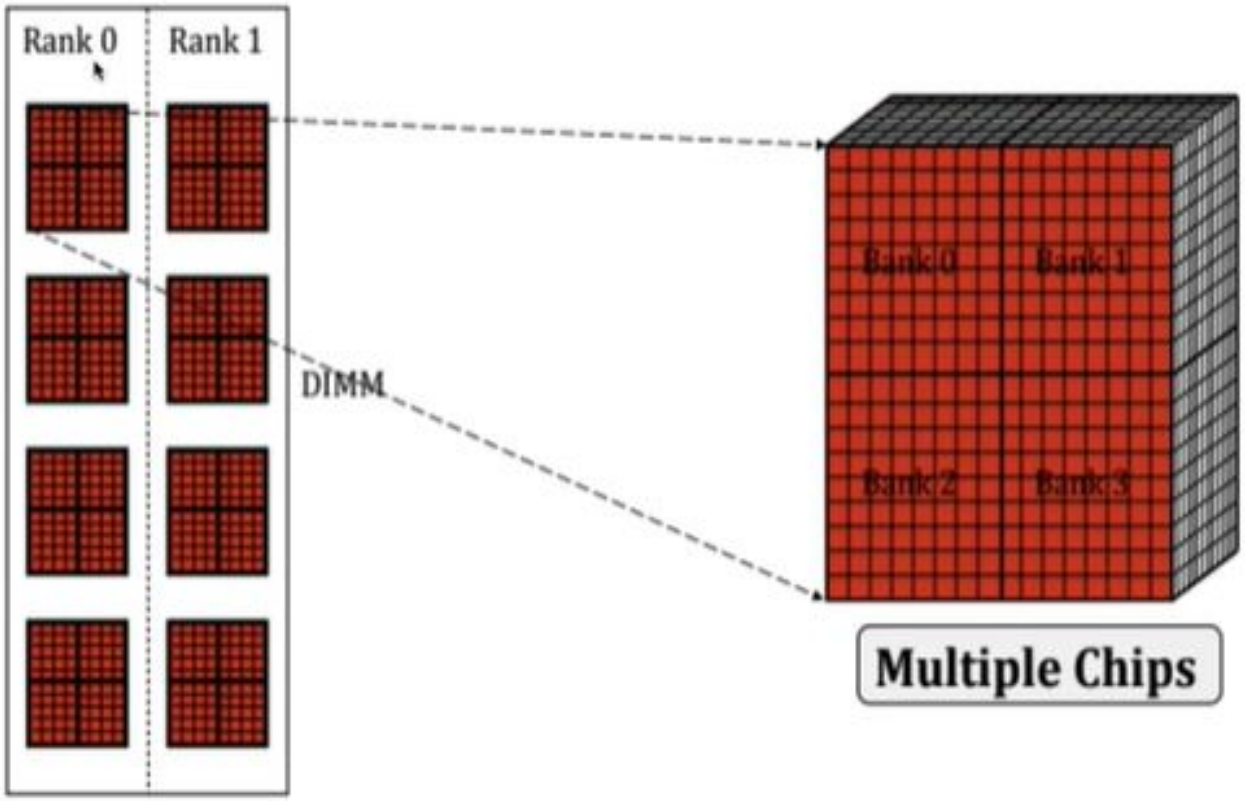

```
khobatha@khobatha-Lenovo-IdeaPad-Y500: ~
khobatha@khobatha-Lenovo-IdeaPad-Y500:~$ sudo lshw -short -C memory
H/W path         Device      Class          Description
=====================================================
/0/0                         memory         128KiB BIOS
/0/4/9                       memory         32KiB L1 cache
/0/4/a                       memory         256KiB L2 cache
/0/4/b                       memory         6MiB L3 cache
/0/8                         memory         32KiB L1 cache
/0/28                        memory         8GiB System Memory
/0/28/0                      memory         4GiB SODIMM DDR3 Synchronous 1600 MHz (0.6 ns)
/0/28/1                      memory         4GiB SODIMM DDR3 Synchronous 1600 MHz (0.6 ns)
khobatha@khobatha-Lenovo-IdeaPad-Y500:~$
```



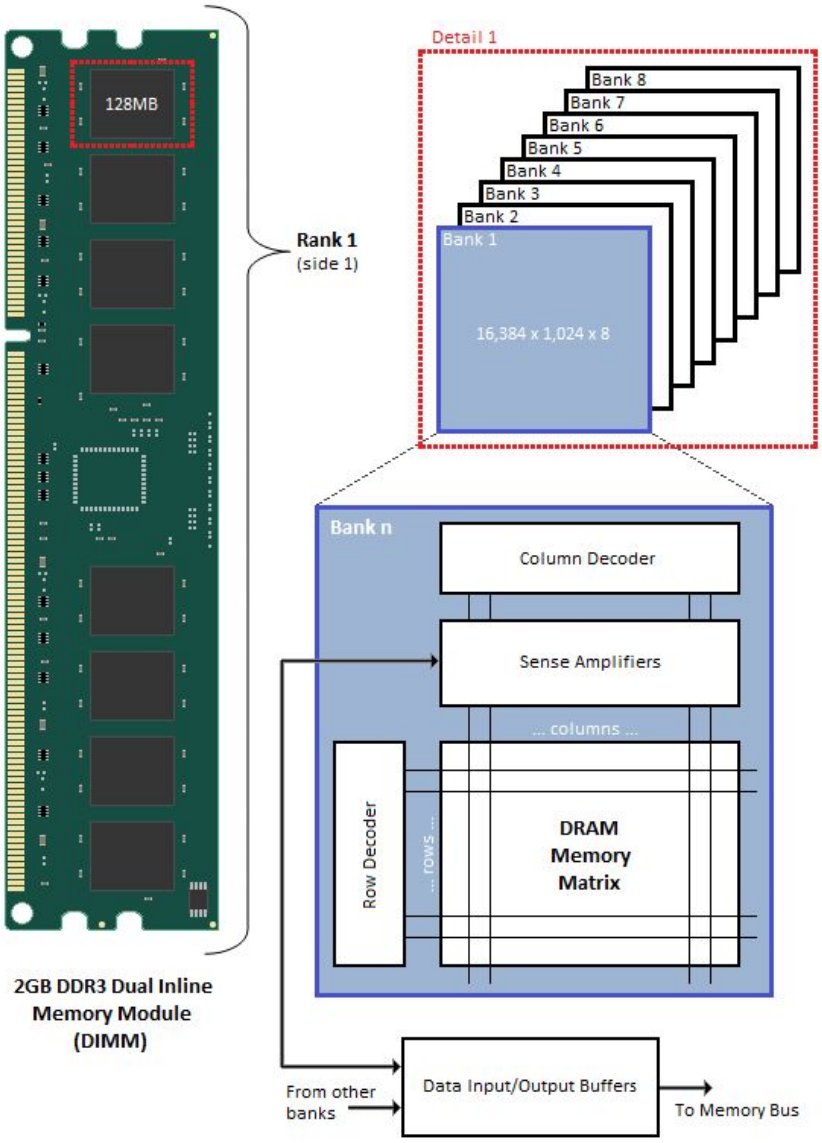
Semiconductor Memories: DRAM Internal Organisation



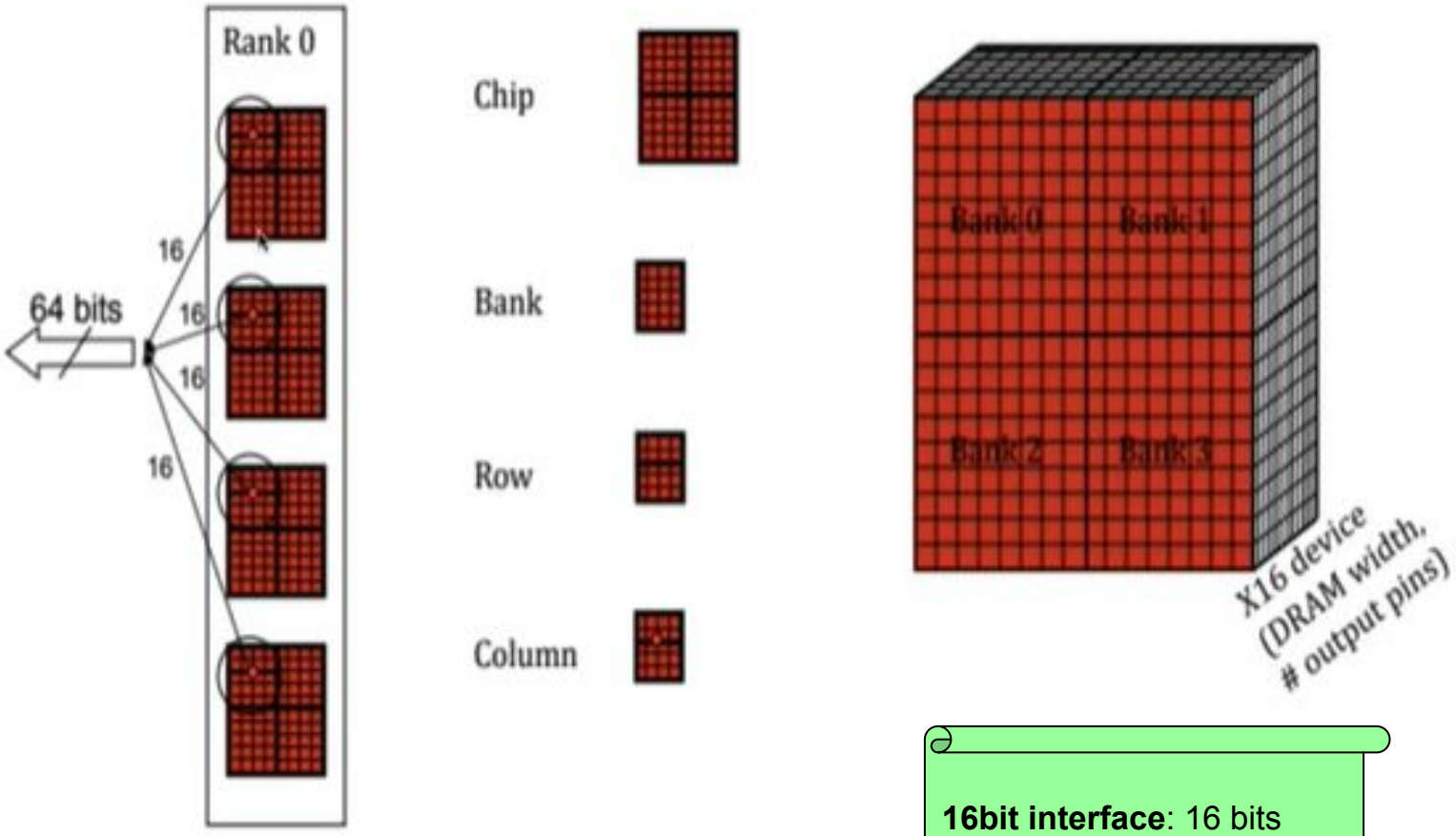
Semiconductor Memories: DRAM Internal Organisation



Semiconductor Memories: DRAM Internal Organisation

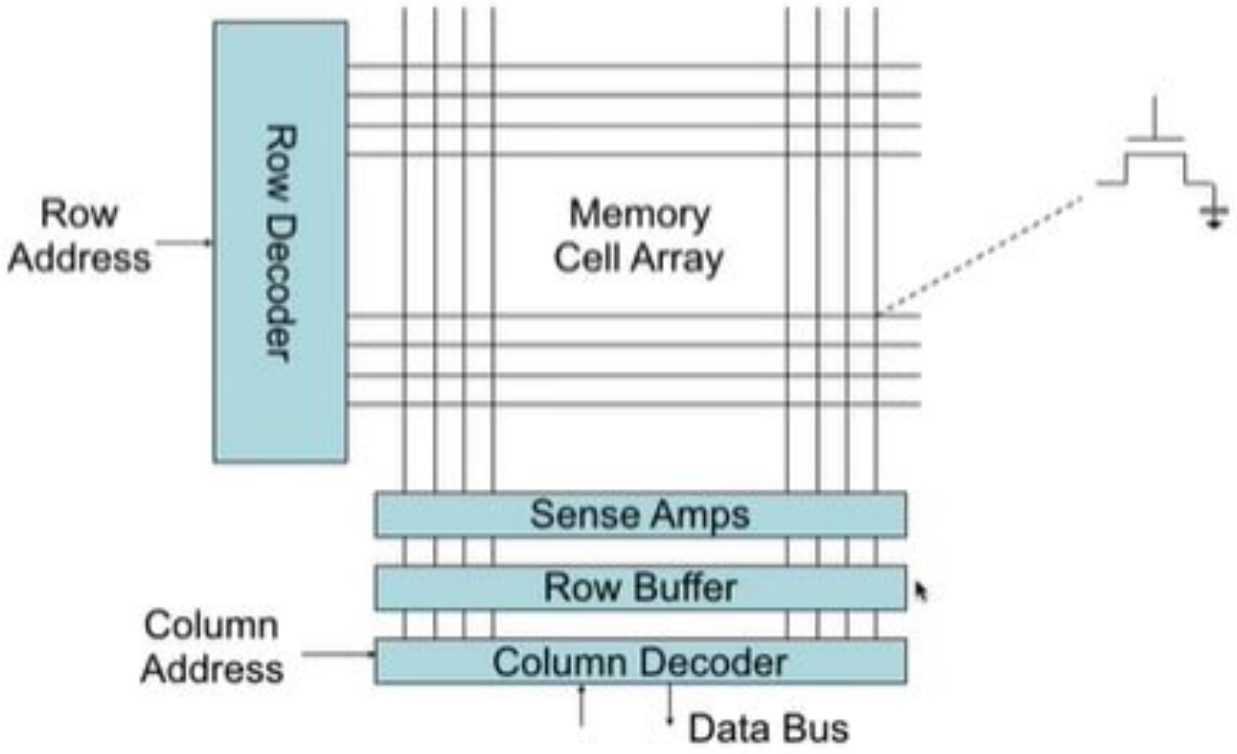


Semiconductor Memories: DRAM Internal Organisation



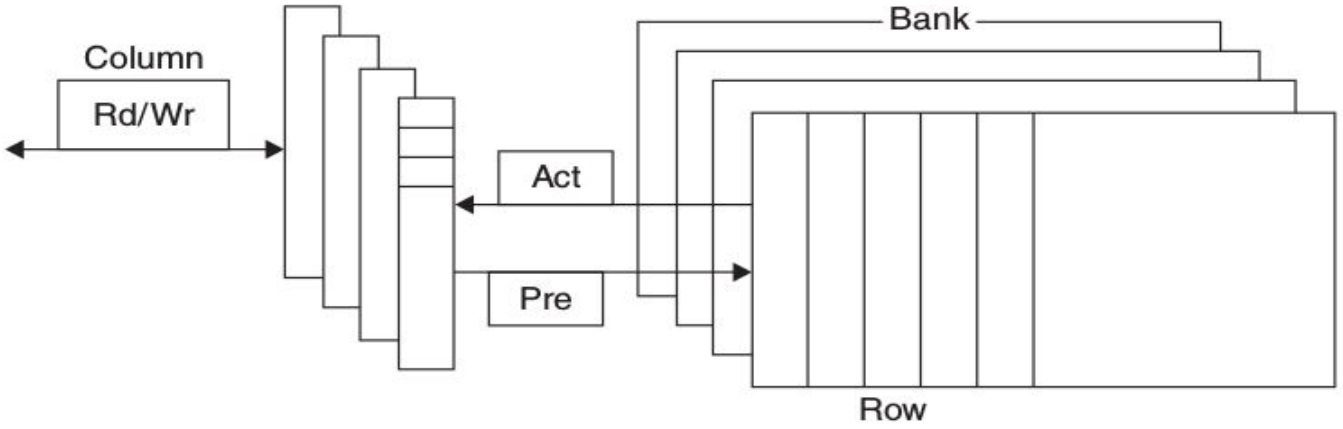
16bit interface: 16 bits from one chip in one go.

Semiconductor Memories: DRAM Internal Organisation

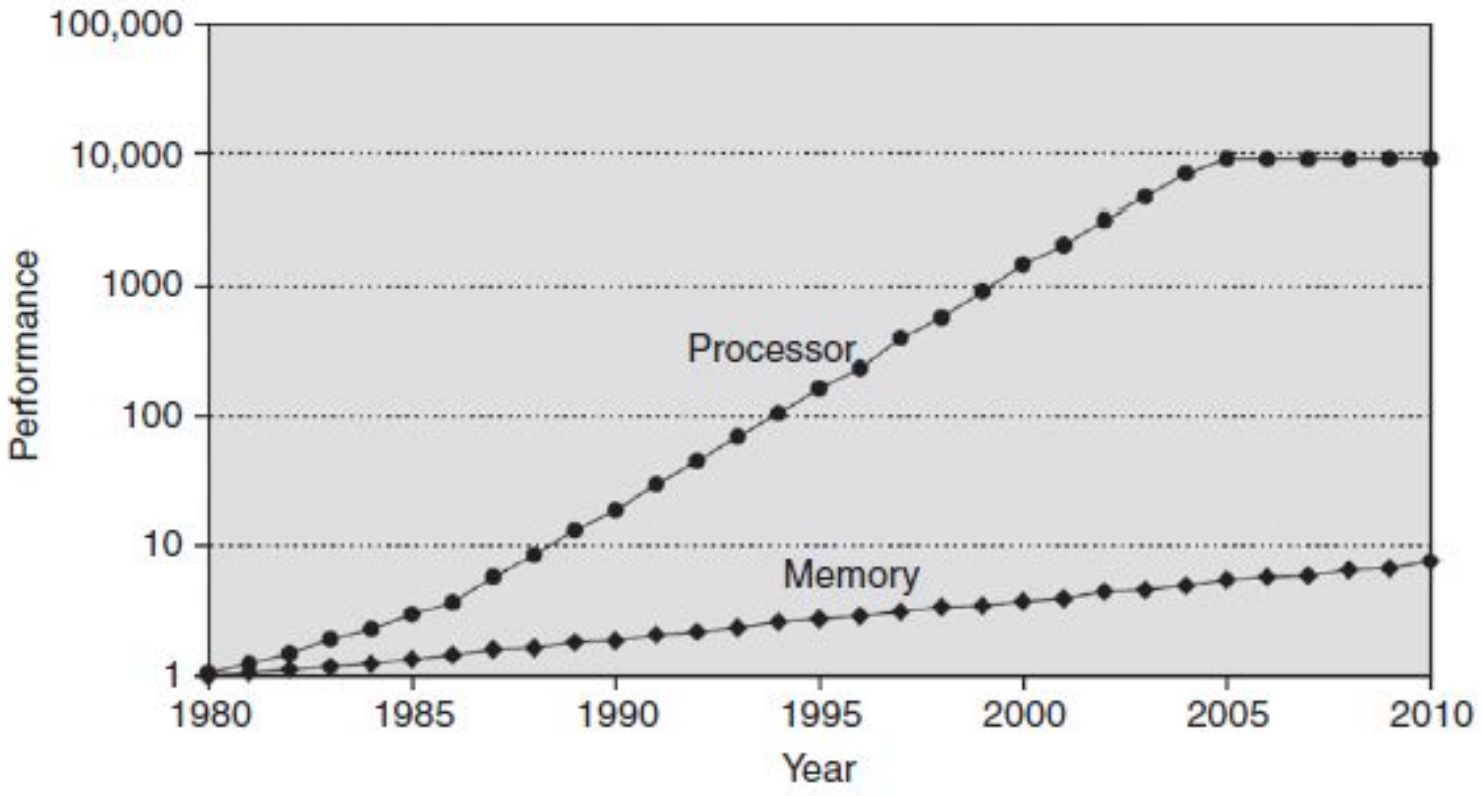


Semiconductor Memories: DRAM Internal Organisation

- Modern DRAMs are organised in **banks** (e.g four for DDR3)
- Each bank consists of a series of **rows**. Sending a **PRE (precharge)** command opens or closes a bank.
- A row address is sent with an **Act (activate)**, which causes the row to transfer to a buffer.
- When the row is in the buffer, it can be transferred by successive column addresses at whatever the width of the DRAM is (typically 4, 8, or 16 bits in DDR3) or by specifying a block transfer and the starting address.
- Each command, as well as block transfers, is synchronized with a clock.

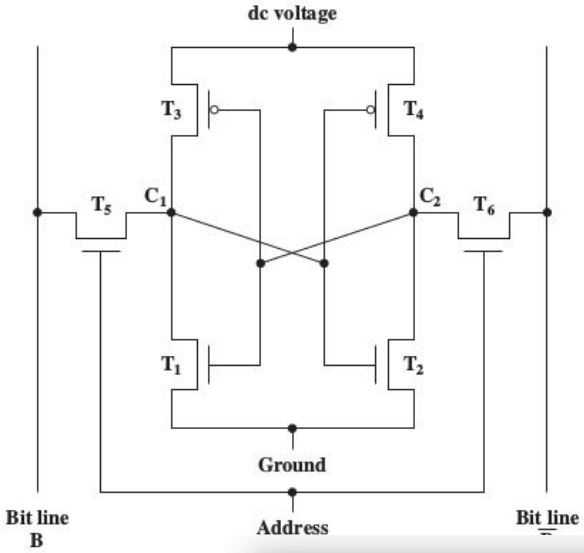


Semiconductor Memories: CPU – DRAM Performance Gap



Semiconductor Memories: SRAM

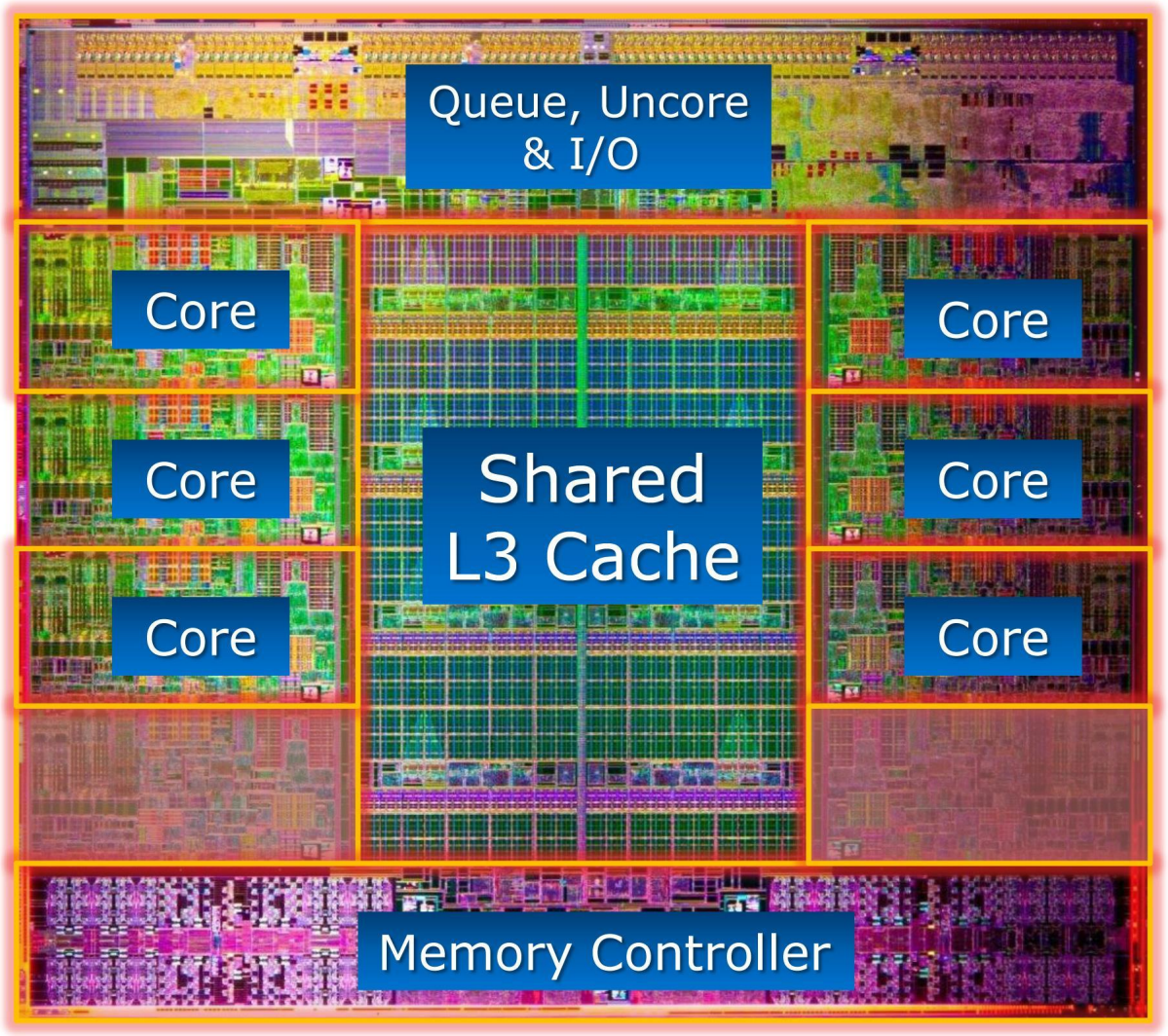
- Structure
  - Flip-flop based storage
- Advantages
  - Speed
  - No refreshes
  - Simplicity
  - Idle power
- Disadvantages
  - Density
  - Cost
  - Volatile
  - Operational power
- Applications
  - CPU cache in computers, workstations, routers
  - LCD screens as storage for image to be displayed
  - Printers as storage for data to be printed




```
khobatha@khobatha-Lenovo-IdeaPad-Y500: ~
khobatha@khobatha-Lenovo-IdeaPad-Y500:~$ sudo lshw -short -C memory
H/W path          Device      Class          Description
=====================================================
/0/0                          memory         128KiB BIOS
/0/4/9                        memory         32KiB L1 cache
/0/4/a                        memory         256KiB L2 cache
/0/4/b                        memory         6MiB L3 cache
/0/8                          memory         32KiB L1 cache
/0/28                         memory         8GiB System Memory
/0/28/0                       memory         4GiB SODIMM DDR3 Synchronous 1600 MHz (0.6 ns)
/0/28/1                       memory         4GiB SODIMM DDR3 Synchronous 1600 MHz (0.6 ns)
khobatha@khobatha-Lenovo-IdeaPad-Y500:~$
```

Semiconductor Memories: SRAM -Intel Core i7 3690X Cache





Semiconductor Memories: Flash

- Structure
  - Type of electrically erasable programmable read-only memory (EEPROM)
- Advantages
  - Cost
  - Nonvolatile
  - Fast read-access time
  - Mechanical shock resistance
- Disadvantages
  - Writes can wear out flash memory bits
  - Wear leveling used to cope with wear limits
  - Wear leveling degrades performance
- Applications
  - Secondary memory in personal mobile devices e.g mobile phones, digital cameras, tablets



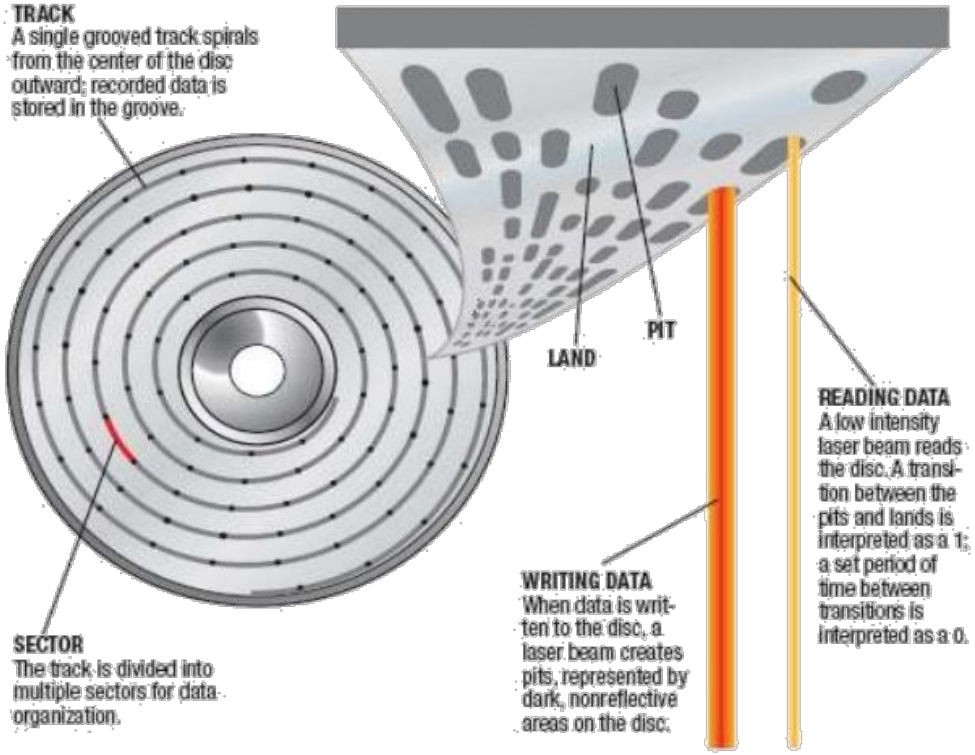
Magnetic Memories

- Structure
  - One or more rigid disks or platters
  - Read/write head
- Advantages
  - Non-volatile
  - Cost
  - Removable
  - Capacity
  - Density
- Disadvantages
  - Speed
- Applications
  - Secondary storage in modern computers



Optical Memories

- Structure and operation
  - Tracks
  - Laser
- Advantages
  - Non-volatile
  - Cost
  - Removable
  - Capacity
  - Density
- Disadvantages
  - Speed
- Applications
  - Multimedia



Lecture Outline

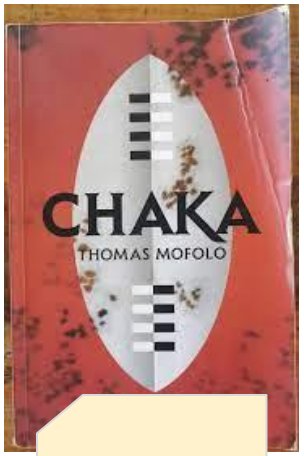
- **Introduction**
  - Fundamental concepts
  - Ideal memory requirements
- **Practical memory technologies**
  - Semiconductor memories
  - Magnetic memories
  - Optical memories
- **Practical memory systems**
  - Hierarchical memory system
- **Cache memory**
  - Mapping strategies
  - Write policies
  - Replacement strategies
- **Virtual memory**
  - Address translation
  - Memory protection
  - Replacement strategies

PRACTICAL MEMORY SYSTEMS:

Memory as a Library System



Authors: I/O

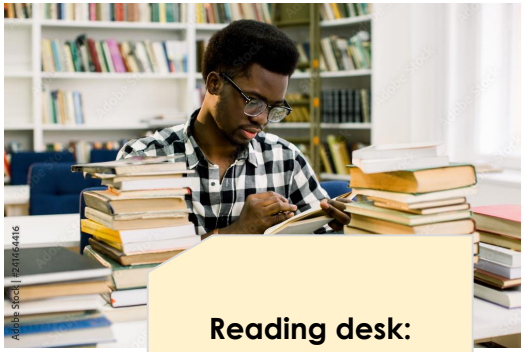


Books: Data



Library basement: Storage

- locality
- cache hit/miss
- miss penalty
- replacement policy



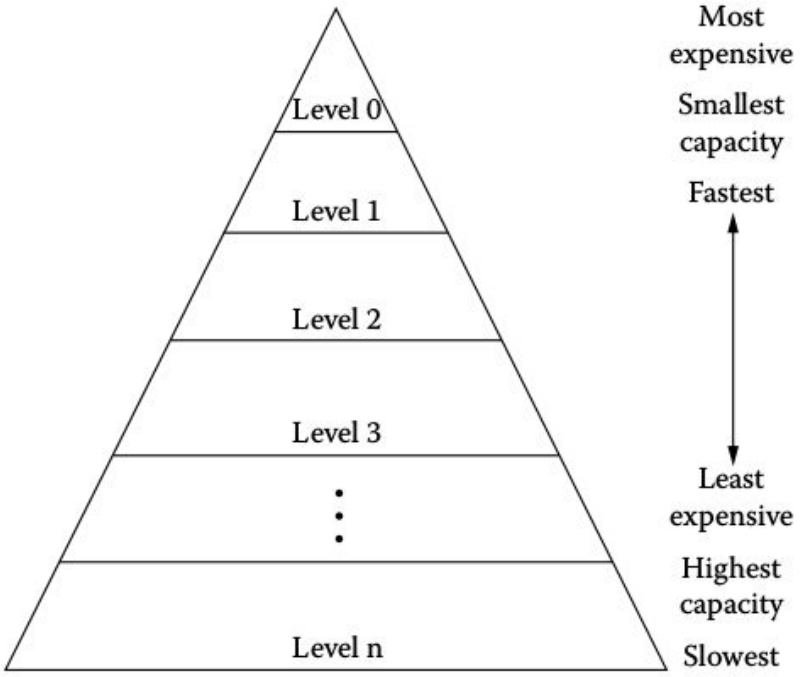
Reading desk: Cache Memory



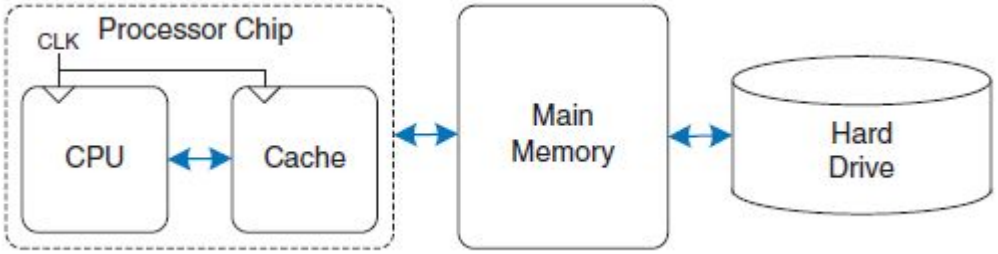
Library shelves: Main memory

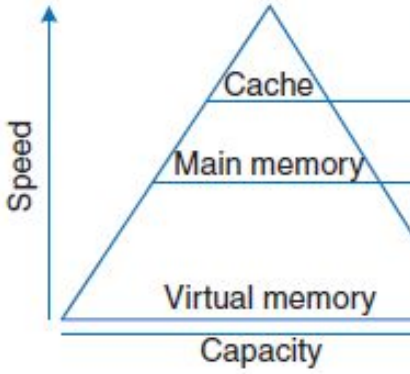
Hierarchical Memory System: Concept

- Requirements
  - Large capacity
  - High density
  - Read/Write capable
  - Inexpensive
  - Removable
  - Nonvolatile
- Reality
  - Practically impossible
- Compromise – hierarchical solution
  - Use a mixture of different device types
  - for trade-off
  - Most expensive, smallest and fastest at the top
  - Least expensive, largest and slowest at the bottom



Hierarchical Memory System





Technology	Price / GB	Access Time (ns)	Bandwidth (GB/s)
SRAM	$10,000	1	25+
DRAM	$10	10–50	10
SSD	$1	100,000	0.5
HDD	$0.1	10,000,000	0.1

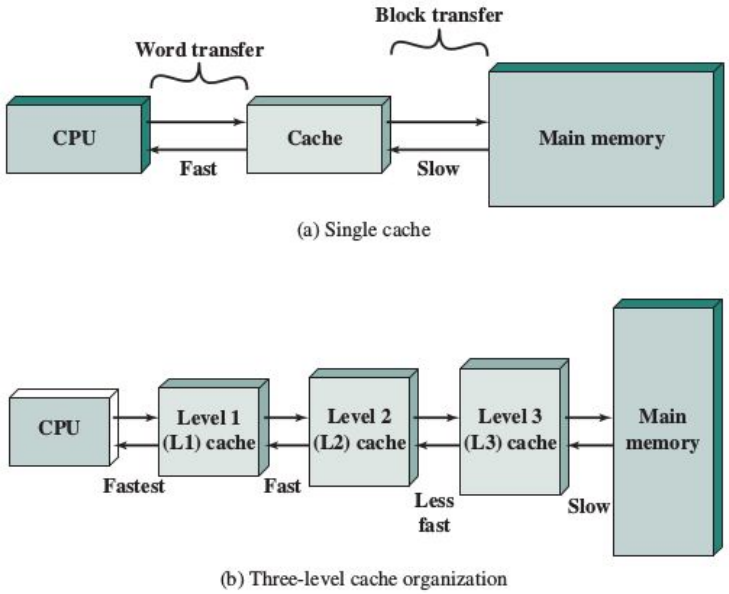
(1) Decrease cost per bit
(2) Increase capacity
(3) Increase access time
(4) Decrease frequency of memory access by the processor through **locality of reference (spatial and temporal)**

Hierarchical Memory: Registers

processor	Generic	Special	Float	SIMD
MOS 6502	2	1xAccu 1x16 SP		
Intel 8080	7x8 or 1x8 + 3x16	1x16 SP		
Intel 8086	8x16 or 8x8 + 4x16			
Motorola 68000	8x32	8x32 address	8x32	
ARM A64	31x64		32x64	
Intel Core i7	16x (64|32|16|8)			32x (512|256|128)
Cell PPE core	32x64		32x64	32x128
Cell SPE core				128x128
CUDA CC 5.x	1x (64|32)			128x512

Hierarchical Memory: Cache Memory

- Cache memory is used by the CPU in a hierarchical memory system to reduce the average cost (time and energy) to access data from main memory
- A cache is smaller, faster and is located closer to a processor core
- Stores copies of data from frequently used main memory locations




```
khobatha@khobatha-Lenovo-IdeaPad-Y500: ~
khobatha@khobatha-Lenovo-IdeaPad-Y500:~$ sudo lshw -short -C memory
H/W path         Device      Class          Description
=====================================================
/0/0                         memory         128KiB BIOS
/0/4/9                       memory         32KiB L1 cache
/0/4/a                       memory         256KiB L2 cache
/0/4/b                       memory         6MiB L3 cache
/0/8                         memory         32KiB L1 cache
/0/28                        memory         8GiB System Memory
/0/28/0                      memory         4GiB SODIMM DDR3 Synchronous 1600 MHz (0.6 ns)
/0/28/1                      memory         4GiB SODIMM DDR3 Synchronous 1600 MHz (0.6 ns)
khobatha@khobatha-Lenovo-IdeaPad-Y500:~$
```

Cache Memory: Intel Core i7 3630QM Cache



```
Handle 0x0008, DMI type 7, 19 bytes
Cache Information
        Socket Designation: L1 Cache
        Configuration: Enabled, Not Socketed, Level 1
        Operational Mode: Write Through
        Location: Internal
        Installed Size: 32 kB
        Maximum Size: 32 kB
        Supported SRAM Types:
                Unknown
        Installed SRAM Type: Unknown
        Speed: Unknown
        Error Correction Type: Parity
        System Type: Data
        Associativity: 8-way Set-associative

Handle 0x0009, DMI type 7, 19 bytes
Cache Information
        Socket Designation: L1 Cache
        Configuration: Enabled, Not Socketed, Level 1
        Operational Mode: Write Through
        Location: Internal
        Installed Size: 32 kB
        Maximum Size: 32 kB
        Supported SRAM Types:
                Unknown
        Installed SRAM Type: Unknown
        Speed: Unknown
        Error Correction Type: Parity
        System Type: Instruction
        Associativity: 8-way Set-associative

Handle 0x000A, DMI type 7, 19 bytes
Cache Information
        Socket Designation: L2 Cache
        Configuration: Enabled, Not Socketed, Level 2
        Operational Mode: Write Through
```

Lecture Outline

- Introduction
  - Fundamental concepts
  - Ideal memory requirements
- Practical memory technologies
  - Semiconductor memories
  - Magnetic memories
- **Practical memory systems**
  - Hierarchical memory system
  - Memory interleaving
- **Cache memory**
  - Mapping strategies
  - Write policies
  - Replacement strategies
- **Virtual memory**
  - Address translation
  - Memory protection
  - Replacement strategies

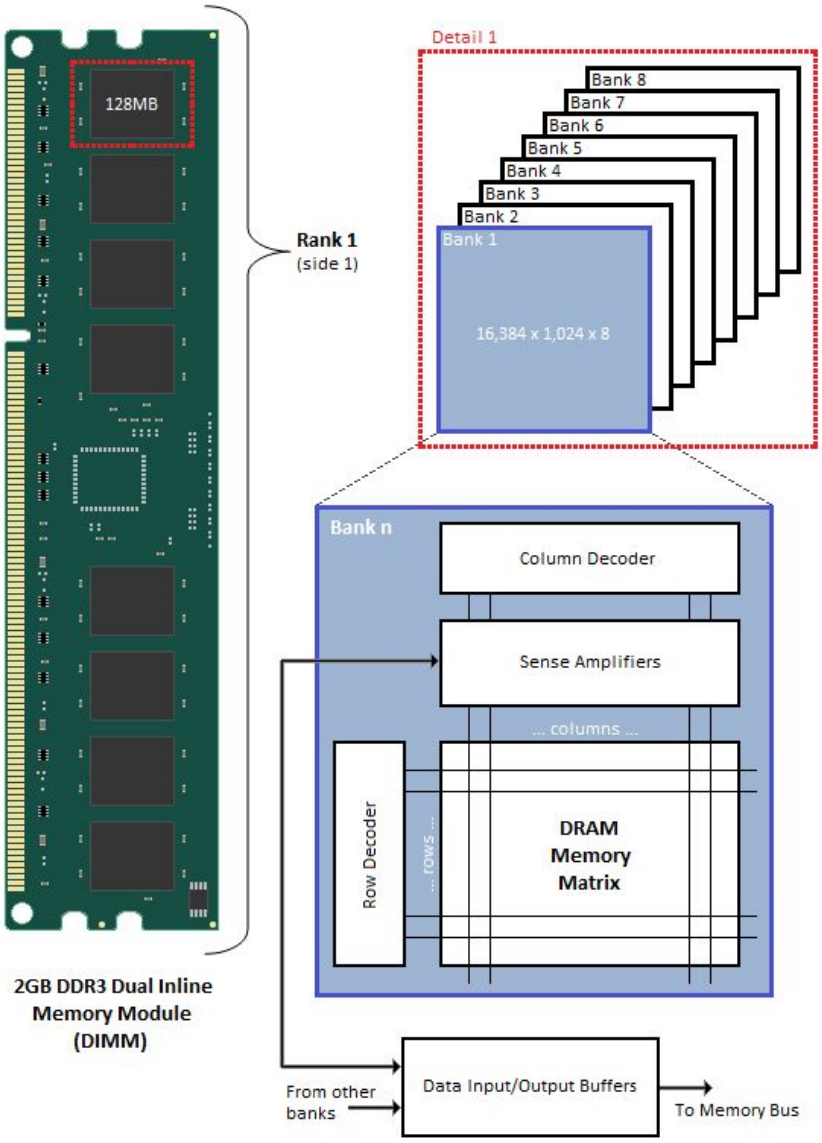
Memory Interleaving Market Analogy



Memory Interleaving

- **Definition**:
  - Memory interleaving is a technique used to improve the speed of memory access in computers.
- **Mechanism:**
  - It involves distributing memory addresses across multiple memory modules, allowing simultaneous access to these modules.
- **Purpose**:
  - The main goal is to increase data throughput and reduce memory access bottlenecks.
- **Types**:
  - **Low-order** - distributes **consecutive memory addresses** across different memory modules.
  - **High-order** - distributes **blocks of memory addresses** across different modules.

Main Memory Organisation (Recap)



Primary Memory: High-order Interleaving

- **High-order**
  - High-order address bits used to select device
  - Simplest and most common way
- **Advantages**
  - Simple
  - Requires few external parts
  - Fail-safe
- **Disadvantages**
  - Idling
  - Memory as fast as one device

Address bits	25-22	21-0
Use	Bank Select	Address to the chip

Module 0	Module 1	Module 2	Module 3	Module 4	Module 5	Module 6	Module 7
0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

Primary Memory: High-order Interleaving Example

Aim:

design 64KB memory using 16K x 8 devices.

No. of 16Ks?

64KB/16KB = 4

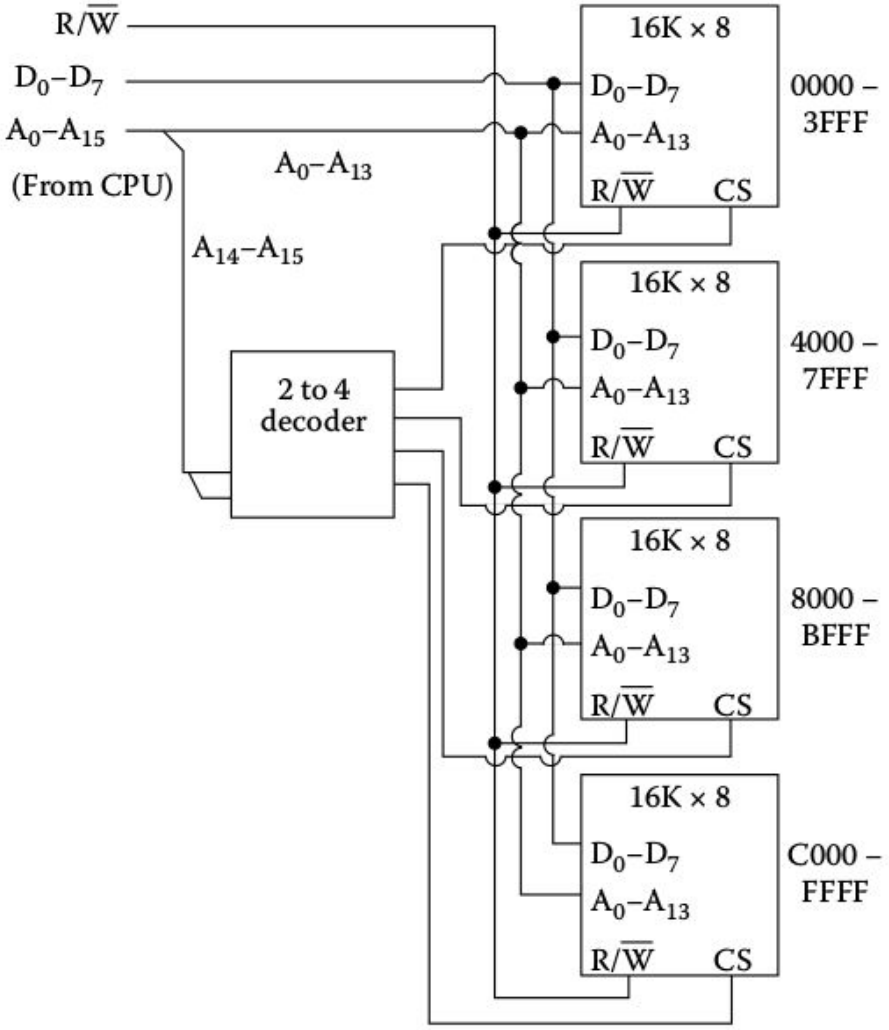
No. of address lines for 64K?

64K=65,536 = 2^{16}, needs 16 address lines

No. of address lines for 16K?

16K= 16,384 = 2^{14}, has 14 address lines

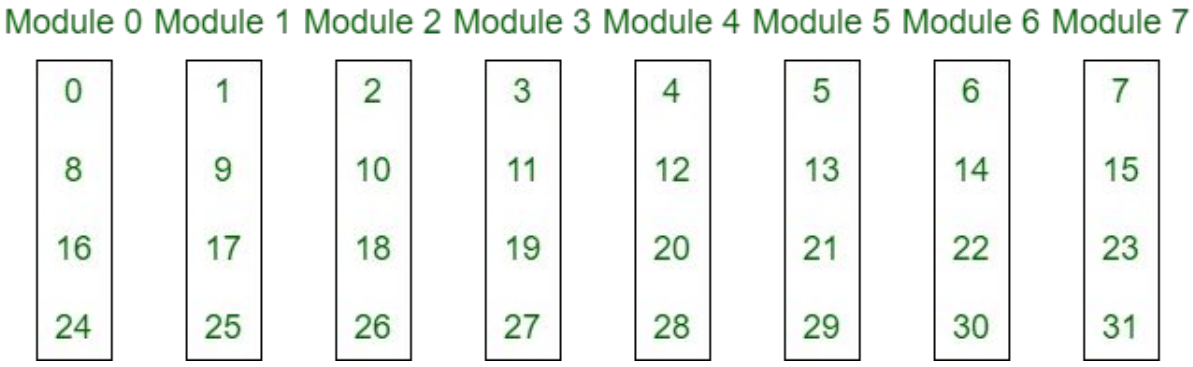
Low-order 14 address bits connect to All devices in common.
High-order 2 address bits used to generate chip select inputs.



Primary Memory: Low-order Interleaving

- **Low-order interleaving**
  - Large main memory constructed from a number of smaller ones
  - Higher-order bits connected in common to chips
  - Lowest-order address bits used to select chip
  - Consecutive address in different devices
- **Advantages**
  - Speed
- **Disadvantages**
  - Complexity

Address bits	25-4	3-0
Use	Address to the chip	Bank Select



Primary Memory: Low-order Interleaving

Aim:

design 64KB memory using 16K x 8 devices.

No. of 16Ks?

64KB/16KB = 4

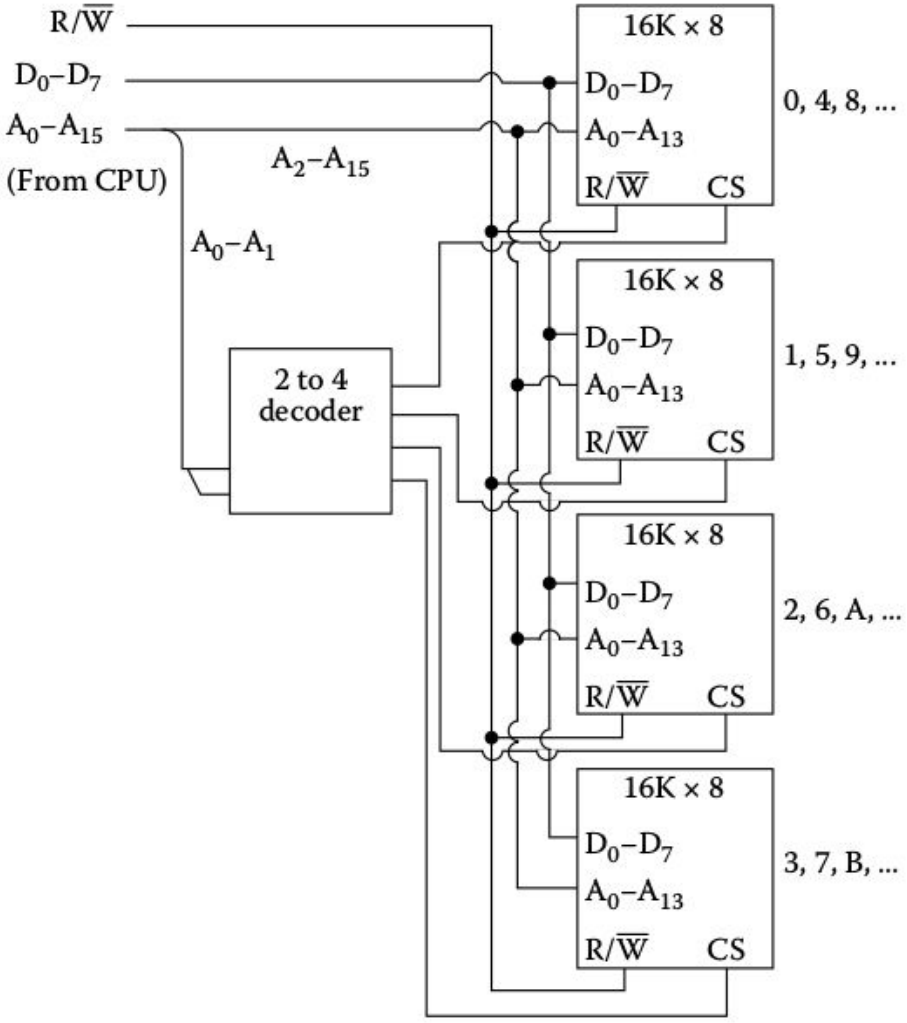
No. of address lines for 64K?

64K = 65,536 = 2^{16}, needs 16 address lines

No. of address lines for 16K?

16K = 16,384 = 2^{14}, has 14 address lines

Low-order 2 address bits connect to generate chip select inputs.
High-order 14 address bits connected to all devices in common.



PRACTICAL MEMORY SYSTEMS:
Performance Measurement

Performance Metrics



- Why performance metrics?
  - Computer designers (and buyers) need quantitative metrics to measure performance of memory systems
  - Evaluate the cost-benefit trade-offs of various alternatives
- Memory system performance metrics
  - Hit rate
  - Miss rate
  - Average memory access time (AMAT)
- Calculations
  - t_{cache}, t_{MM}, t_{VM} are cache, main memory and virtual memory access times and
  - MR_{cache} and MR_{MM} are the cache and main memory miss rate





$$AMAT = t_{\text{cache}} + MR_{\text{cache}}(t_{MM} + MR_{MM}t_{VM})$$

Example: Performance Measure (Question)



Q: Suppose a program has 2000 data access instructions (loads or stores), and 1250 of these requested data values are found in the cache. The other 750 data values are supplied to the processor by main memory or disk memory. What are the miss and hit rates for the cache?

Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%

Example: Performance Measure (Solution)



Q: Suppose a program has 2000 data access instructions (loads or stores), and 1250 of these requested data values are found in the cache. The other 750 data values are supplied to the processor by main memory or disk memory. What are the miss and hit rates for the cache?

Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%

A: The miss rate is 750/2000 = 0.375 = 37.5%.

The hit rate is 1250/2000 = 0.625 = 1 − 0.375 = 62.5%.

Example: Performance Measure (Question)



Q: Suppose a computer system has a memory organization with only two levels of hierarchy, a cache and main memory. What is the average memory access time given the access times and miss rates in the table below?

Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%

Example: Performance Measure (Solution)



Q: Suppose a computer system has a memory organization with only two levels of hierarchy, a cache and main memory. What is the average memory access time given the access times and miss rates in the table below?

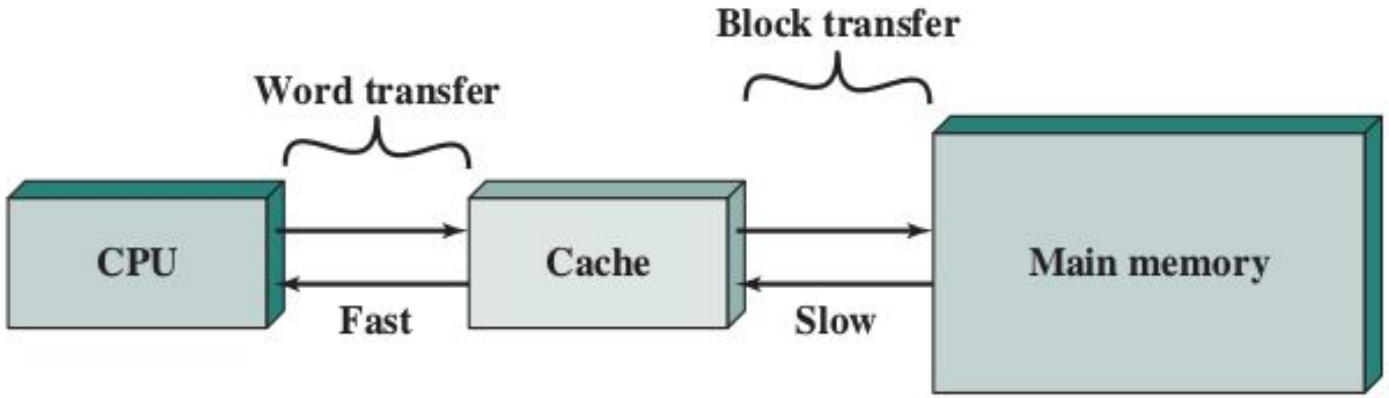
Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%

A: The average memory access time is 1 + 0.1(100) = 11 cycles.

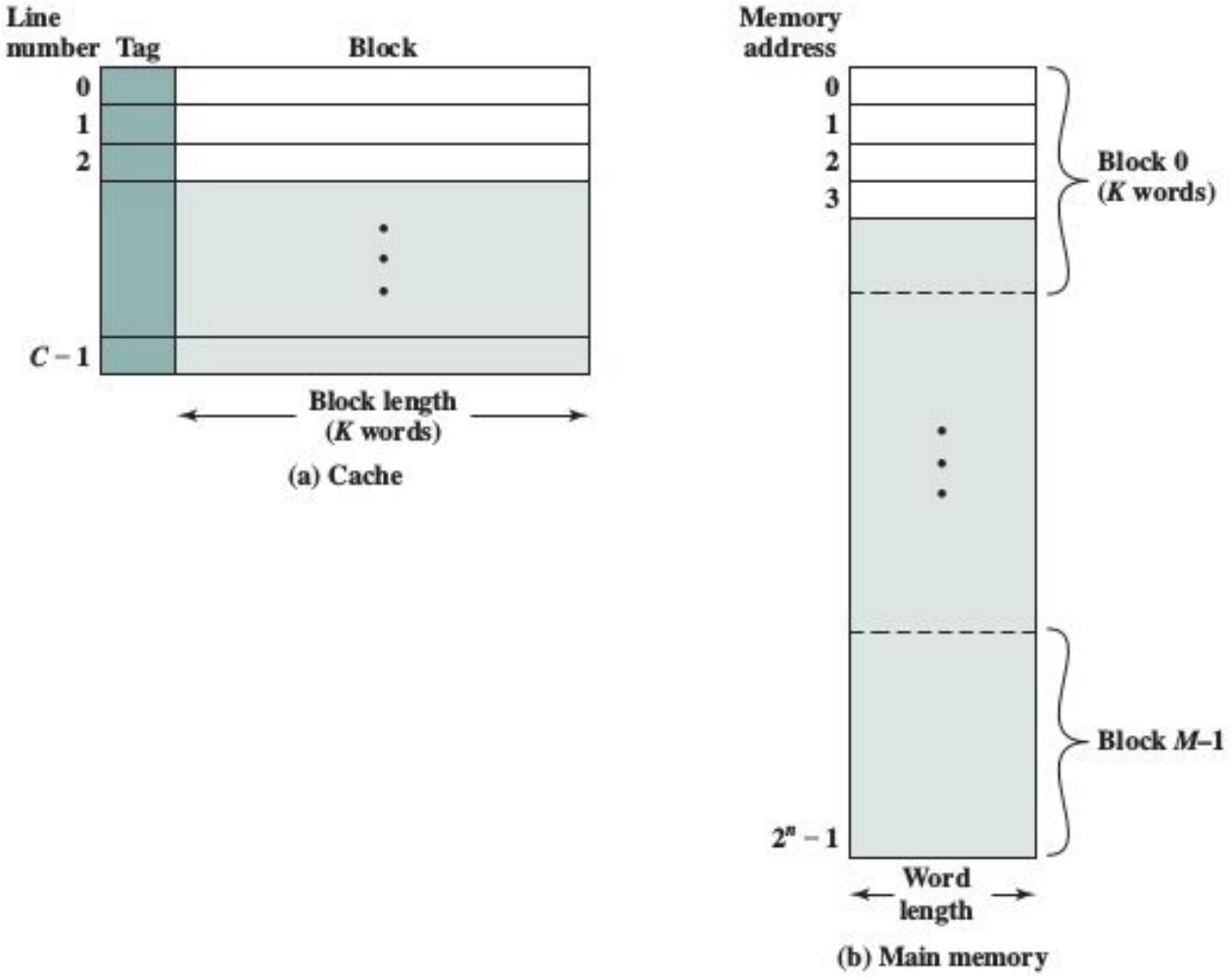
CACHE MEMORY:


Book:
Data
Shelves:
Main
memory
Reader:
Processor
Desk:
Cache
memory
Pan-African History
THE
FOR AFRICA
Pakenham

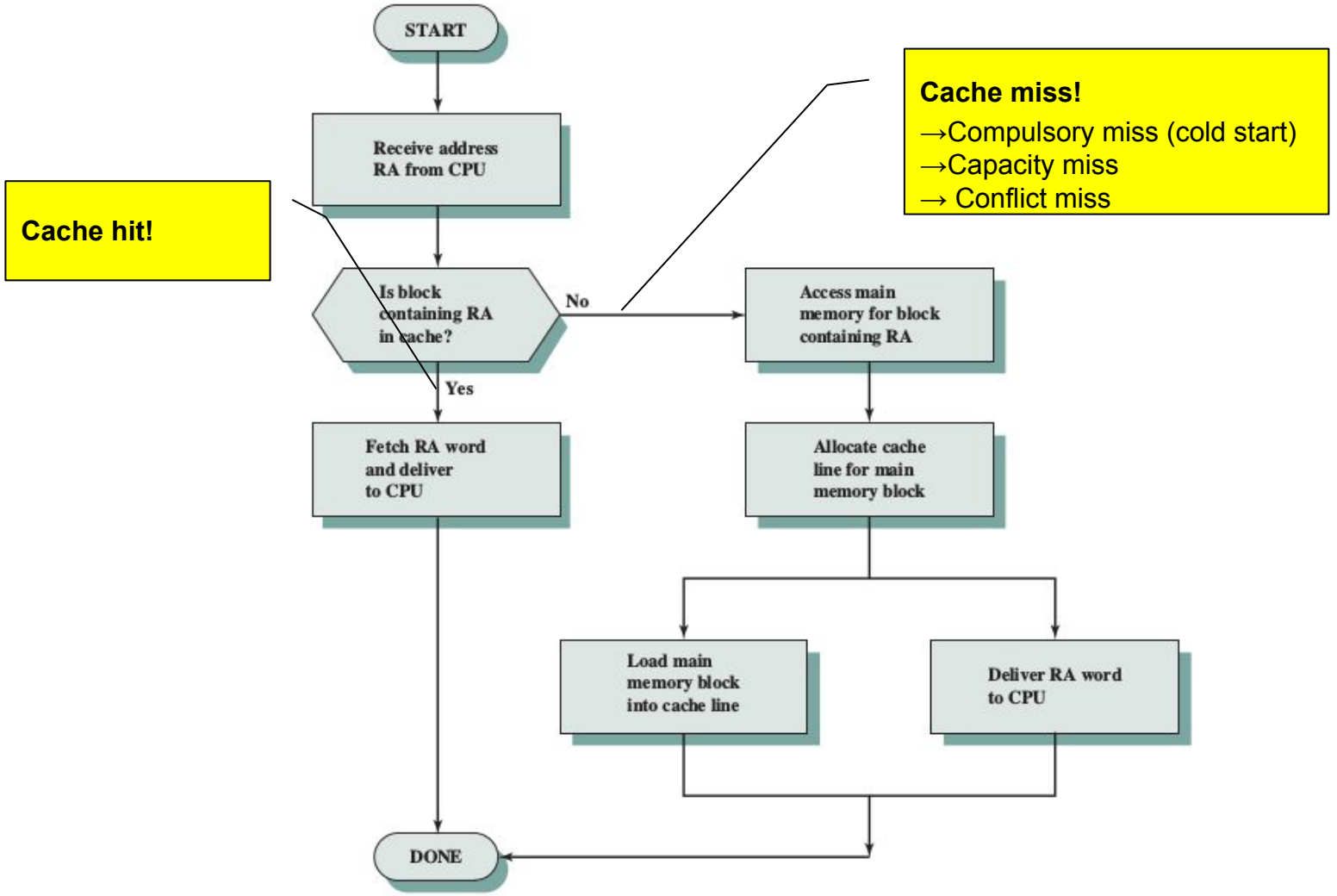
Data Transfer: Memory - Cache, Cache - CPU



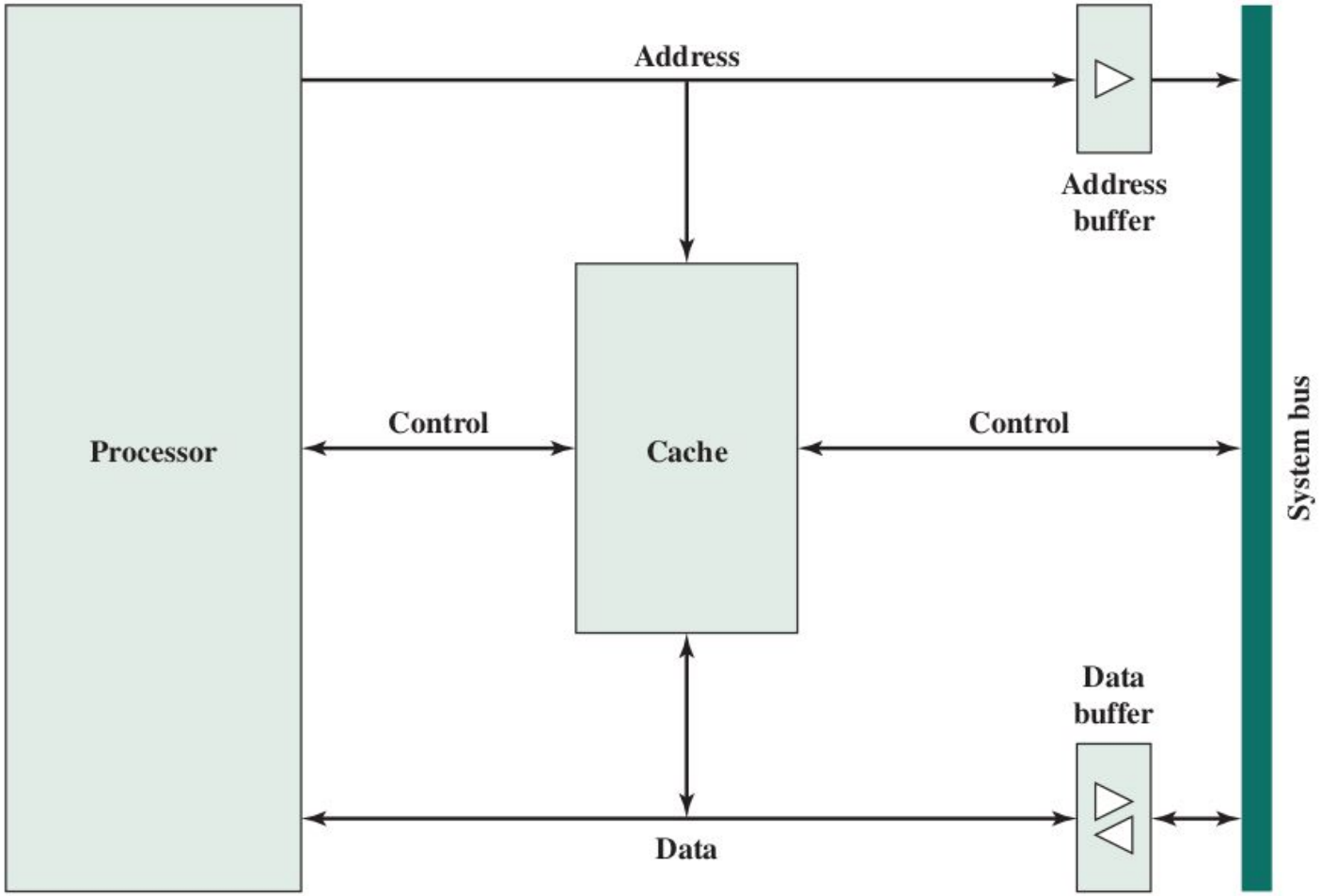
Structure: Cache and Main Memory



Data Transfer in Detail: CPU Read Operation



Typical Cache Organization

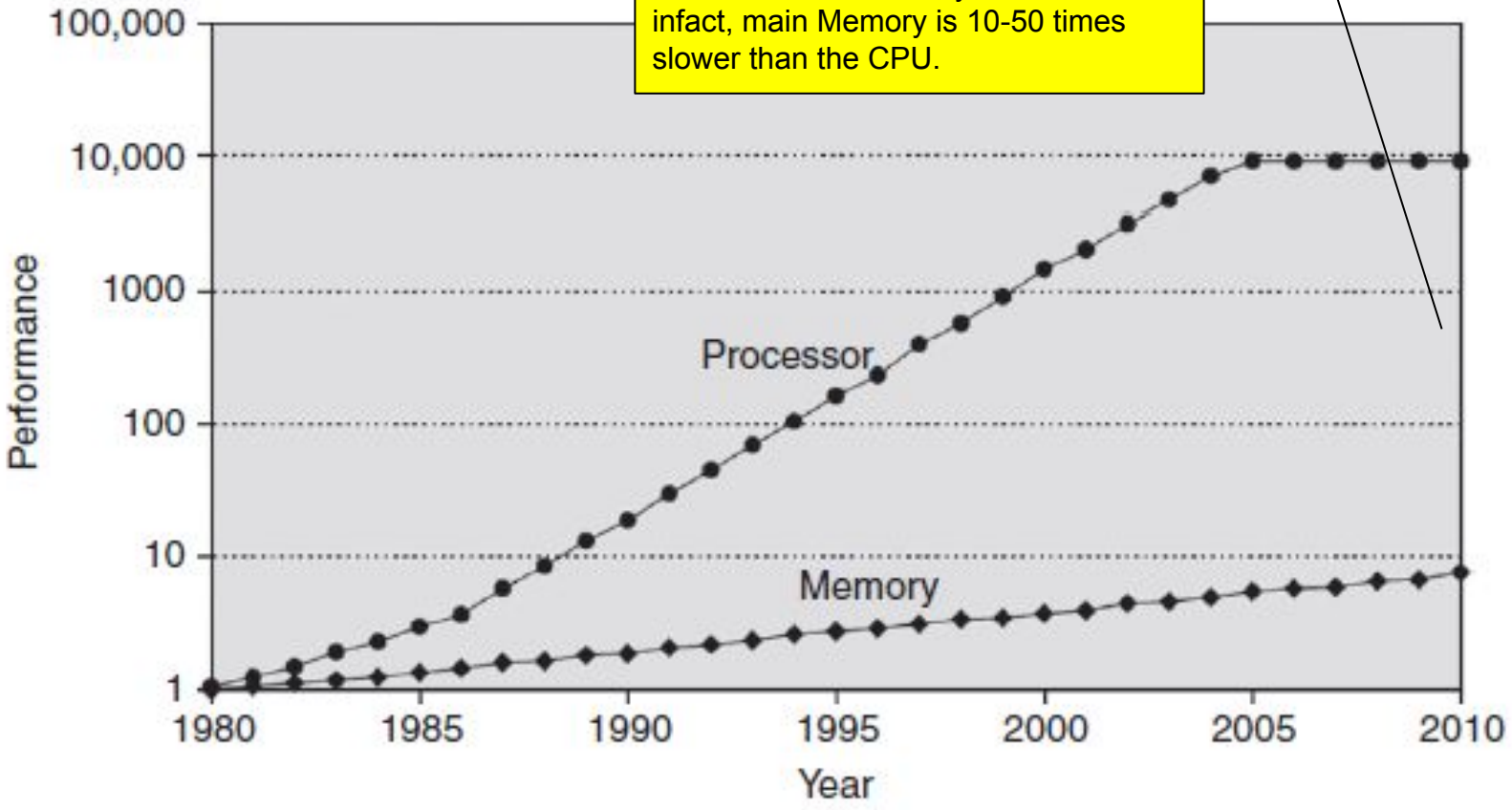


Elements of Cache Design

Cache Addresses	**Write Policy**
Logical	Write through
Physical	Write back
Cache Size	**Line Size**
Mapping Function	**Number of Caches**
Direct	Single or two level
Associative	Unified or split
Set associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

The Goal of Cache: Fast Main Memory Access

The goal of cache memory is to reduce this CPU-memory gap by illuding the CPU that main memory is fast. While infact, main Memory is 10-50 times slower than the CPU.



Keys to Achieving the Goal: Locality of Reference

Is there some kind of **predictability** in the way the reader accesses the books on the main shelves?



Generally, definitely! If she first picked a book on "**Pan-Africanism**", she is highly likely to pick **near-by books** on Africa in the near-future. she is also highly likely to pick **the same book again**!

This is called **locality of reference** principle:
(1) spatial locality - she is highly likely to pick **near-by Books** on Africa in the near-future.
(2) temporal locality - she is also highly likely to pick **the same book again** !

Keys to Achieving the Goal: Locality of Reference

```
27
28  .text                           # add the following stuff to the text segment
29  .globl main
30
31  main:
32          li $t1,0                # initialise sum=0
33          li $t0,0                # initialise n=0
34
35  loop:   add $t1,$t1,$t0         # calculate running sum
36          li $v0, 4               # prompt for n
37          la $a0, msg_promptx
38          syscall
39
40          li $v0, 5               # read in n
41          syscall
42          move $t0, $v0
43
44          bnez $t0, loop          # if input not zero, repeat
45
46          li $v0, 4               # otherwise, print the result message
47          la $a0, msg_result
48          syscall
```

Exercise: Improving Access Time Through Efficient Cache Design

Q: An 11-cycle average memory access time means that the processor spends ten cycles waiting for data for every one cycle actually using that data. What cache miss rate is needed to reduce the average memory access time to 1.5 cycles given the access times in the table below?

Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%

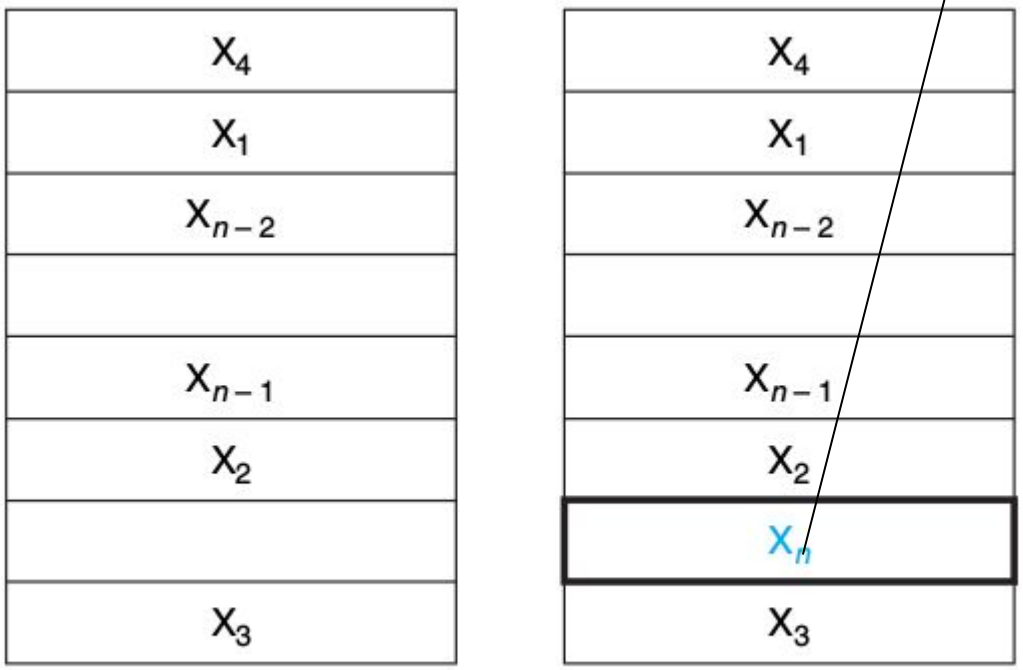
Exercise: Improving Access Time Through Efficient Cache Design

Q: An 11-cycle average memory access time means that the processor spends ten cycles waiting for data for every one cycle actually using that data. What cache miss rate is needed to reduce the average memory access time to 1.5 cycles given the access times in the table below?

Memory Level	Access Time (Cycles)	Miss Rate
Cache	1	10%
Main Memory	100	0%

A: If the miss rate is *m*, the average access time is 1 + 100*m*. Setting this time to 1.5 and solving for m requires a cache miss rate of 0.5%.

Mapping Strategies: Overview



a. Before the reference to X_n

b. After the reference to X_n

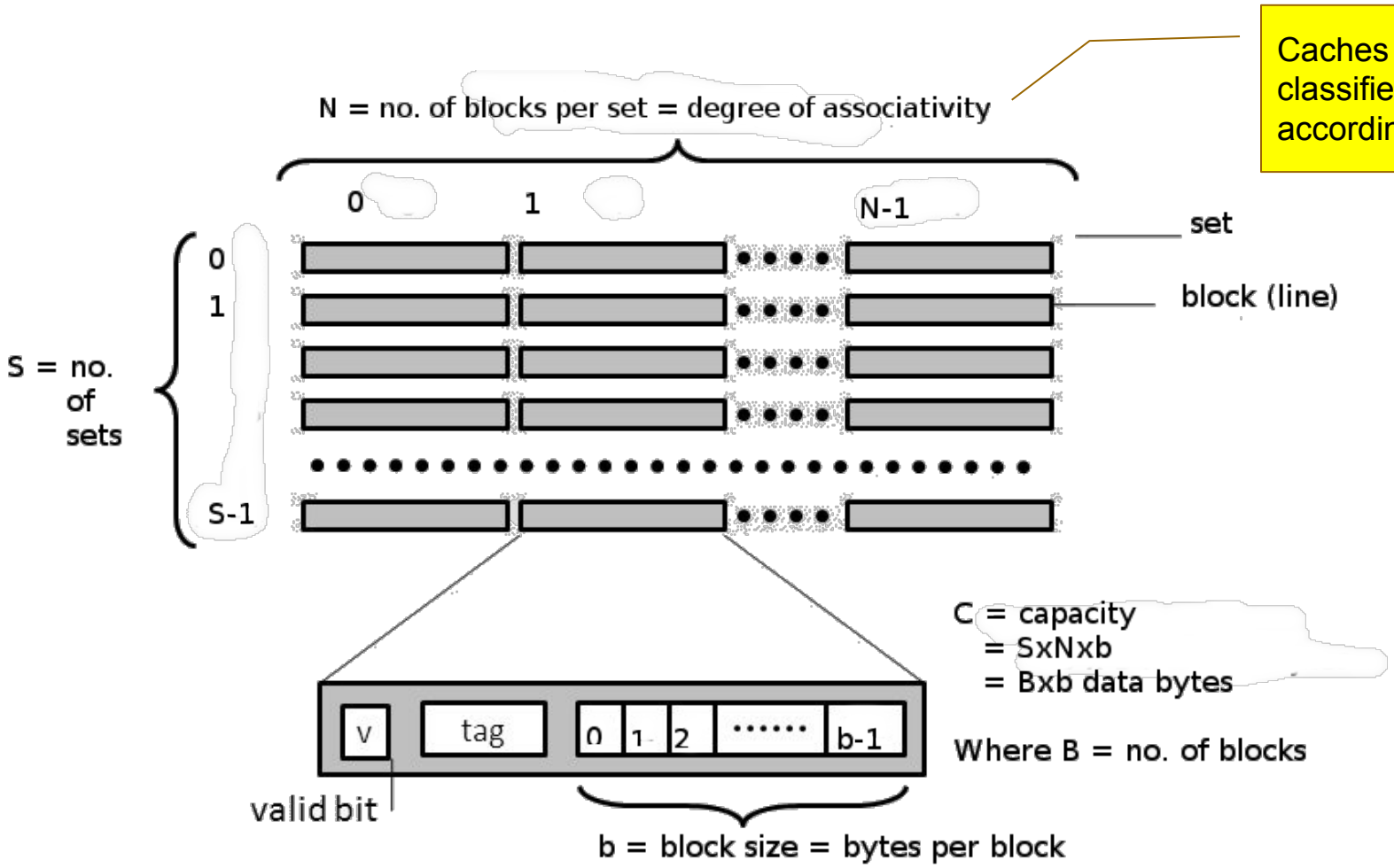
(1) Given a memory address,

how do we know if a data item (X_n), is in the cache or not?

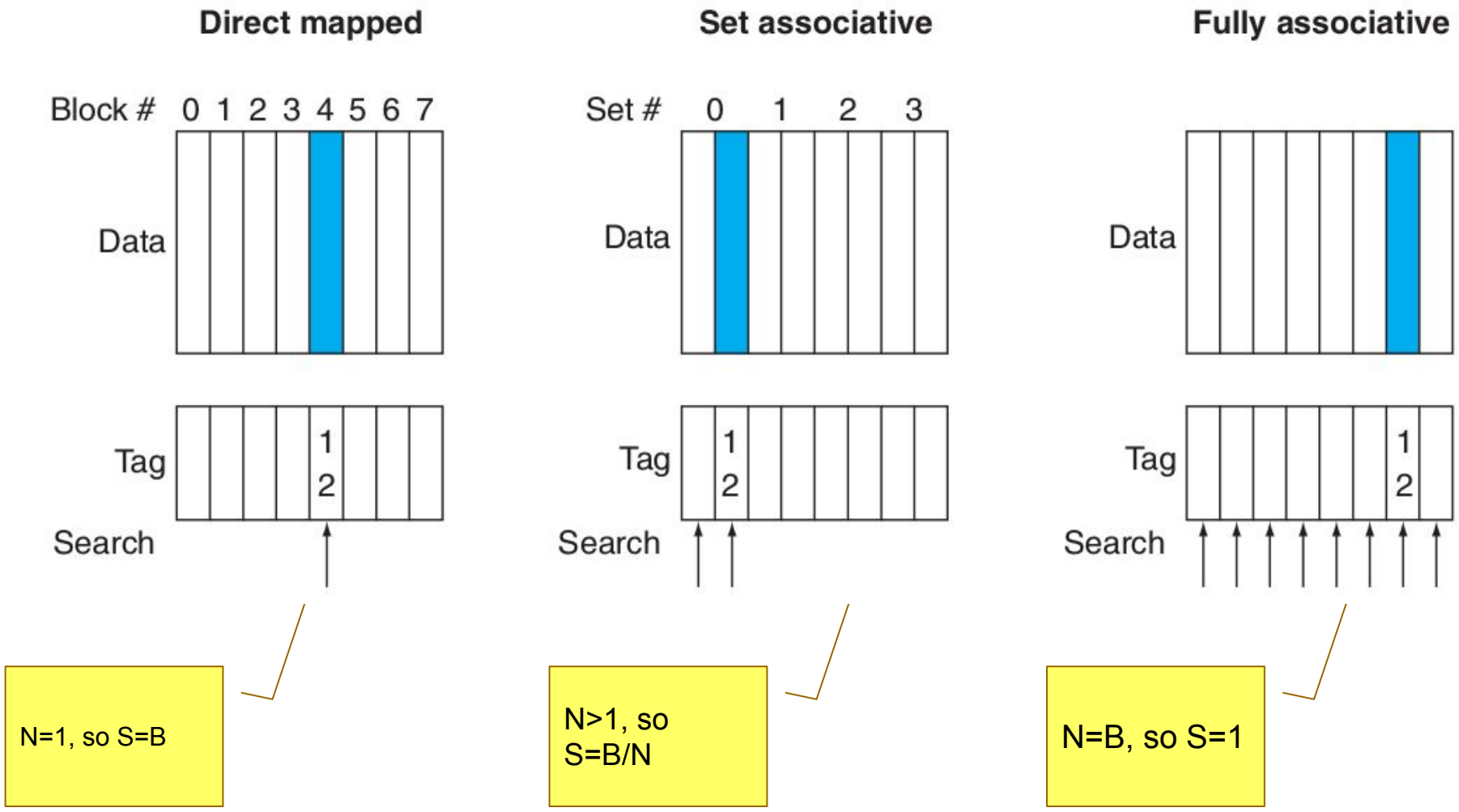
(2) If it is, **how do we find it?**

→There must a memory-cache address mapping scheme.
→The simplest way is to assign a cache location based on the address of the word in memory (**direct map**).
→Another way is to let any memory word reside anywhere in the cache (**fully associative**).
→Use a hybrid of direct map And associative (**set associative**)

General Cache Organization: C = *f(S, N, b) = f(B,b)*



Mapping Strategies: Overview



Mapping Strategies: Direct Mapping Analogy

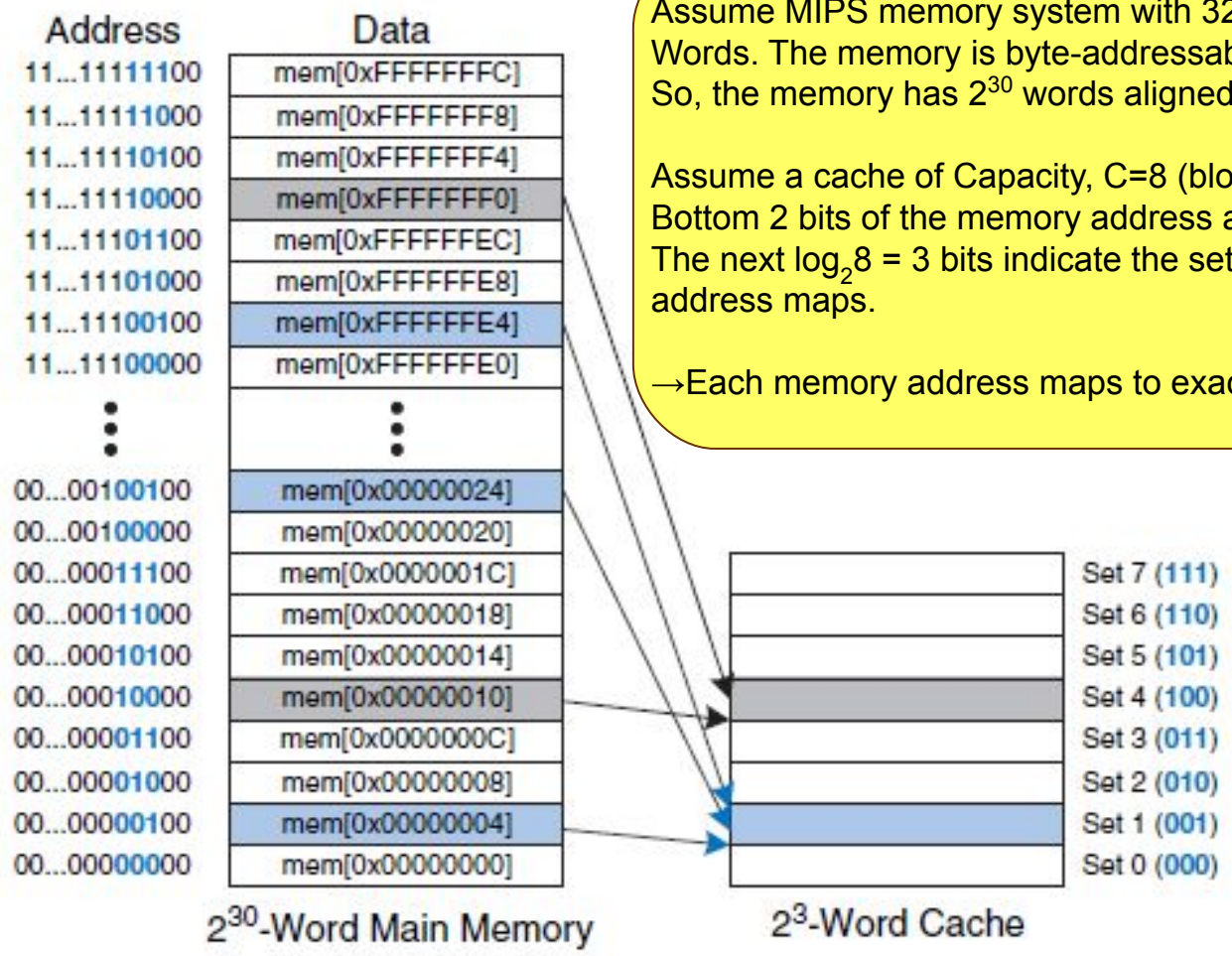




Imagine there are 26 locations on the desk to store a block of books from a shelf. A block's desk location is determined by the first letter of the book titles. Strictly one block of books can occupy each location at a time.

→Each book has exactly one possible location.

Direct Mapping: Memory System

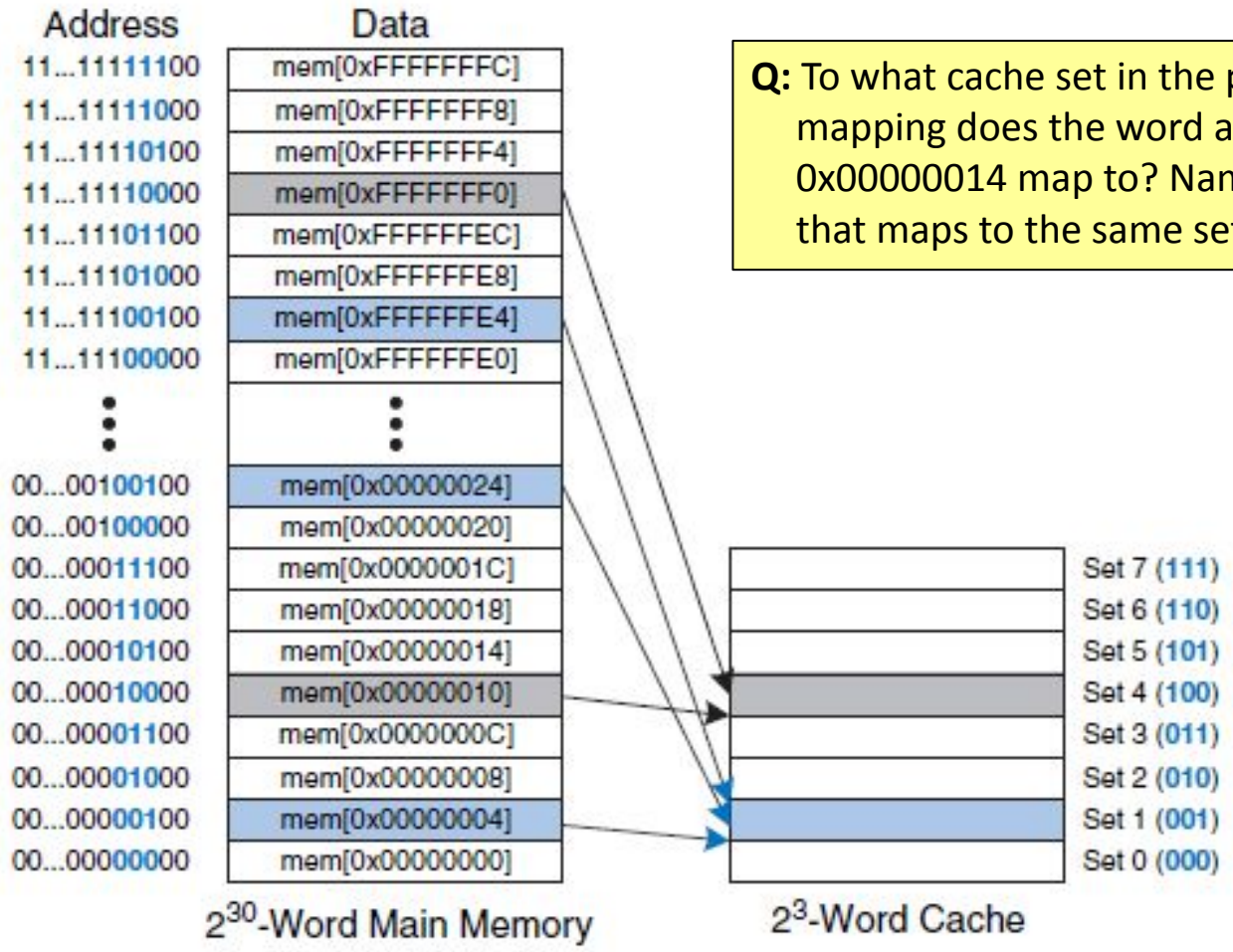


Assume MIPS memory system with 32-bit addresses and 32-bit Words. The memory is byte-addressable, each word is four bytes. So, the memory has 2^{30} words aligned on word boundaries.

Assume a cache of Capacity, C=8 (blocks) and one word block size (b). Bottom 2 bits of the memory address always 00, due to alignment. The next $\log_2 8 = 3$ bits indicate the set onto which the memory address maps.

→Each memory address maps to exactly one set in the cache.

Exercise: Direct Mapping

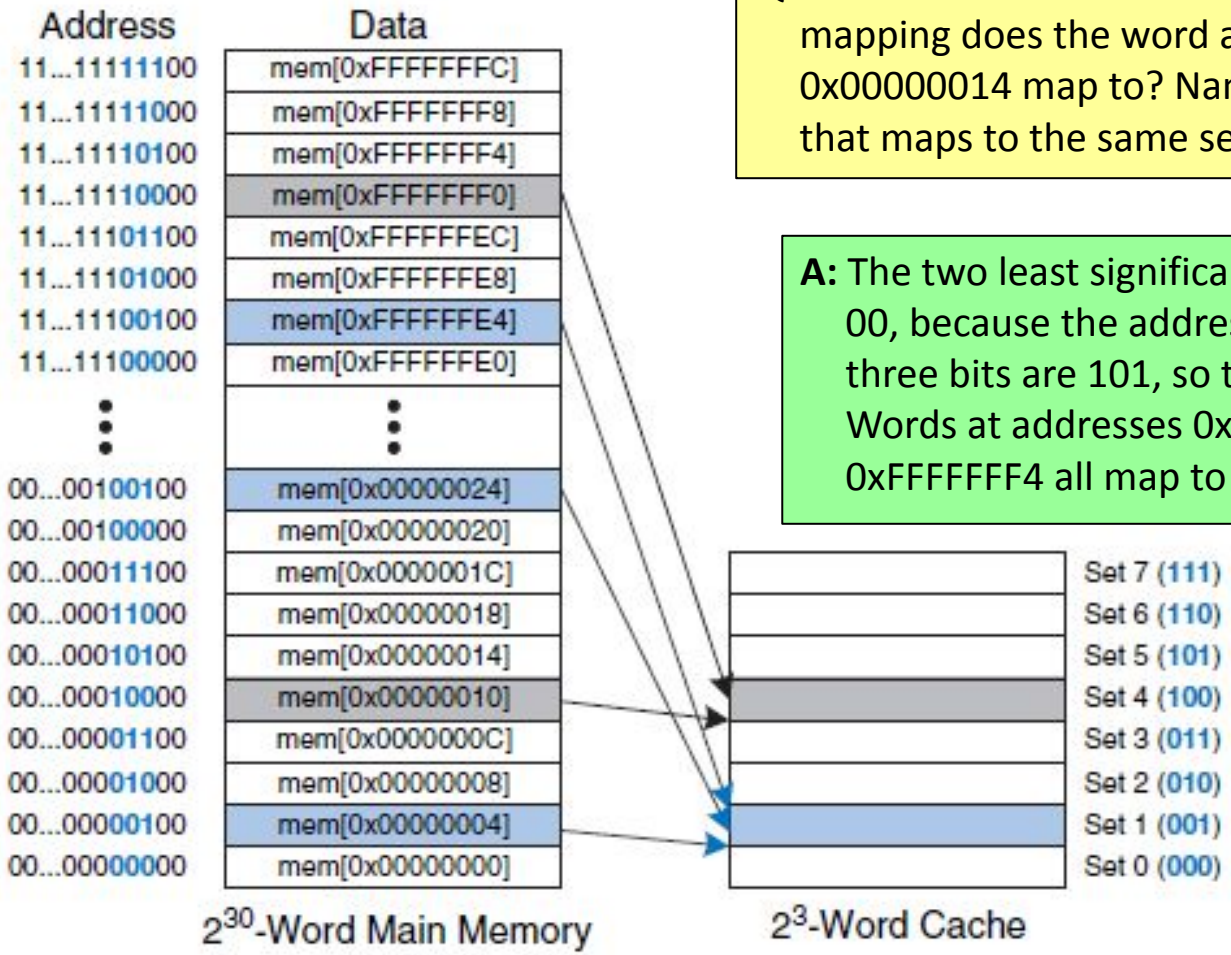


Q: To what cache set in the previous cache mapping does the word at address 0x00000014 map to? Name another address that maps to the same set.

Exercise: Direct Mapping

Q: To what cache set in the previous cache mapping does the word at address 0x00000014 map to? Name another address that maps to the same set.

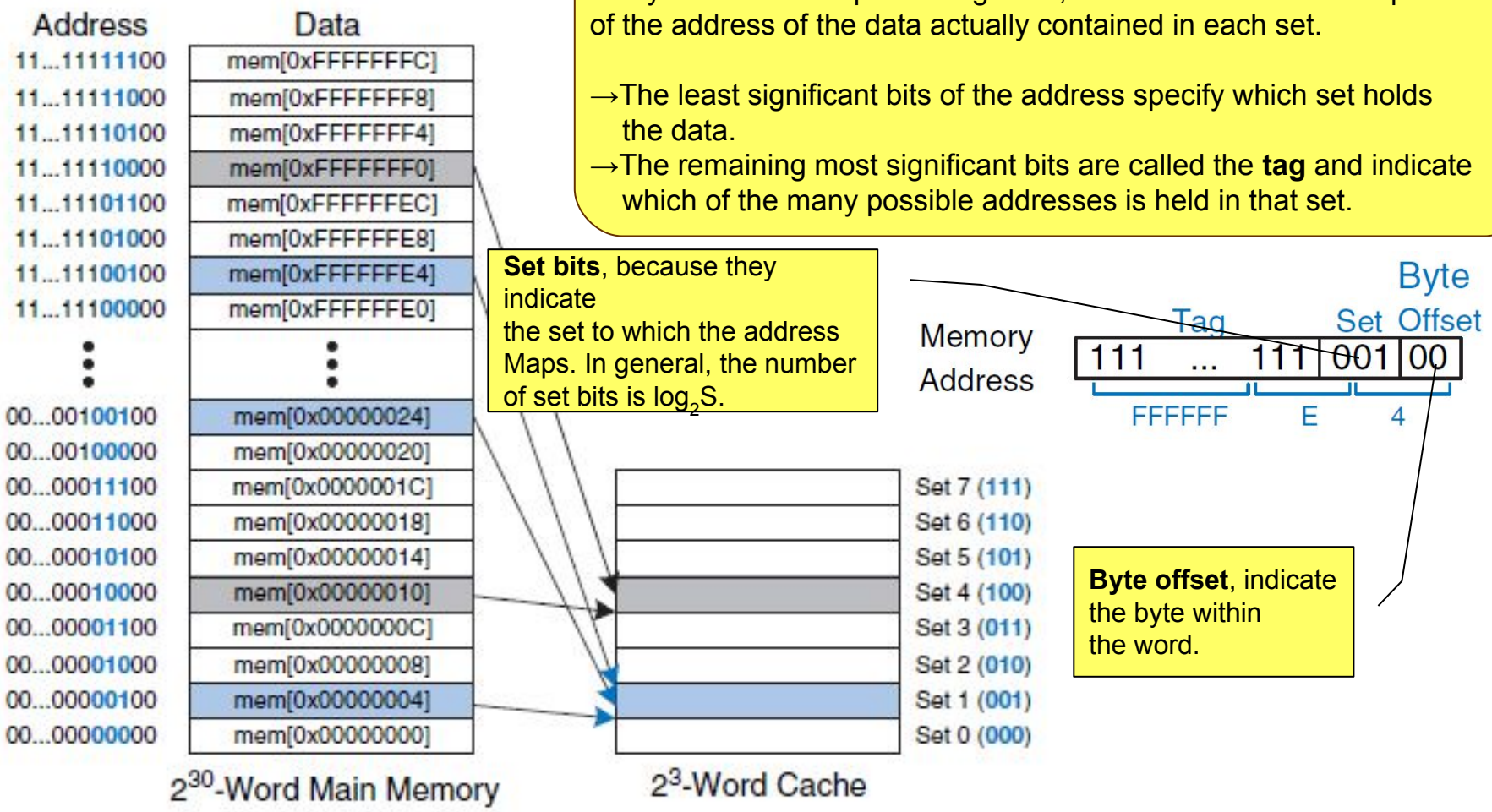
A: The two least significant bits of the address are 00, because the address is word aligned. The next three bits are 101, so the word maps to set 5. Words at addresses 0x34, 0x54, 0x74, . . . , 0xFFFFFFF4 all map to this same set.



Direct Mapping: Cache Fields

Many addresses map to a single set, the cache must also keep track of the address of the data actually contained in each set.

→The least significant bits of the address specify which set holds the data.
→The remaining most significant bits are called the **tag** and indicate which of the many possible addresses is held in that set.



Exercise: Direct Mapping

Q: Find the number of set and tag bits for a direct mapped cache with 1024 (2^{10}) sets and a one-word block size. The address size is 32 bits.

Exercise: Direct Mapping

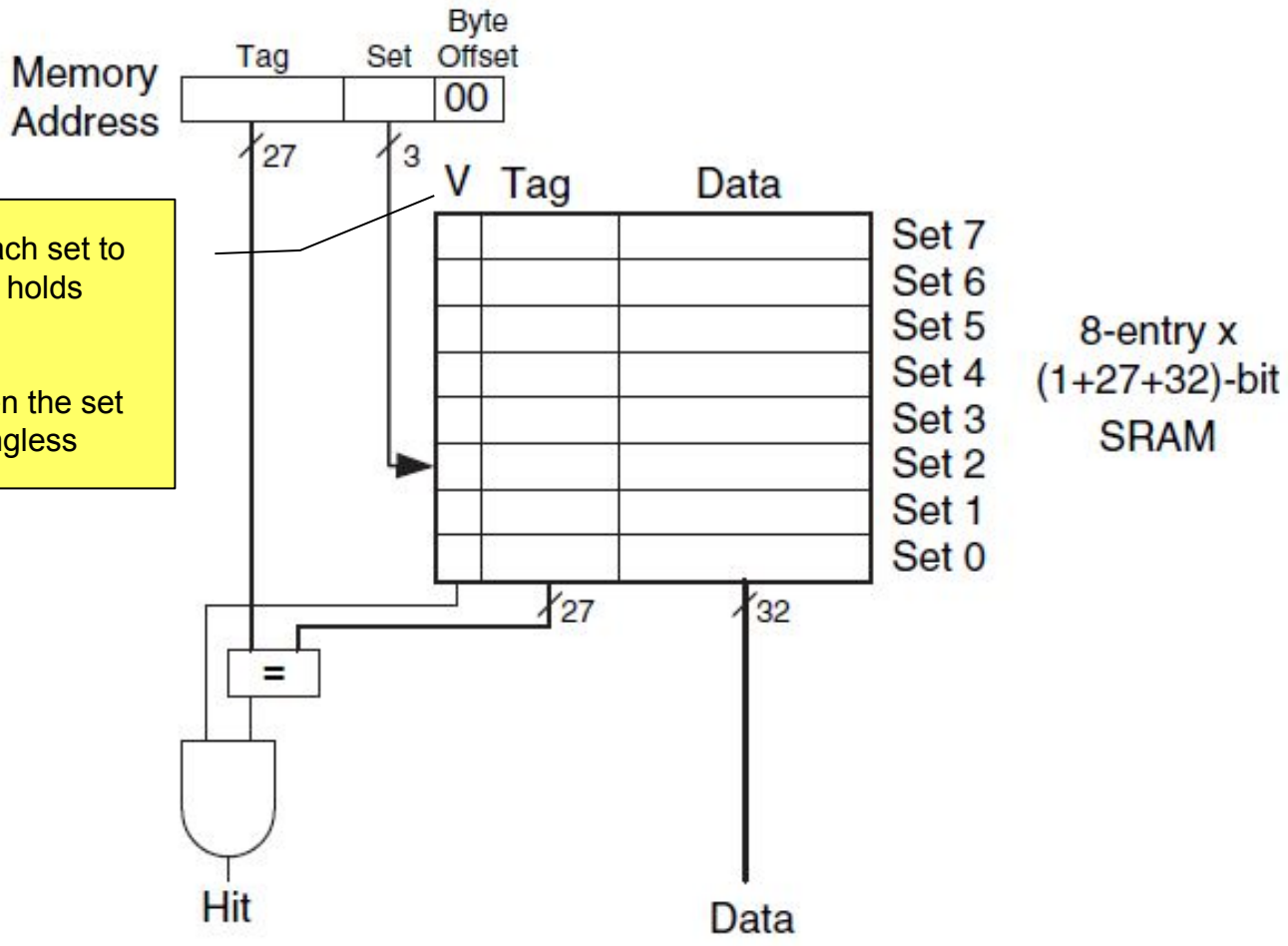
Q: Find the number of set and tag bits for a direct mapped cache with 1024 (2^{10}) sets and a one-word block size. The address size is 32 bits.

A: A cache with 2^{10} sets requires $\log_2(2^{10}) = 10$ set bits. The two least significant bits of the address are the byte offset, and the remaining $32 - 10 - 2 = 20$ bits form the tag.

Direct Mapping: Hardware Implementation

Valid bit used for each set to indicate whether set holds valid data or not.

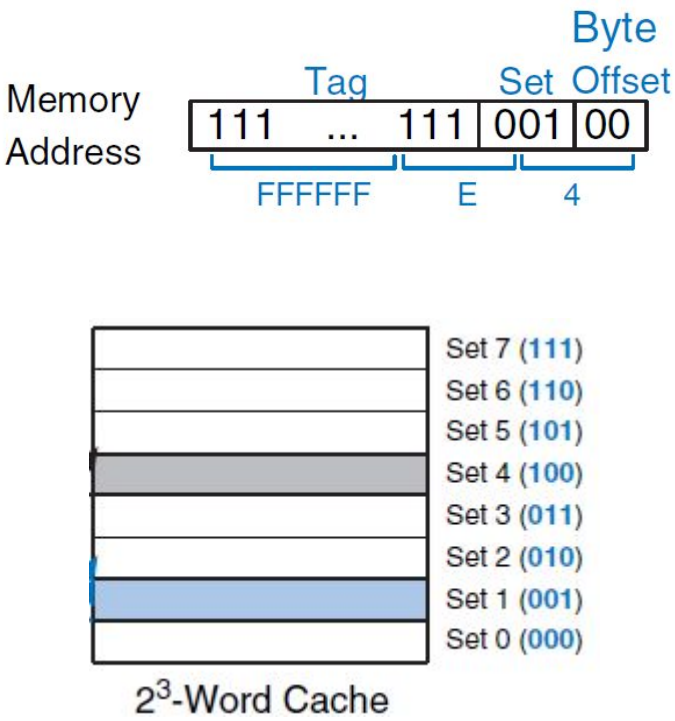
→If valid bit is 0, then the set contents are meaningless



Exercise: Temporal Locality with a Direct Mapped Cache

Q: Loops are a common source of temporal and spatial locality in applications. Using the eight-entry cache describe earlier, show the contents of the cache after executing the following silly loop in MIPS assembly code. Assume that the cache is initially empty. Make any other necessary assumptions. What is the miss rate?

```
        addi $t0, $0, 5
loop: beq $t0, $0, done
        lw $t1, 0x4($0)
        lw $t2, 0xC($0)
        lw $t3, 0x8($0)
        addi $t0, $t0, −1
        j loop
done:
```



Exercise: Temporal Locality with a Direct Mapped Cache

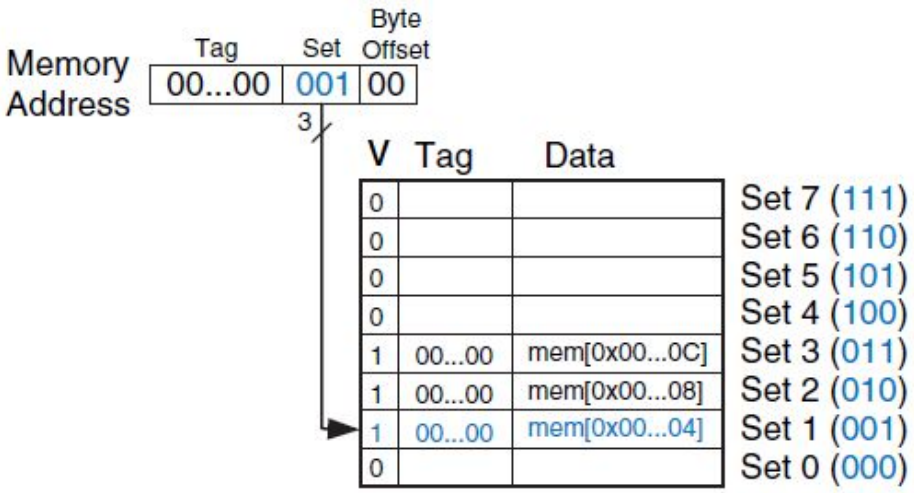
A:

(1) The program involves a loop that iterates for 5 times. There are three lw instructions in each iteration, therefore the loop has a total of 15 (3x5) memory accesses.

(2) Initially, the cache is empty and each of the first three memory accesses to addresses 0x4, 0xC and 0x8 results in data fetched from main memory into cache sets 1, 3 and 2 respectively.

(3) The next four time the loop executes, the data is found in the cache.

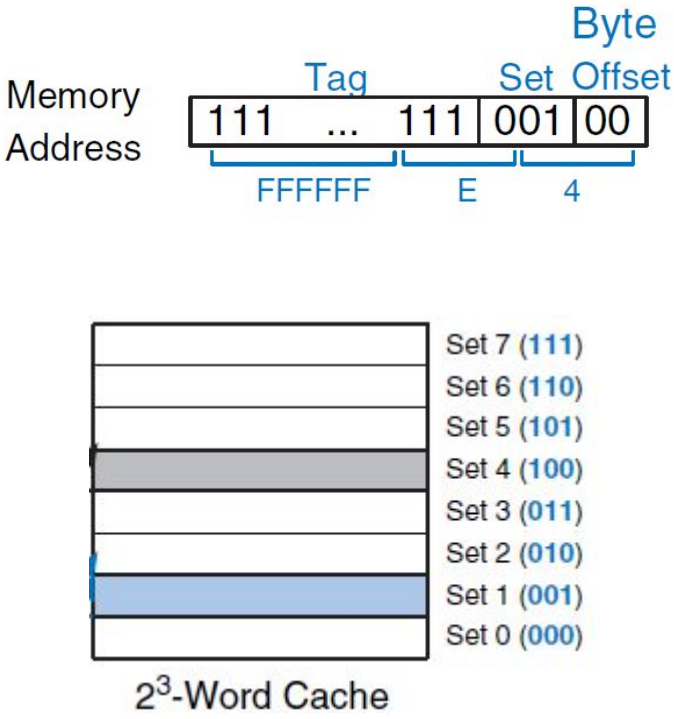
Therefore the miss rate is 3/15 = 20%. The figure below shows the contents of the cache during the last request to memory address 0x4.



Exercise: Cache Block Conflict on Direct Mapped Cache

Q: What is the miss rate when the following loop is executed on the eight-word direct mapped cache described previously? Assume that the cache is initially empty.

```
        addi $t0, $0, 5
loop: beq $t0, $0, done
        lw $t1, 0x4($0)
        lw $t2, 0x24($0)
        addi $t0, $t0, –1
        j loop
done:
```



Exercise: Cache Block Conflict on Direct Mapped Cache

A:

(1) The program involves 5 iterations. There are two memory accesses per iteration, therefore, a total of 10 memory accesses.

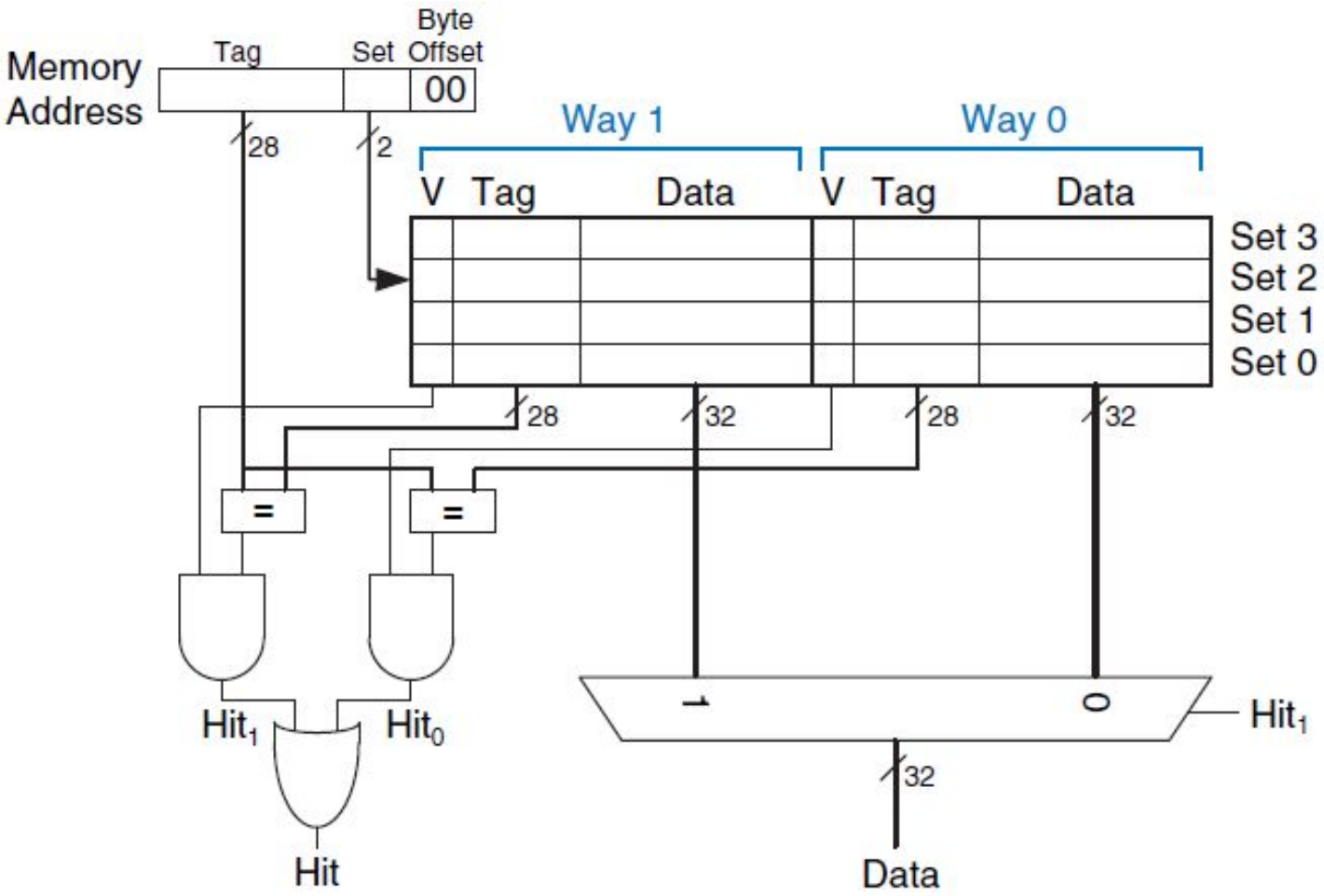
(2) The cache is initially empty, therefore, the access to memory address 0x4 results in the fetching of data from main memory into the cache set 1.

(3) The next access to address 0x24 also maps to cache set 1, therefore since this is a direct mapped cache (N=1), the old block of data is evicted to make space for the new one.

(4) Upon the second iteration of the loop, the process repeats and the cache must fetch data from 0x4 and therefore must evict data from 0x24.

Therefore, the miss rate is 10/10=100%

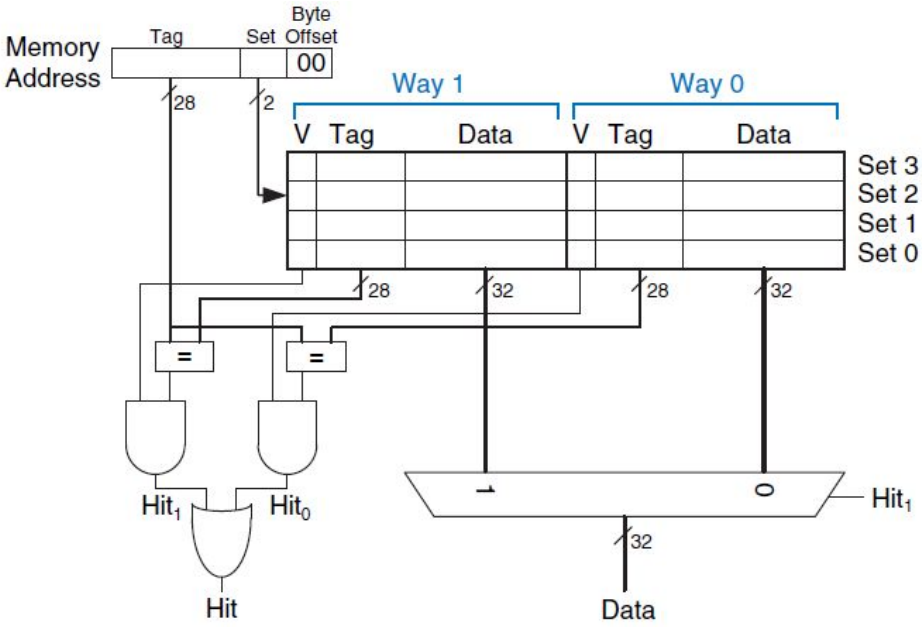
Mapping Strategies: Set-Associative Mapping



Exercise: Set Associative Cache Miss Rate

Q: What is the miss rate when the following loop is executed on the eight-word two-way set-associative cache shown on the right? Assume that the cache is initially empty.

```
        addi $t0, $0, 5
loop: beq $t0, $0, done
        lw $t1, 0x4($0)
        lw $t2, 0x24($0)
        addi $t0, $t0, –1
        j loop
done:
```



Exercise: Set Associative Cache Miss Rate

A: Similarly, there are 10 memory accesses in total and the cache is initially empty.

(1) Both memory addresses 0x4 and 0x24 map to cache set 1.

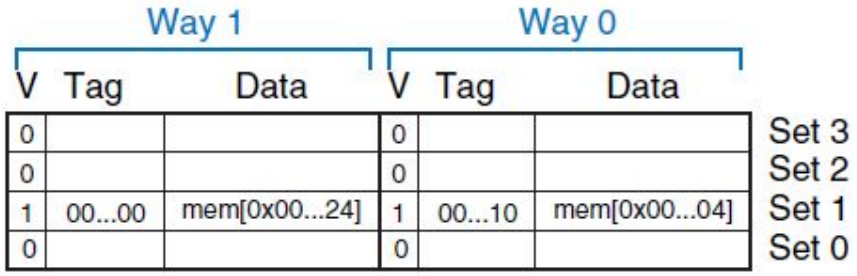
(2) However, the cache has two ways, therefore can accommodate data from both addresses.

(3) Thus, during the first loop iteration, there are two cache misses and therefore the cache loads both words into the two ways of set 1.

(4) On the four subsequent iterations, the cache hits.

(5) Therefore, the miss rate is 2/10 = 20%.

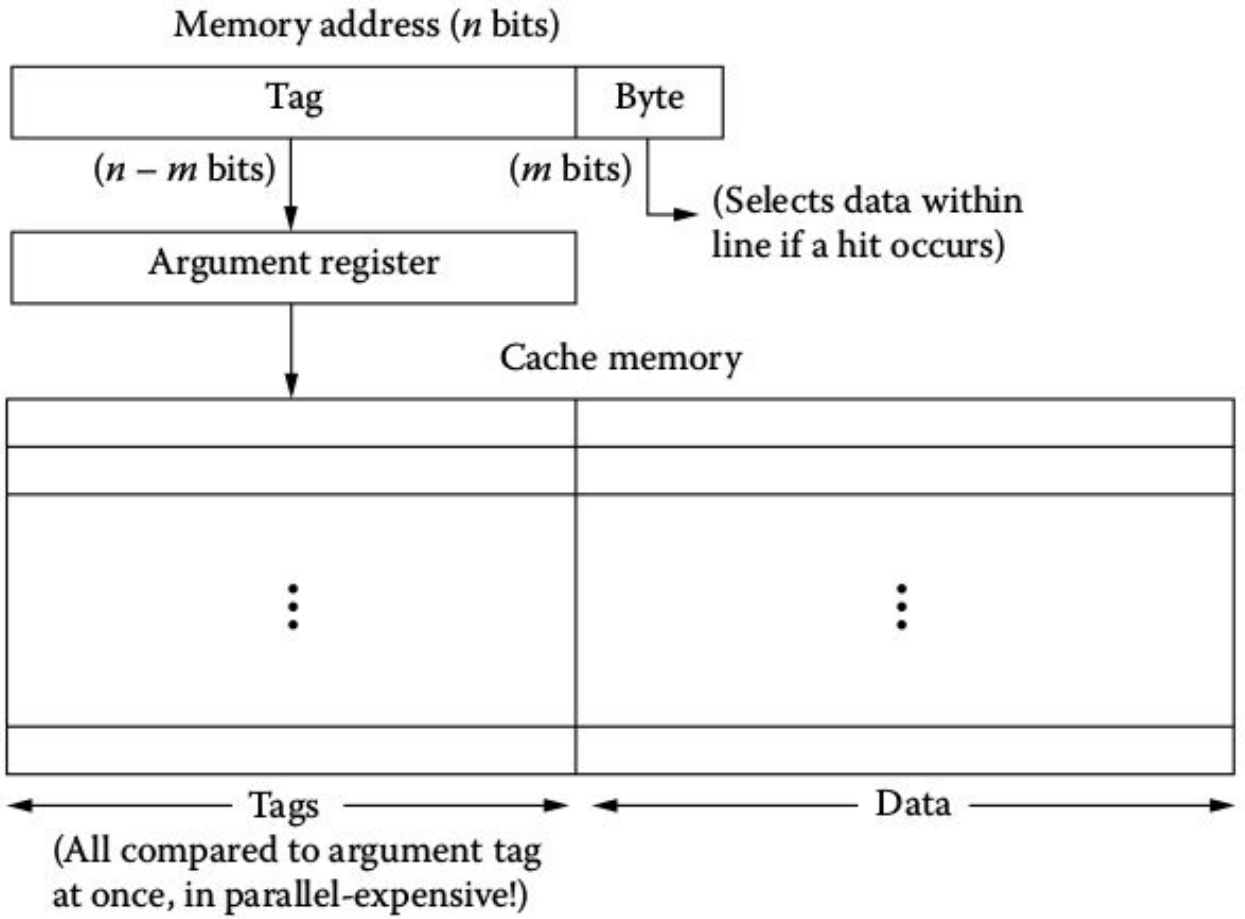
(6) Recall that the direct mapped cache of the same size had a miss rate of 100%.



Mapping Strategies: Fully-Associative Mapping



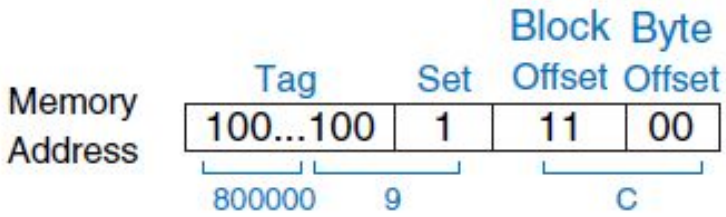
Mapping Strategies: Fully-Associative Cache Hardware

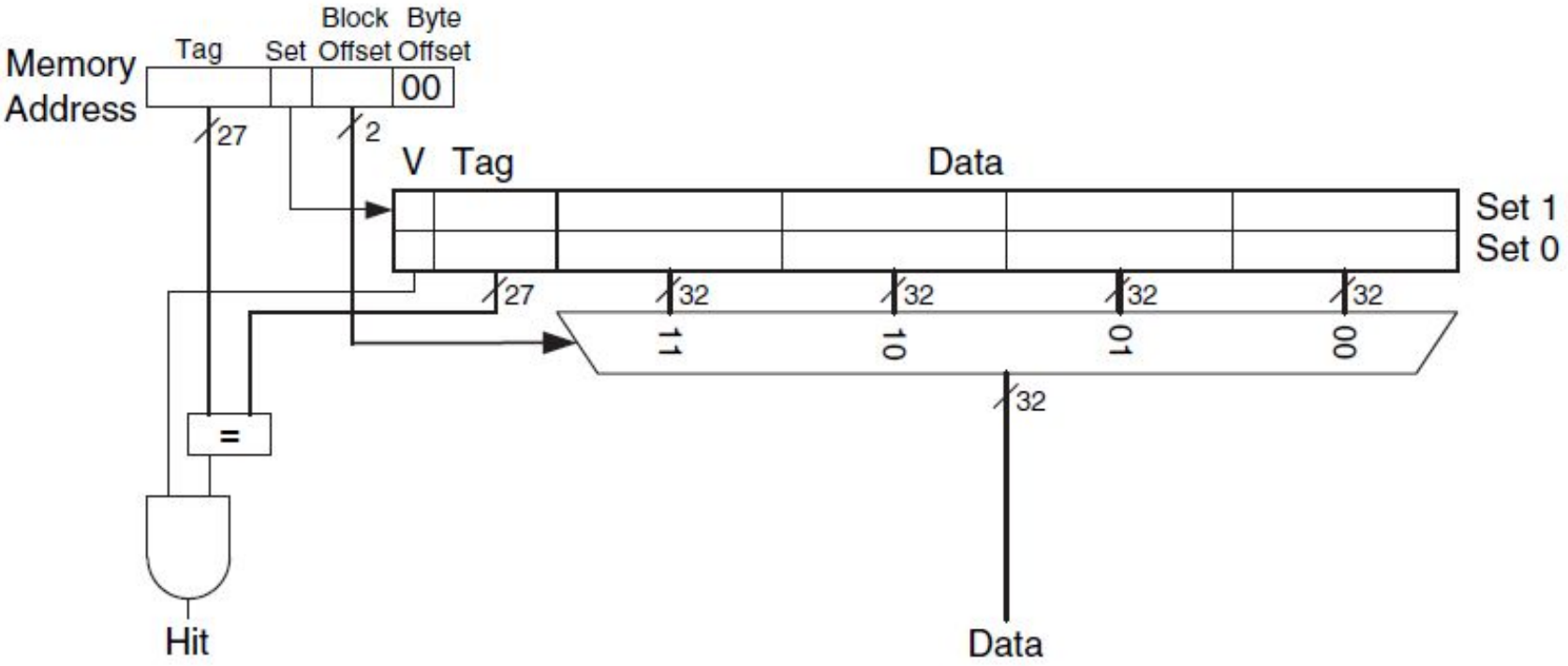


Larger Block Size and Spatial Locality

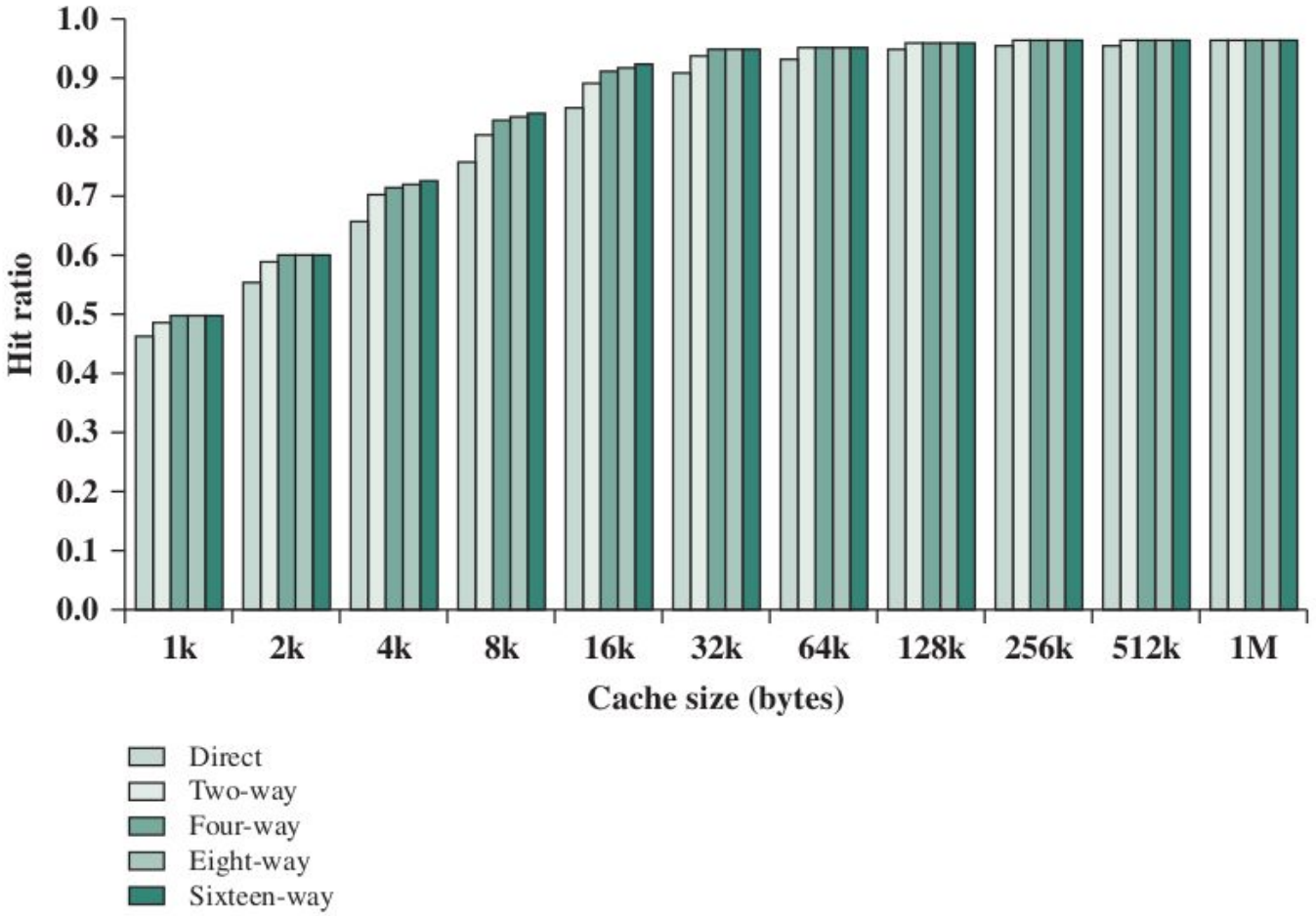
- Previous examples were able to take advantage only of temporal locality
  - Block size was one word.
- To exploit spatial locality, a cache uses larger blocks to hold several consecutive words.
- Advantages
  - When a miss occurs and a word is fetched, adjacent words also fetched. Therefore, possibility of increased hit rate due to spatial locality.
- Disadvantages
  - Large block size means a fixed-size cache will have fewer blocks. This may lead to more conflicts, increasing the miss rate.
  - Increased miss penalty due to large block size
- Most practical programs benefit from larger block sizes!

Direct Mapped Cache: Four Word Block Size





Cache Performance Comparison

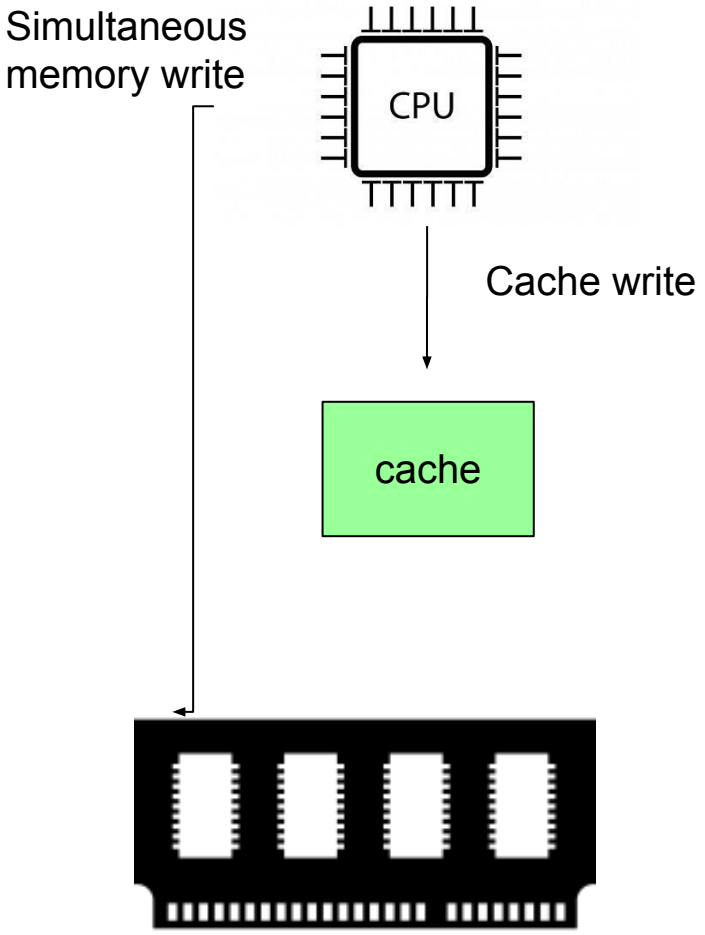


Cache Write Policies: Overview

- Cache operation straight-forward as long as memory accesses are limited to reads
  - That is, in the case of an instruction only cache
- However, practical cache design typically includes write access to cached locations
  - Data cache in a system with a split cache architecture (Harvard)
  - Instruction/data cache in a system with a unified cache archiecture (Von Neumann)
- Cache write accesses complicates cache design because of what must happen when the a cache block is displaced from the cache
  - If the contents of that block have only been read, no problem. The block can simply be over-written by the new data
  - But if any location in that block has been modified, we must make sure main memory reflects the updated contents before loading new data in their place
- This can be done in two ways
  - Write through
  - Write back

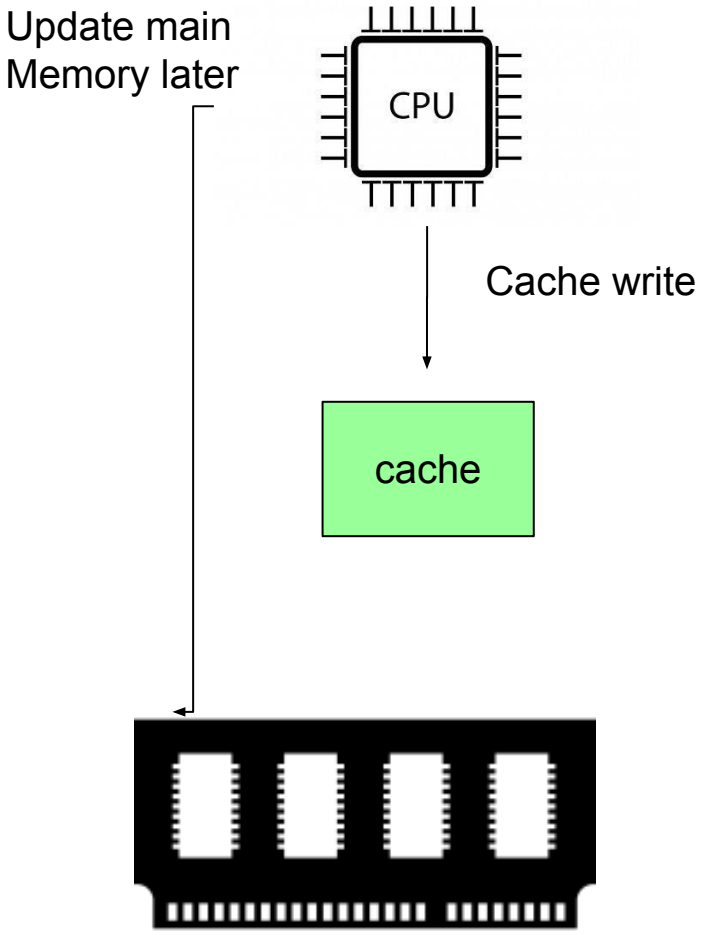
Write Policies: Write-Through

- Every time we write to a location in cache, we also perform a write to main memory
  - These two writes can usually be started at the same time
  - Main memory write takes longer and thus determines the overall time for the write operation.
- Advantages
  - Simple to build into the hardware
  - Main memory always has most current data
- Disadvantages
  - Multiple cache accesses result in extraneous write exact a performance penalty

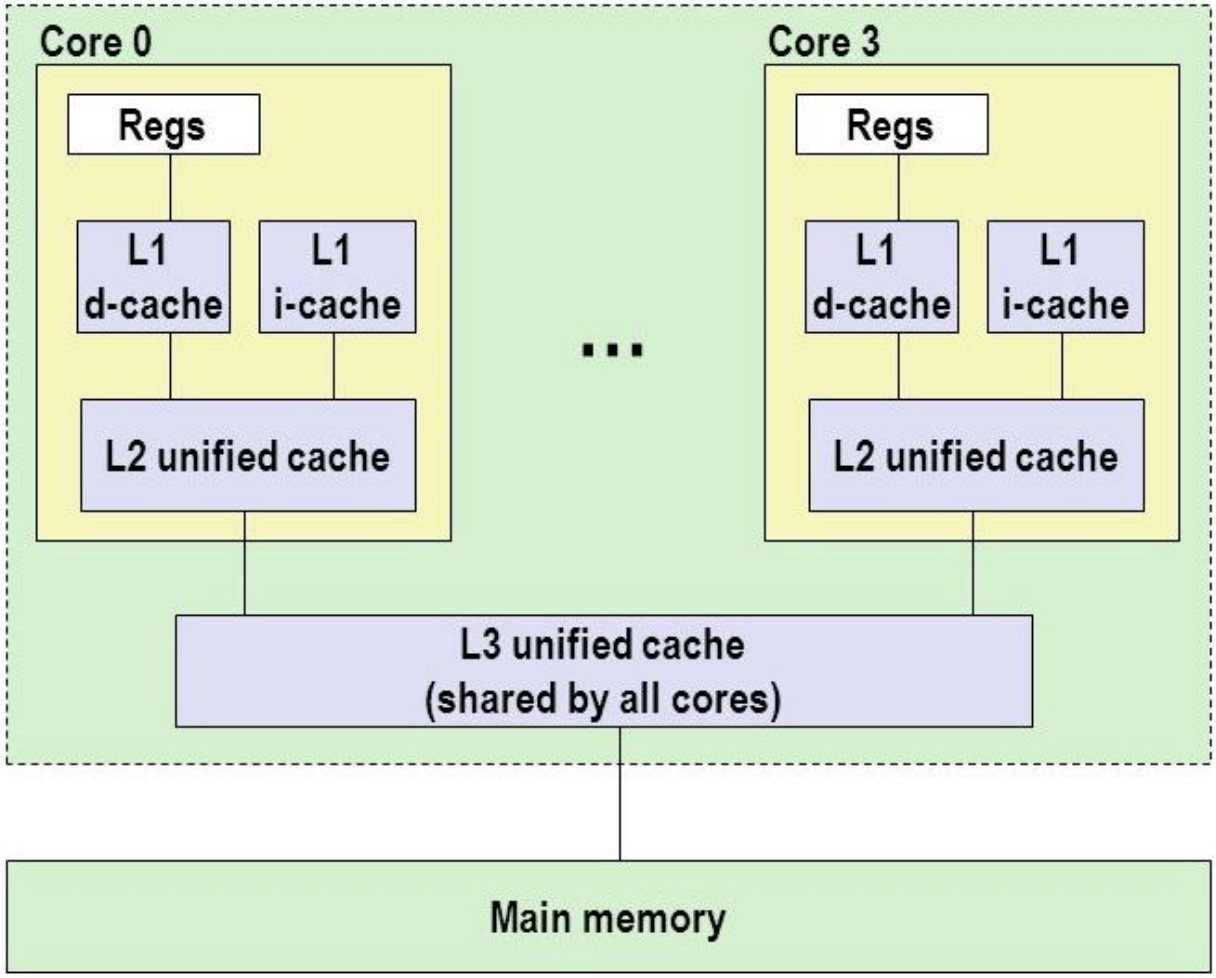


Write Policies: Write-Back

- On a write that hits the cache, only the cache location is updated.
- Main memory is only written when a block that has been modified is displaced from the cache to make room for a new block to be loaded.
- Advantages
  - Can improve performance if write done frequently
- Disadvantages
  - More complex to implement (e.g require extra “dirty bits” and logic to examine each word)
  - Data in main memory stale



Intel Core i7 Cache Hierarchy



L1 i-cache and d-cache:
32 KB, 8-way,
Access: 4 cycles

L2 unified cache:
256 KB, 8-way,
Access: 11 cycles

L3 unified cache:
8 MB, 16-way,
Access: 30-40 cycles

Block size: 64 bytes for all caches.

MIPS Cache Evolution

Year	CPU	MHz	L1 Cache	L2 Cache
1985	R2000	16.7	none	none
1990	R3000	33	32 KB direct mapped	none
1991	R4000	100	8 KB direct mapped	1 MB direct mapped
1995	R10000	250	32 KB two-way	4 MB two-way
2001	R14000	600	32 KB two-way	16 MB two-way
2004	R16000A	800	64 KB two-way	16 MB two-way
2010	MIPS32 1074K	1500	32 KB	variable size

* Adapted from D. Sweetman, *See MIPS Run*, Morgan Kaufmann, 1999.

Modern Cache Characteristics: General Purpose and Mobile/Embedded Processors

	ISA	MICROARCHITECTURE	NUMBER OF CORES	CACHE
AMD PHENOM [11], [15]	X86	THREE-WAY OUT-OF-ORDER SUPERSCALAR, 128-B SIMD	FOUR	64 KB IL1 AND DL1/ CORE, 256 KB L2/CORE, 2-6 MB L3
INTEL CORE I7 [2], [5]	X86	FOUR-WAY OUT-OF-ORDER, TWO-WAY SMT, 128-B SIMD	TWO TO EIGHT	32 KB IL1 AND DL1/ CORE, 256 KB L2/CORE, 8 MB L3
SUN NIAGARA T2 [16], [17]	SPARC	TWO-WAY IN-ORDER, EIGHT-WAY SMT	EIGHT	16 KB IL1 AND 8 KB DL1/ CORE, 4 MB L2
INTEL ATOM [18], [5]	X86	TWO-WAY IN-ORDER, TWO-WAY SMT, 128-B SIMD	ONE TO TWO	32 KB IL1 AND DL1/ CORE, 512 KB L2/CORE
ARM CORTEX-A9† [6]	ARM	THREE-WAY OUT-OF-ORDER	ONE TO FOUR	(16,32,64) KB IL1 AND DL1/CORE, UP TO 2 MB L2
XMOS XS1-G4 [19]	XCORE	ONE-WAY IN-ORDER, EIGHT-WAY SMT	FOUR	64 KB LCL STORE/CORE







Modern Cache Characteristics: High Performance Processors

	ISA	MICROARCHITECTURE	NUMBER OF CORES	CACHE
AMD RADEON R700 [20]	N/A	FIVE-WAY VLIW	160 CORES, 16 CORES PER SIMD BLOCK, TEN BLOCKS	16 KB LCL STORE/SIMD BLOCK
NVIDIA G200 [8], [21]	N/A	ONE-WAY IN-ORDER	240, EIGHT CORES PER SIMD UNIT, 30 SIMD UNITS	16 KB LCL STORE/EIGHT CORES
INTEL LARRABEE† [22]	X86	TWO-WAY IN-ORDER, 4-WAY SMT, 512-B SIMD	UP TO 48†	32 KB IL1 AND 32 KB DL1/ CORE, 4 MB L2
IBM CELL [9], [23]	POWER	TWO-WAY IN-ORDER, TWO-WAY SMT PPU, 2-WAY IN-ORDER 128-B SIMD SPU	1 PPU, EIGHT SPUs	PPU: 32 KB IL1 AND 32 KB DL1, 512 KB L2; SPU: 256 KB LCL STORE
MICROSOFT XENON [10]	POWER	TWO-WAY IN-ORDER, TWO-WAY SMT, 128-B SIMD	THREE	32 KB IL1 AND 32 KB DL1/ CORE, 1 MB L2





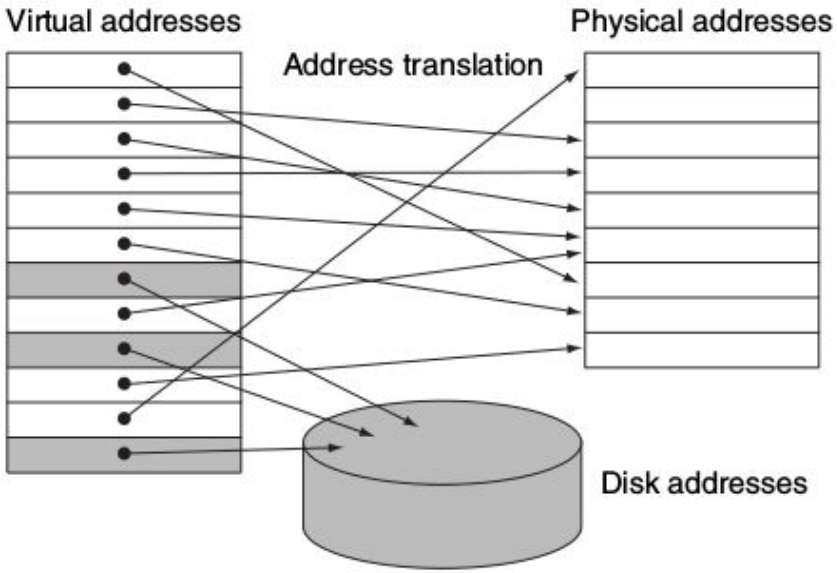
Intel Cache Evolution

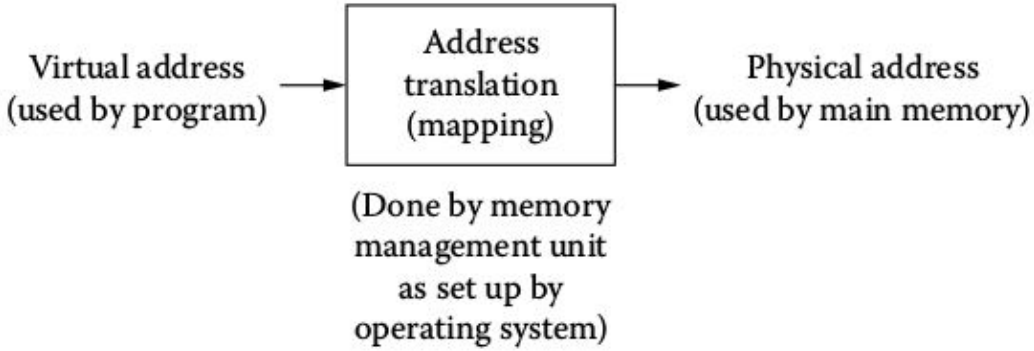
Problem	Solution	Processor on Which Feature First Appears
External memory slower than the system bus.	Add external cache using faster memory technology.	386
Increased processor speed results in external bus becoming a bottleneck for cache access.	Move external cache on-chip, operating at the same speed as the processor.	486
Internal cache is rather small, due to limited space on chip.	Add external L2 cache using faster technology than main memory.	486
Contention occurs when both the Instruction Prefetcher and the Execution Unit simultaneously require access to the cache. In that case, the Prefetcher is stalled while the Execution Unit's data access takes place.	Create separate data and instruction caches.	Pentium
Increased processor speed results in external bus becoming a bottleneck for L2 cache access.	Create separate back-side bus that runs at higher speed than the main (front-side) external bus. The BSB is dedicated to the L2 cache.	Pentium Pro
	Move L2 cache on to the processor chip.	Pentium II
Some applications deal with massive databases and must have rapid access to large amounts of data. The on-chip caches are too small.	Add external L3 cache.	Pentium III
	Move L3 cache on-chip.	Pentium 4

VIRTUAL MEMORY:

What is Virtual Memory?

- Memory as a “cache” for secondary storage
- Motivations
  - **efficient and safe sharing of memory among multiple programs** and
  - **remove the programming burdens** of a small, limited amount of main memory.
- Key terms
  - Page
  - Page fault
  - Virtual address
  - Address mapping (translation)
  - Relocation

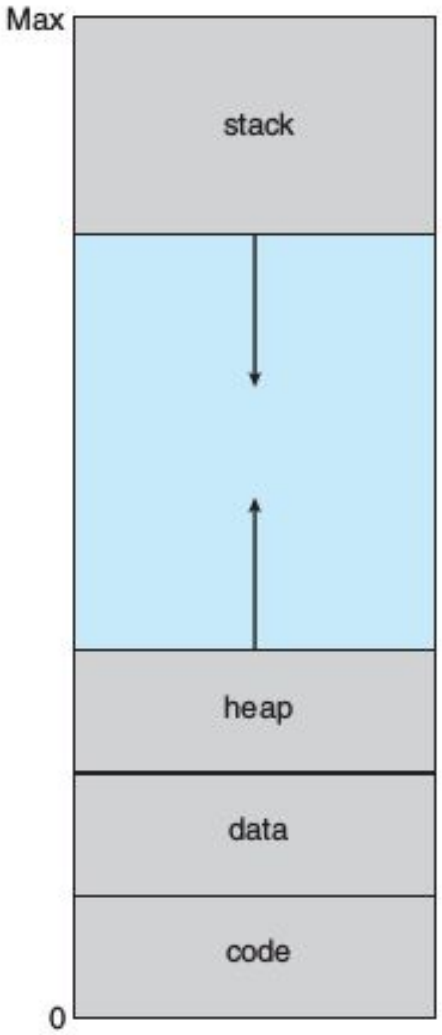




Virtual Memory Basics

- **Each program has its own large but limited virtual address space (logical address space)**
  - Size chosen such that address space provided exceeds the demands of any application likely to be run on the system
  - In the past, 32-bit virtual addresses were common → 4GB virtual address space
  - Applications have gotten larger recently: 48-bit virtual address → 256TB of virtual space
  - 64-bit → currently unimaginable 16EB
  - The aim is to give the programmer (and compiler) the illusion of a huge main memory exclusively "owned" by his or her program
- **Virtual address translated to physical addresses before any actual memory access**
  - Each programs has its own large virtual address space
  - However, in a multitasking system, several programs have to share the same, smaller physical memory, which uses its own addresses
  - Therefore, virtual addresses must be translated to physical addresses
  - Translation done in hardware (memory management unit) and software (OS)

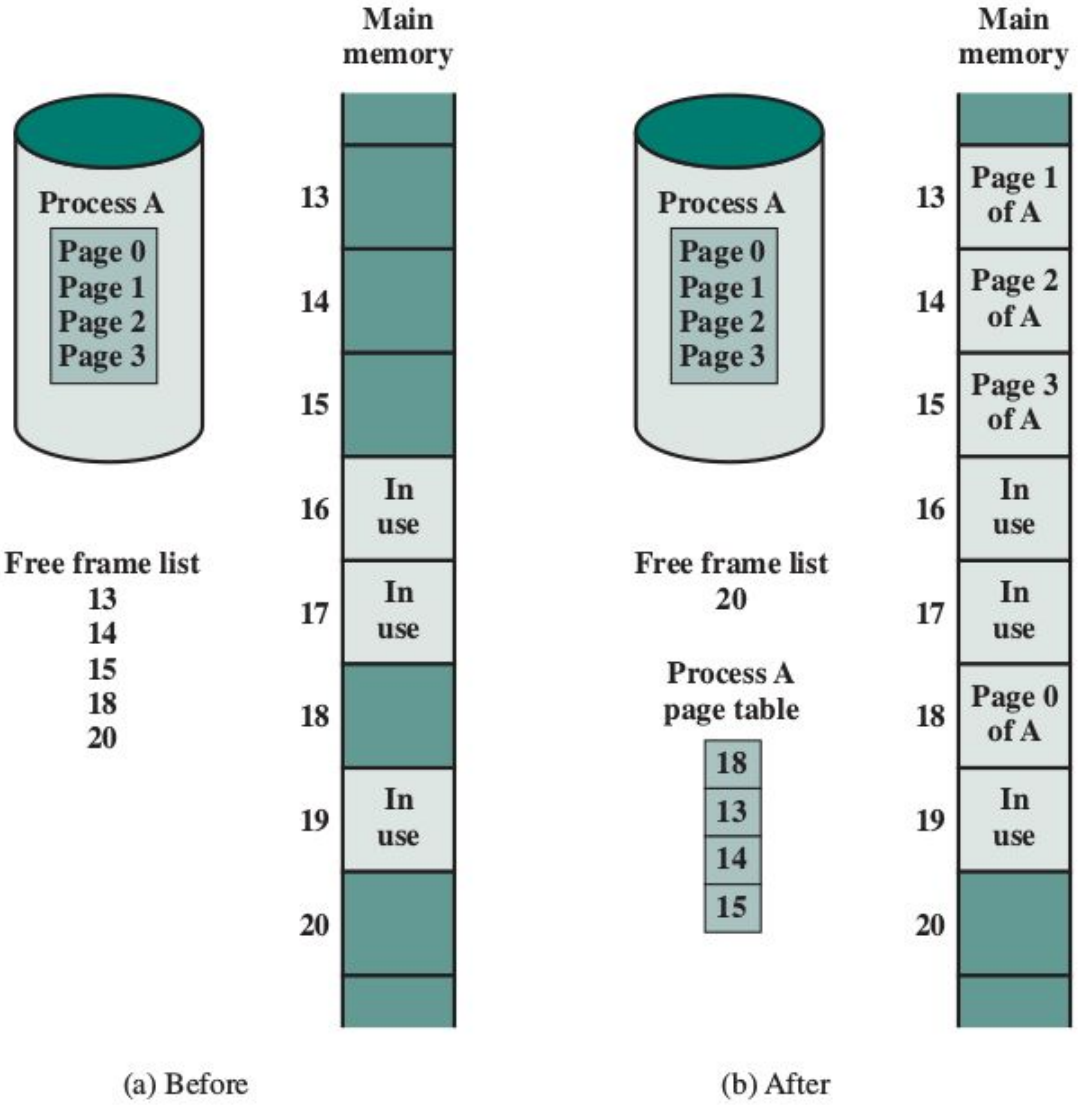
Virtual Address Space



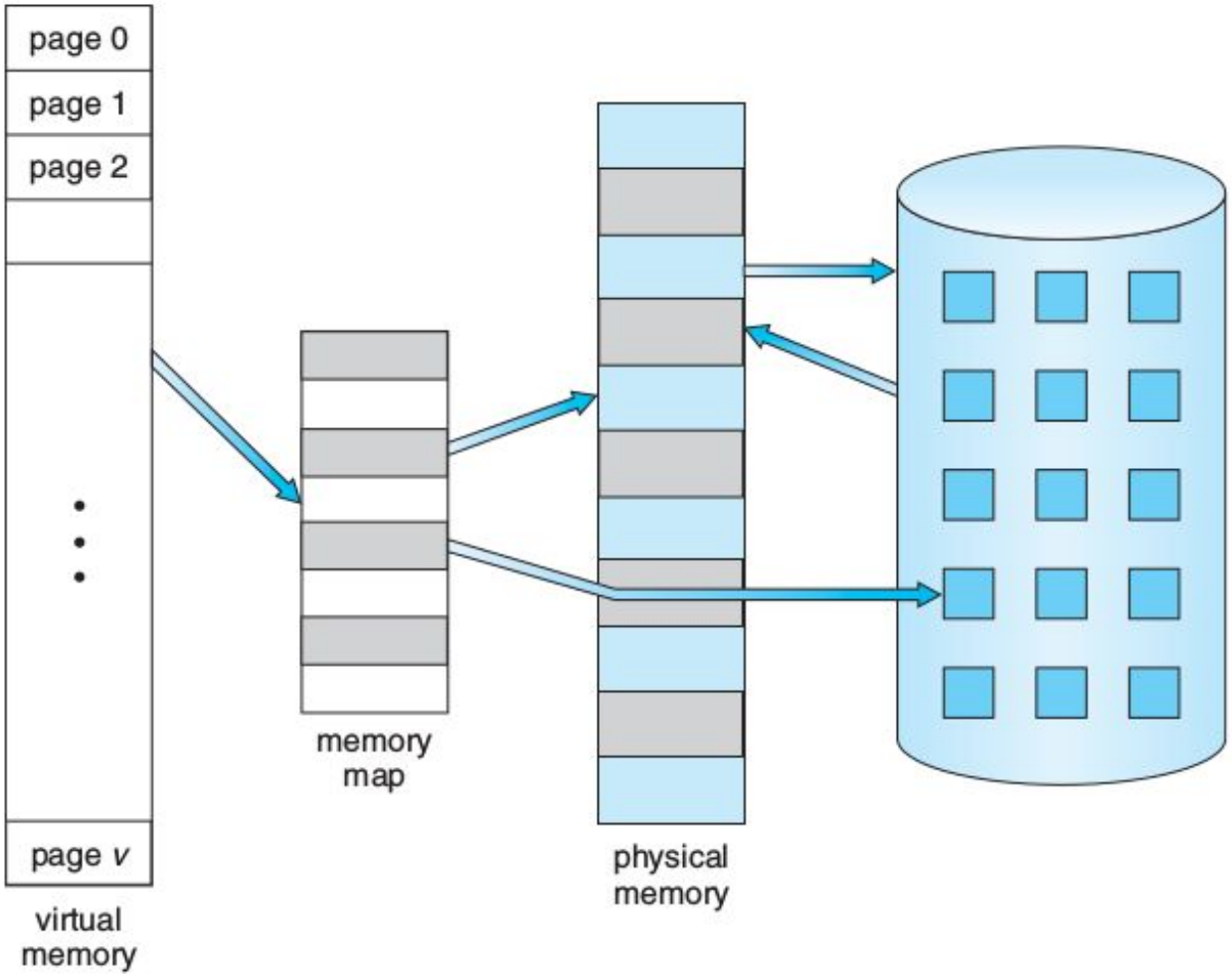
Paged Virtual Memory (aka demand-paged)

- Page is a block of memory moved between main and secondary memory
- **Fixed page sizes**
  - Different page sizes across different architectures and OS (e.g 512B – 16KB)
  - Some modern systems allow huge pages, e.g several MB
- **Paged virtual memory address fields**
  - Virtual page number
  - Page offset (depends on page size e.g 4KB (2^{12}), 12 LSB for offset)
  - Physical page offset always same as the virtual page offset

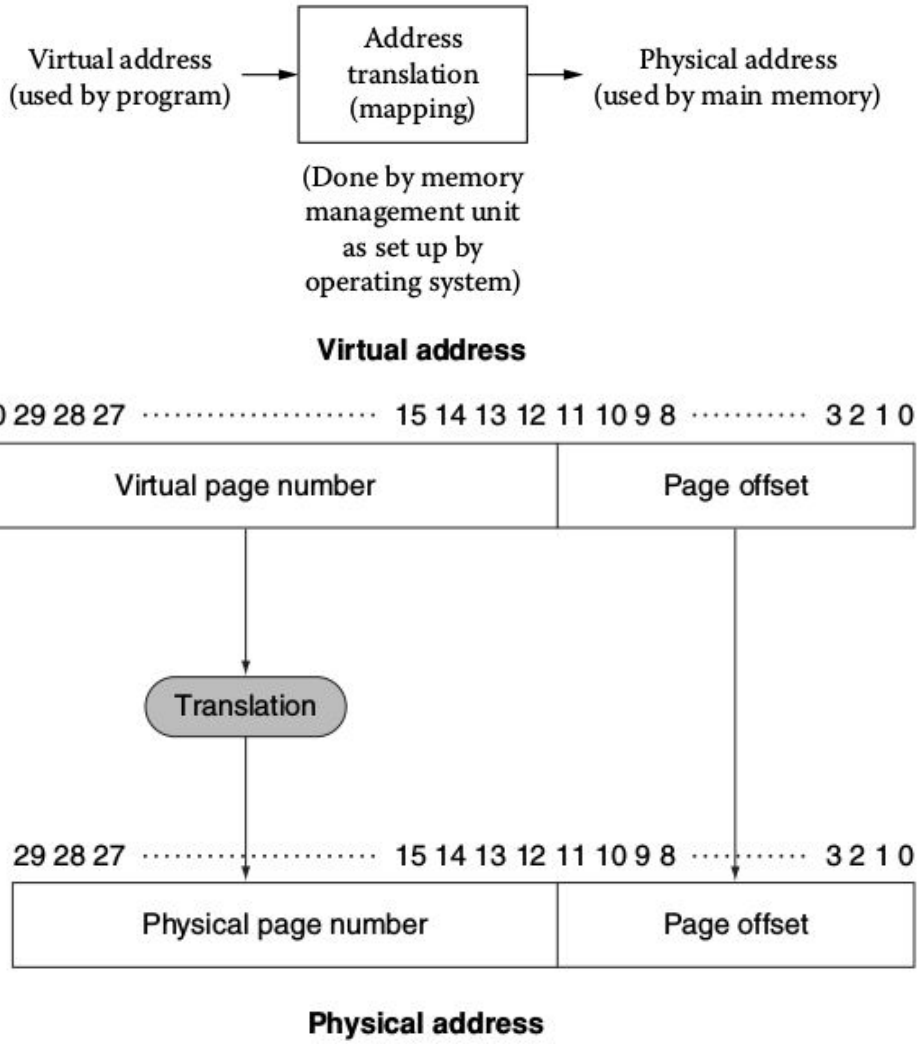
Paged Virtual Memory: Frame Allocation



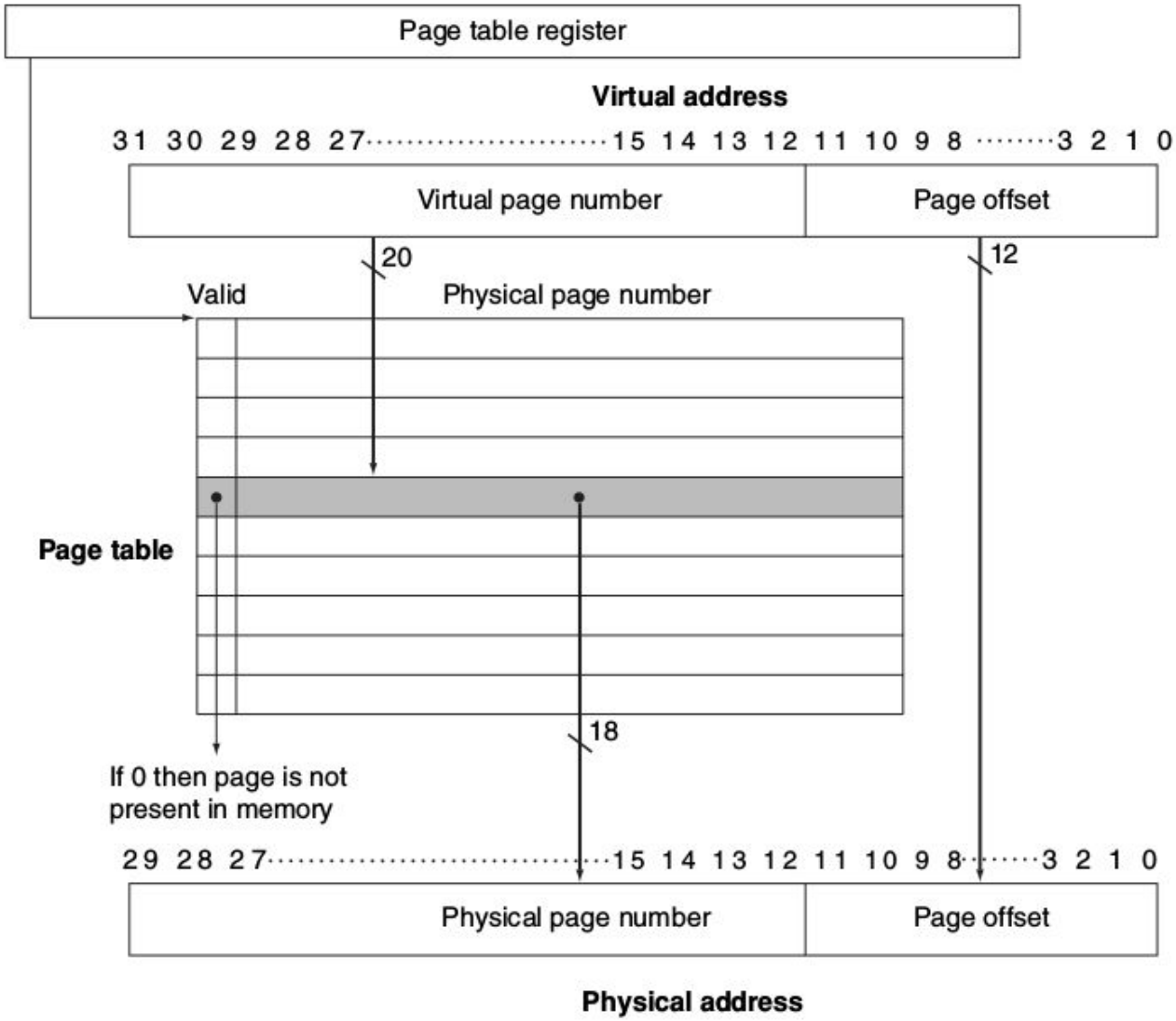
Paged Virtual Memory (aka demand-paged)



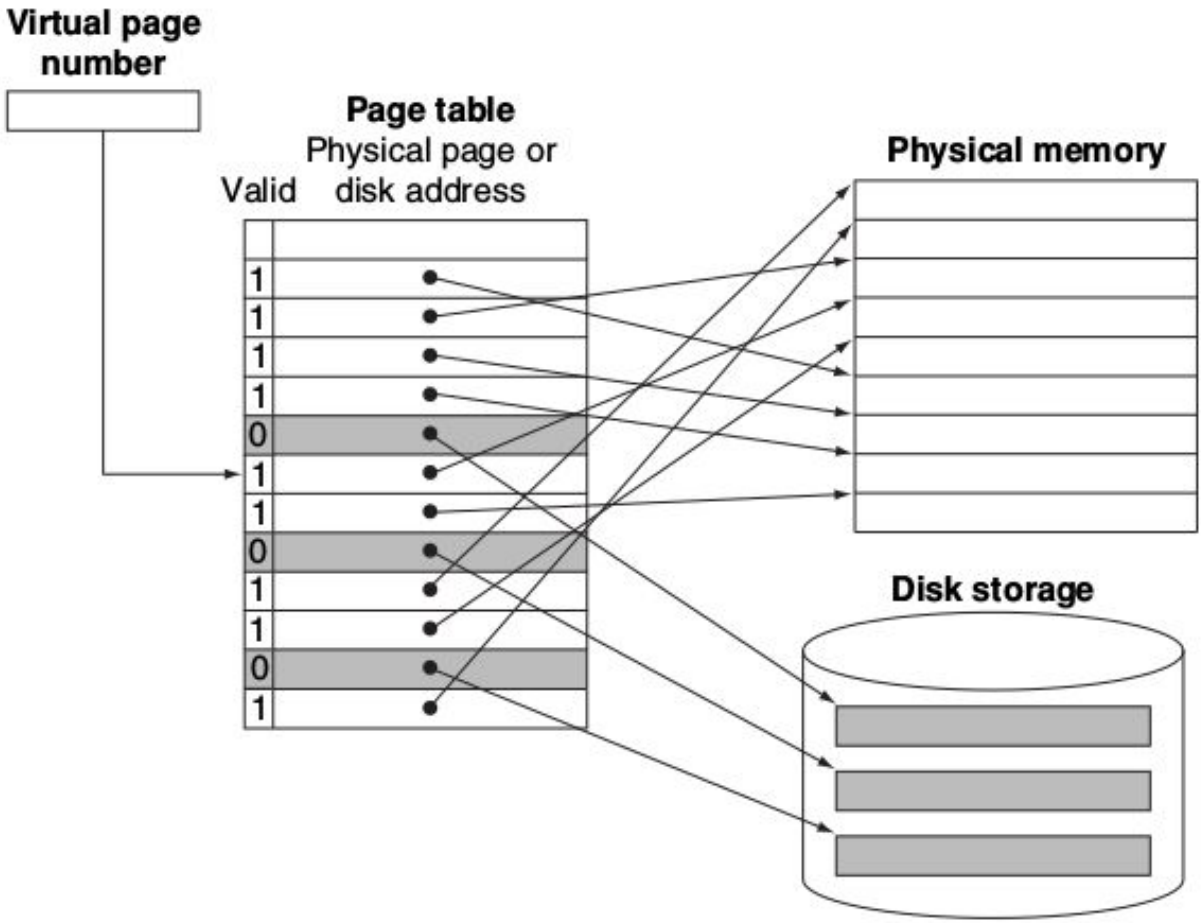
Paged Virtual Memory Address Translation



Paged Virtual Memory Address Translation

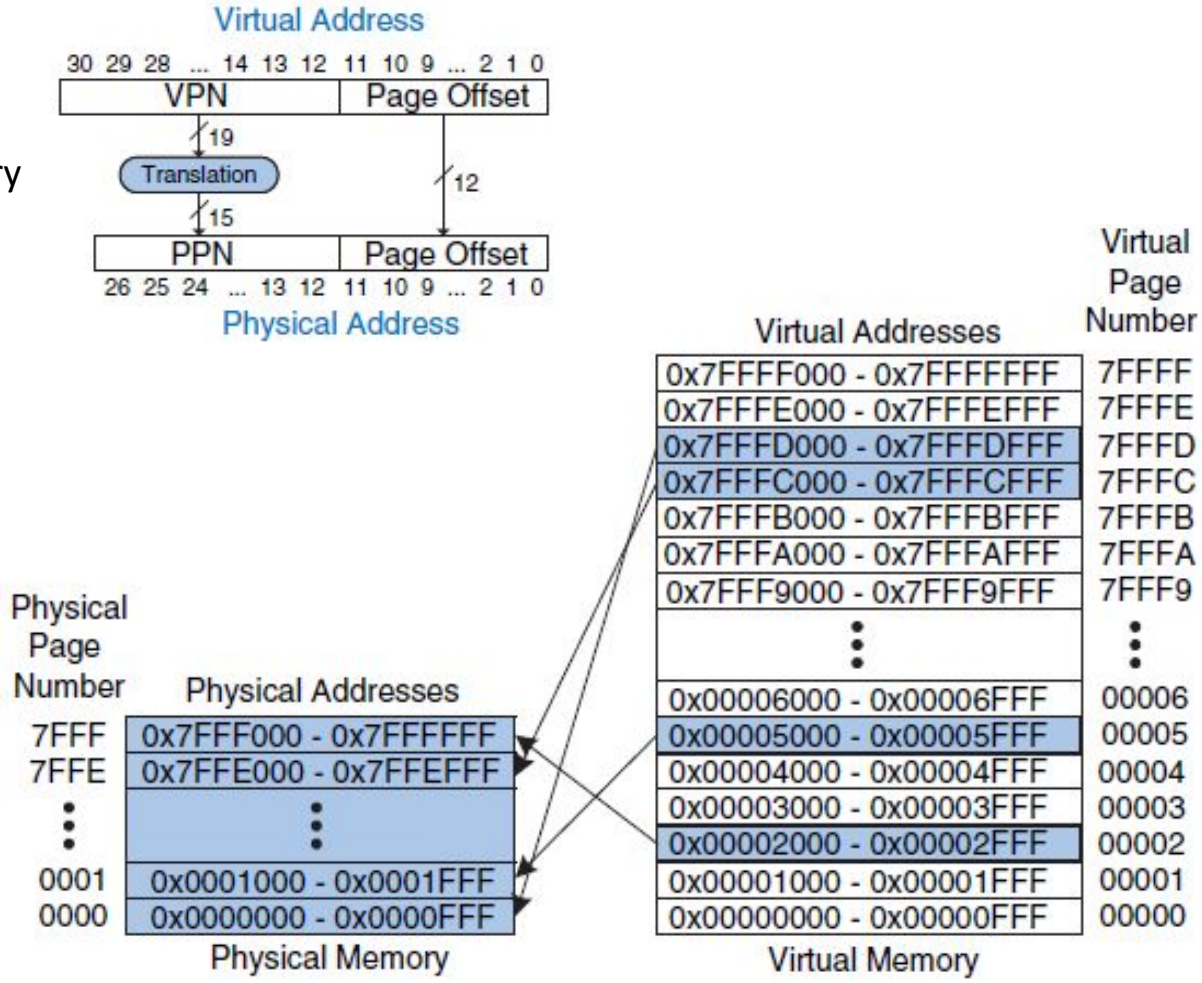


Paged Virtual Memory Address Translation



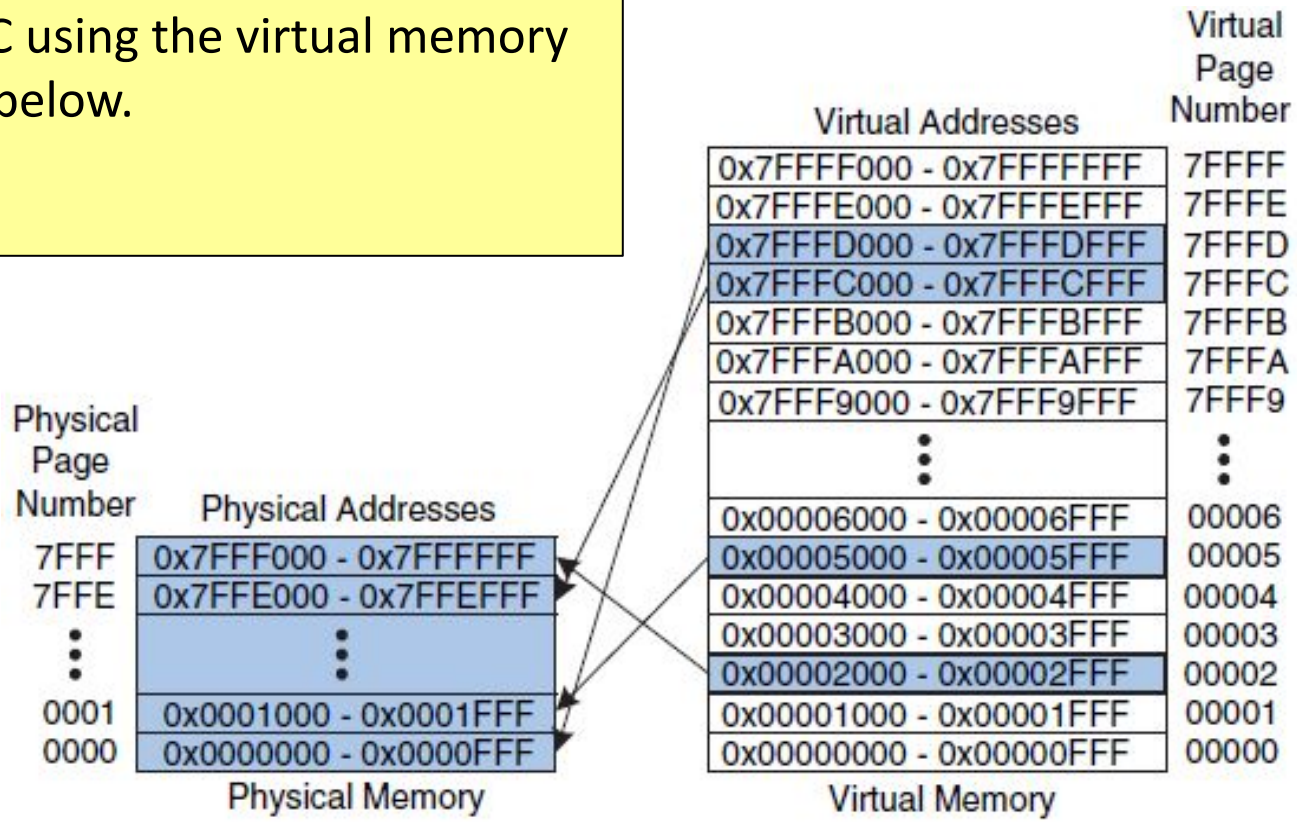
Paged Virtual Memory Address Translation

- Organisation
  - 2GB virtual memory
  - 128MB physical memory
  - 4KB page size

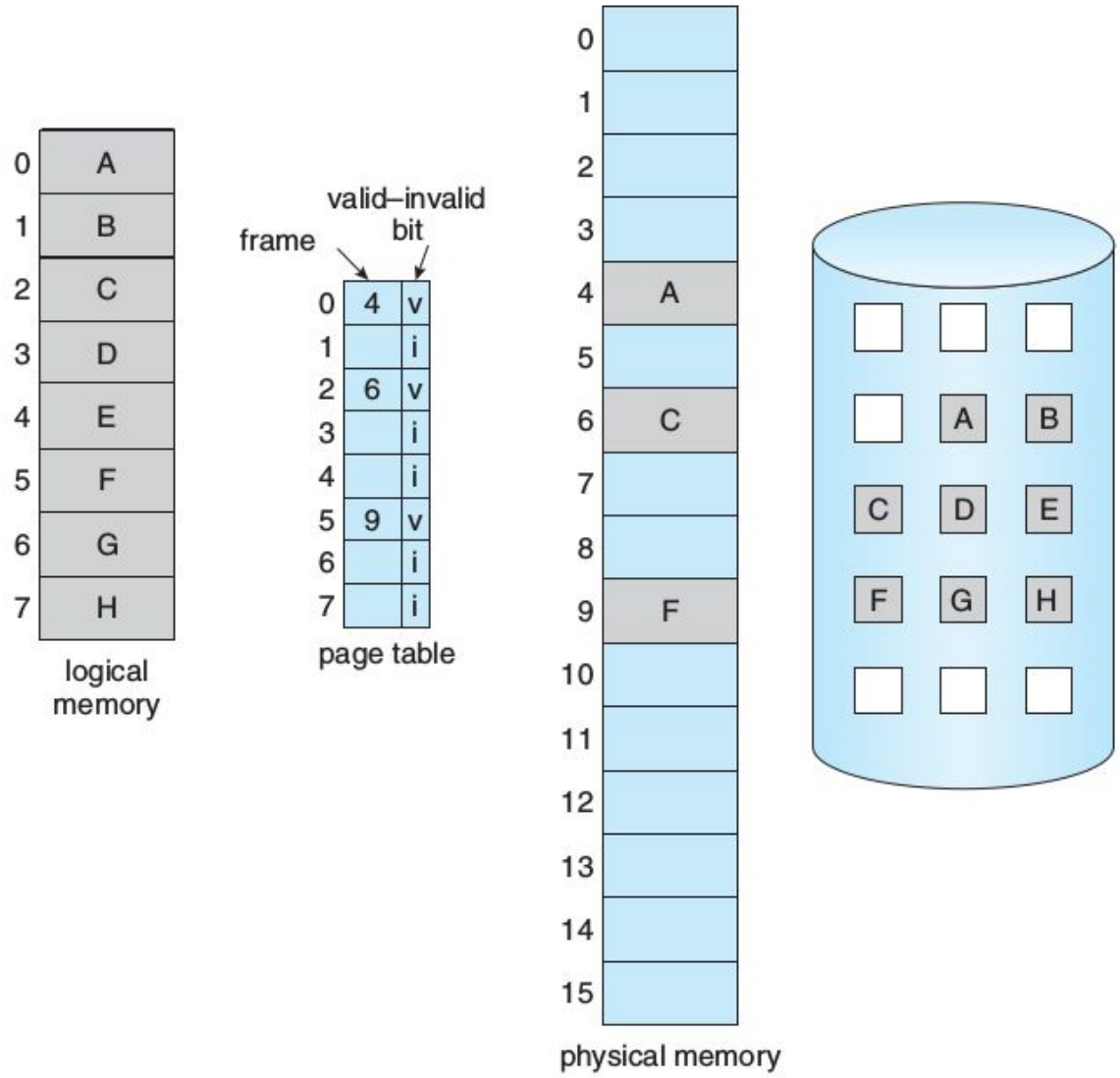


Exercise: Paged Virtual Memory Address Translation

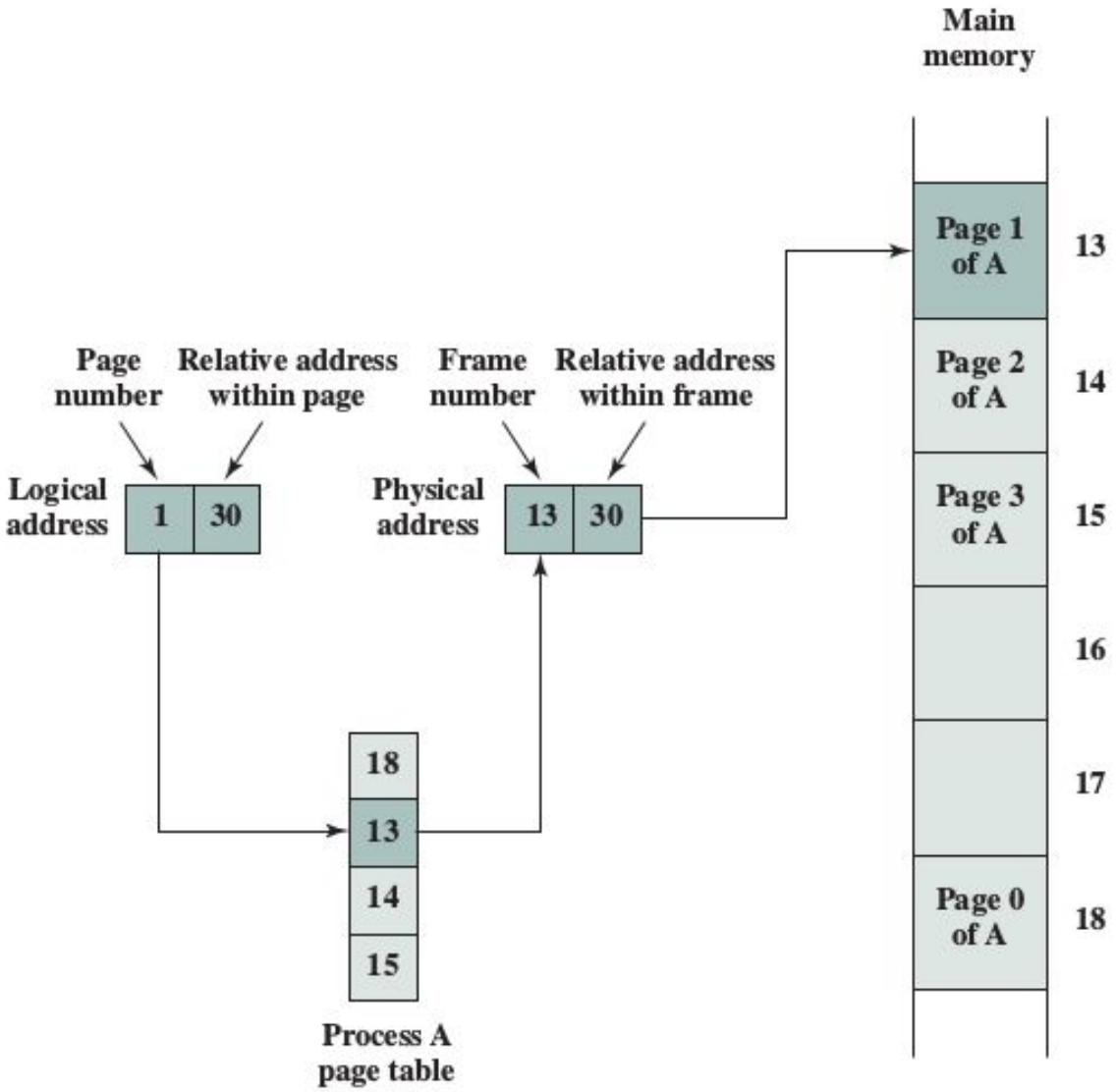
Q: Find the physical address of virtual address 0x247C using the virtual memory system shown below.



Paged Virtual Memory Address Translation: Page Table



Paged Virtual Memory Address Translation: Page Table



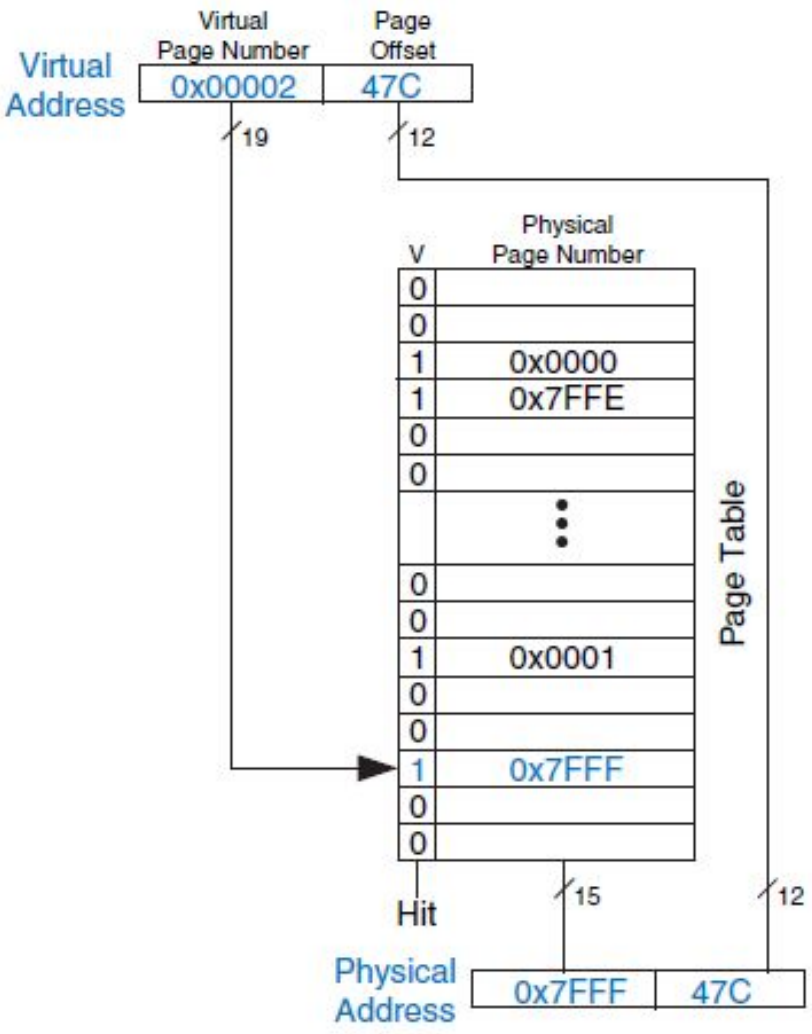
Exercise: Paged Virtual Memory Address Translation

Q: Find the physical address of virtual address 0x247C using the page table shown on the left.

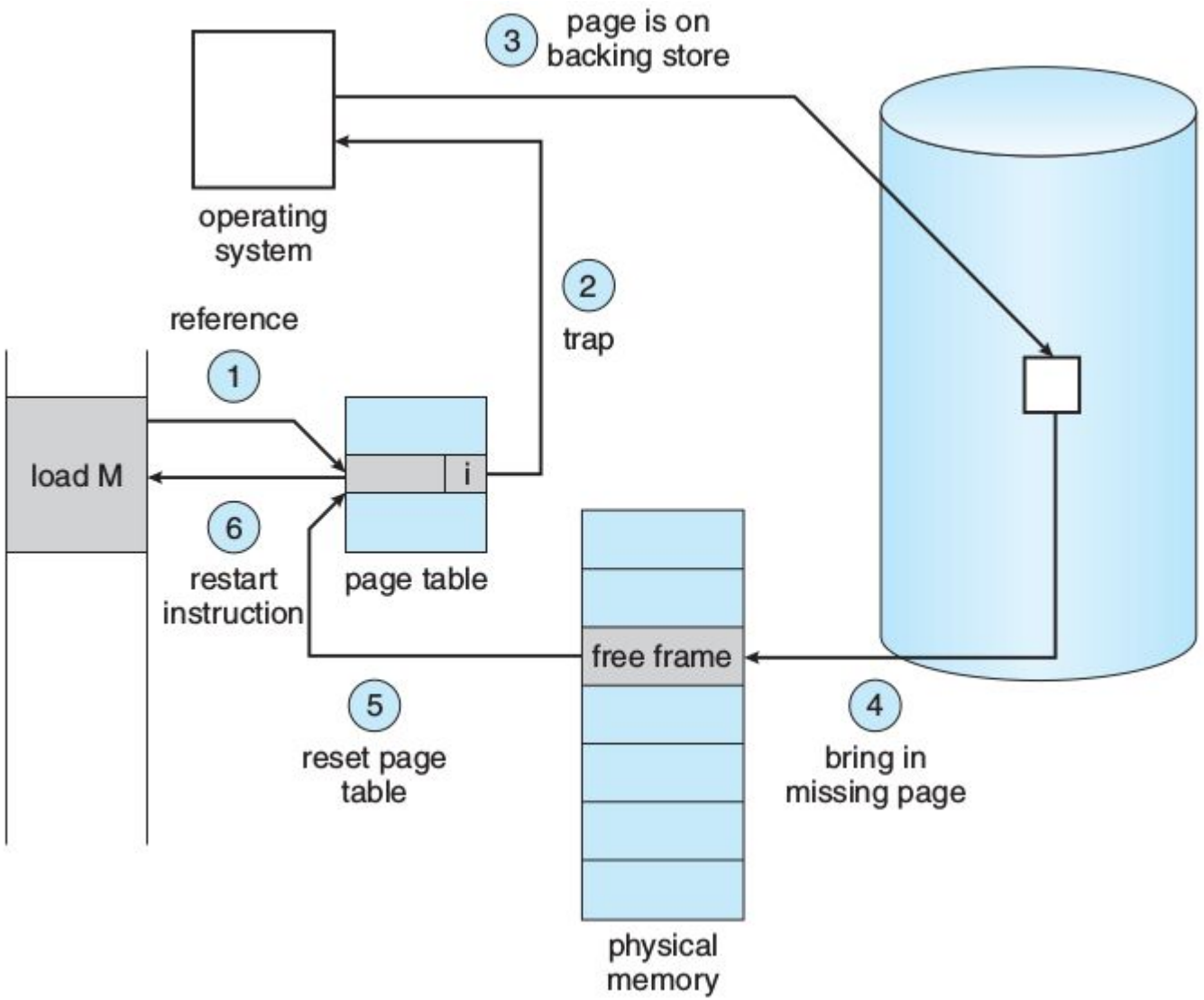
V	Physical Page Number	Virtual Page Number
0		7FFFF
0		7FFFE
1	0x0000	7FFFD
1	0x7FFE	7FFFC
0		7FFFB
0		7FFFA
	⋮	⋮
0		00007
0		00006
1	0x0001	00005
0		00004
0		00003
1	0x7FFF	00002
0		00001
0		00000

Page Table

Solution: Paged Virtual Memory Address Translation



Paged Virtual Memory: Handling Page Faults



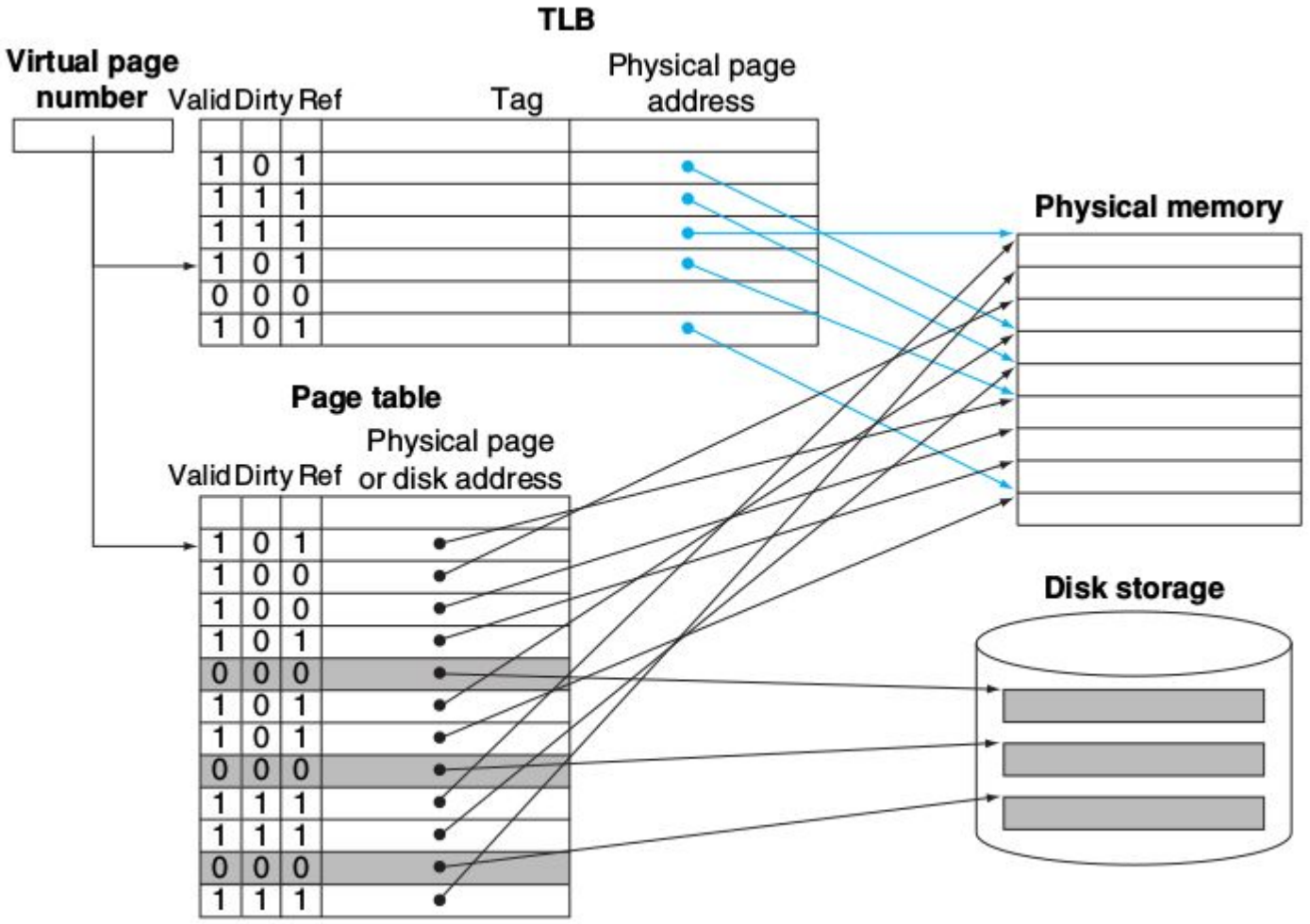
What about writes?

- **Write-through could be OK between cache and main memory**
  - The difference between the access time to the cache and main memory is tens to hundreds of cycles, and write-through schemes can be used
  - Although we need a write buffer to hide the latency of the write from the processor.
- **Write-through impractical for virtual memory**
  - In a virtual memory system, writes to the next level of the hierarchy (disk) can take millions of processor clock cycles;
  - Therefore, building a write buffer to allow the system to write-through to disk would be completely impractical.
- **Use write-back instead**
  - Instead, virtual memory systems must use write-back,
  - Performing the individual writes into the page in memory, and copying the page back to disk when it is replaced in the memory.

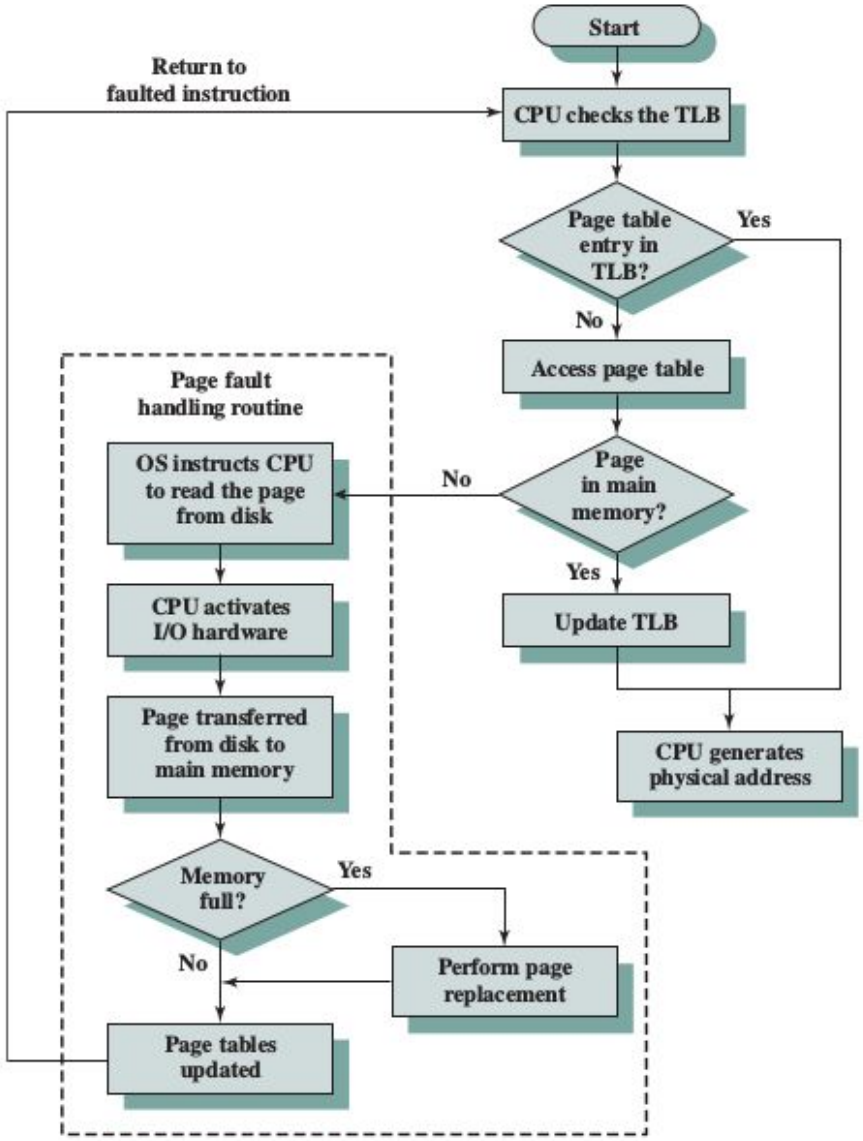
The Cost of Virtual Memory: Virtual Address Translation Slows Performance

- **The page table can be stored anywhere in physical memory, at the discretion of the OS.**
  - The processor typically uses a dedicated register, called the page table register, to store the base address of the page table in physical memory.
- **To perform a load or store:**
  - the processor must first translate the virtual address to a physical address and then access the data at that physical address.
  - The processor extracts the virtual page number from the virtual address and adds it to the page table register to find the physical address of the page table entry.
  - The processor then reads this page table entry from physical memory to obtain the physical page number. If the entry is valid, it merges this physical page number with the page offset to create the physical address.
  - Finally, it reads or writes data at this physical address.
- **Because the page table is stored in physical memory, each load or store involves two physical memory accesses!**

Faster Address Translation: Translation Look-aside Buffer (TLB)



Faster Address Translation: Paging and TLB

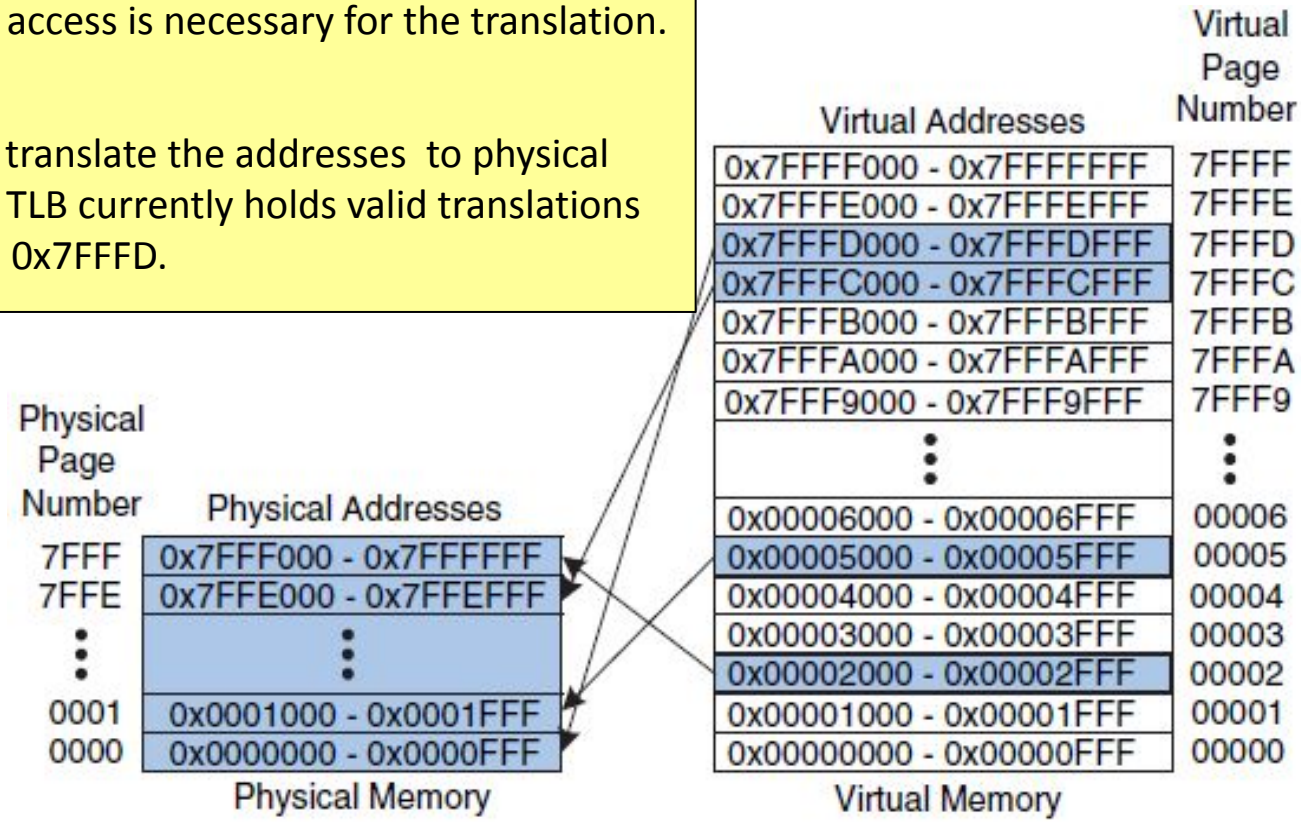


Exercise: Paged Virtual Memory Address Translation

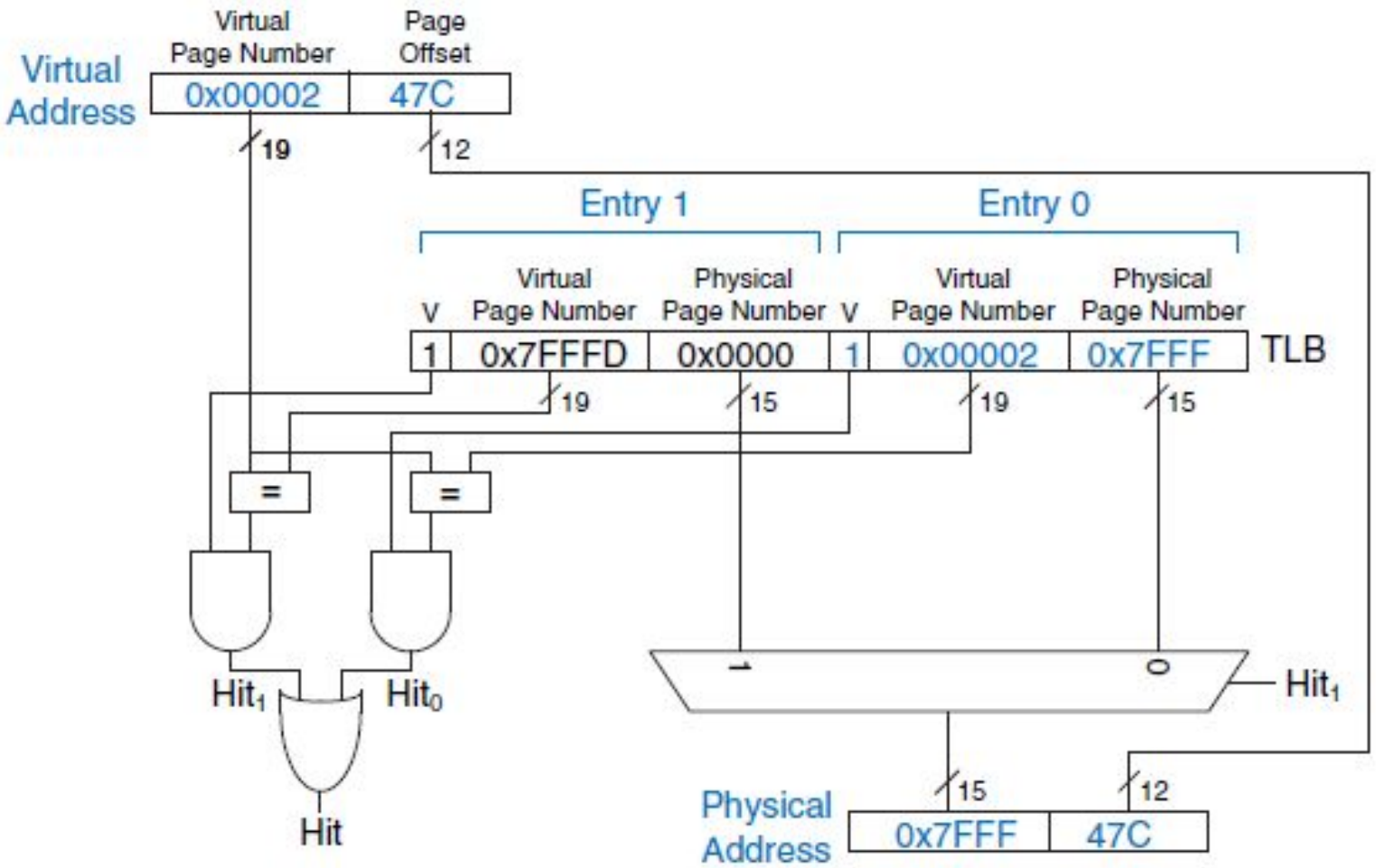
Q: Consider the virtual memory system of shown on the left. Given virtual addresses 0x247C and 0x5FB0

(1) Explain why page table access is necessary for the translation. If it is.

(2) Use a two-entry TLB to translate the addresses to physical addresses. Suppose the TLB currently holds valid translations of virtual pages 0x2 and 0x7FFFD.



Solution: Address Translation with Two-entry TLB



Replacement Policies

- First in first out (FIFO)
- Least recently used (LRU)
- Least frequently used (LFU)
- Most frequently used (MFU)

